

GPIO 입출력 제어

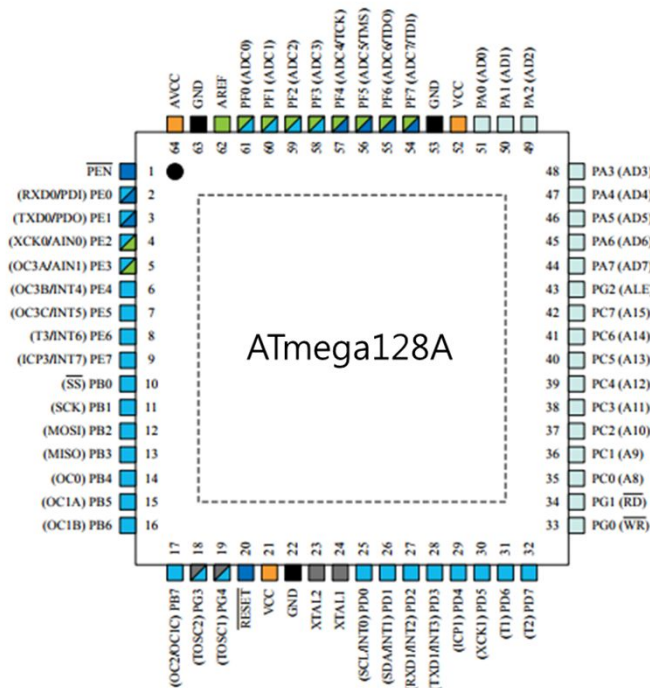
- 마이크로 컨트롤러와 GPIO
- AVR 마이크로 컨트롤러의 입출력 포트
- 입출력포트 제어용 레지스터 익히기
- 예제 및 응용 실습(LED, SWITCH, FND)



엣지아이랩

마이크로 컨트롤러와 GPIO

- GPIO(General Purpose Input Output)
 - 범용으로 사용되는 입출력 포트
 - 설계자가 마음대로 변형하면서 제어할 수 있도록 제공해주는 입출력 포트
 - 입력과 출력을 선택할 수 있고, 0과 1의 출력 신호를 임의로 만들수 있는 구조
 - 입력으로 사용할 때는 외부 인터럽트를 처리할 수 있도록 하는 경우가 많음
 - 입출력 방향 전환용 레지스터와 출력용/입력용 데이터 레지스터등이 필요
 - 마이크로 컨트롤러에서는 대부분의 핀들을 GPIO로 설정하는 경우가 많음



ATmega128A Pin Out

AVR 마이크로 컨트롤러의 입출력 포트

- AVR 마이크로 컨트롤러 입출력 포트
 - 6개의 8비트 I/O포트와 1개의 5비트 I/O포트로 구성
 - 출력 포트의 버퍼는 많은 유입 전류와 유출 전류(최대 40mA)를 사용할 수 있음
 - 모든 포트 핀은 개별적으로 내부 풀업 저항을 사용할 수 있음
 - 모든 I/O핀은 VCC와 GND사이에 다이오드를 접속하여 포트를 보호
 - Read-Modify-Write기능을 가지고 있어, 비트 단위의 포트 설정이 가능
 - 각 포트에 대한 데이터 출력용 레지스터(PORTx)와 데이터 입출력 방향 지정용 레지스터(Data Direction Register: DDRx), 그리고 데이터 입력용 레지스터(PINx)를 보유

AVR 마이크로 컨트롤러의 입출력 포트

- 입출력 포트 제어용 레지스터
 - DDRx 레지스터
 - 입출력의 방향설정을 하기 위한 레지스터
 - DDRA~DDRG 레지스터의 해당 비트에 '1'을 쓰면 출력, '0'을 쓰면 입력으로 설정
 - PORTx 레지스터
 - 데이터 출력용 레지스터
 - 출력을 원하는 데이터 값을 PORTx 레지스터에 넣어줌
 - PINx 레지스터
 - 데이터 입력용 레지스터
 - PINx 레지스터에 해당하는 값을 읽으면 해당 핀의 값이 읽음
 - SFIOR 레지스터
 - Special Function IO Register
 - AVR 입출력 포트의 특수 기능을 제어하기 위한 레지스터
 - SFIOR의 비트2(PUD: Pull-Up Disable)를 '1'로 세트하면 풀업 저항을 비활성화

AVR 마이크로 컨트롤러의 입출력 포트

- ATmega128A의 범용 입출력 포트 : A 포트(PA7~PA0)
 - 내부 풀업 저항이 있는 8비트 양방향 입출력 단자
 - 외부 메모리를 둘 경우에는 주소버스(A7-A0)와 데이터버스(D7-D0)로 사용

포트 핀	부가 기능
PA7	AD7(외부 메모리 인터페이스 주소와 데이터 비트7)
PA6	AD6(외부 메모리 인터페이스 주소와 데이터 비트6)
PA5	AD5(외부 메모리 인터페이스 주소와 데이터 비트5)
PA4	AD4(외부 메모리 인터페이스 주소와 데이터 비트4)
PA3	AD3(외부 메모리 인터페이스 주소와 데이터 비트3)
PA2	AD2(외부 메모리 인터페이스 주소와 데이터 비트2)
PA1	AD1(외부 메모리 인터페이스 주소와 데이터 비트1)
PA0	AD0(외부 메모리 인터페이스 주소와 데이터 비트0)

AVR 마이크로 컨트롤러의 입출력 포트

- ATmega128A의 범용 입출력 포트 : B 포트(PB7~PB0)
 - 내부 풀업 저항이 있는 8비트 양방향 입출력 단자
 - 타이머/카운터나 **SPI**용 혹은 **PWM** 단자로도 사용

포트 핀	부가 기능
PB7	OC2/OC1C(출력비교 또는 타이머/카운터2의 PWM출력, 또는 출력 비교와 타이머/카운터1의 PWM출력 C)
PB6	OC1B(출력비교 또는 타이머/카운터1의 PWM출력 B)
PB5	OC1A(출력비교 또는 타이머/카운터1의 PWM출력 A)
PB4	OC0(출력비교 또는 타이머/카운터0의 PWM출력)
PB3	MISO(SPI 버스 마스터 입력/종속 출력)
PB2	MOSI(SPI 버스 마스터 출력/종속 입력)
PB1	SCK(SPI 버스 직렬 클럭)
PB0	/SS(SPI 종속 선택 입력)

AVR 마이크로 컨트롤러의 입출력 포트

- ATmega128A의 범용 입출력 포트 : C 포트(PC7~PC0)
 - 내부 풀업 저항이 있는 8비트 양방향 입출력 단자
 - 외부 메모리를 둘 경우에는 주소버스(A15-A8)로 사용

포트 핀	부가기능
PC7	AD7(외부 메모리 인터페이스 주소 비트15)
PC6	AD6(외부 메모리 인터페이스 주소 비트14)
PC5	AD5(외부 메모리 인터페이스 주소 비트13)
PC4	AD4(외부 메모리 인터페이스 주소 비트12)
PC3	AD3(외부 메모리 인터페이스 주소 비트11)
PC2	AD2(외부 메모리 인터페이스 주소 비트10)
PC1	AD1(외부 메모리 인터페이스 주소 비트 9)
PC0	AD0(외부 메모리 인터페이스 주소 비트8)

AVR 마이크로 컨트롤러의 입출력 포트

- ATmega128A의 범용 입출력 포트 : D 포트(PD7~PD0)
 - 내부 풀업 저항이 있는 8비트 양방향 입출력 단자
 - 타이머/카운터나 외부 인터럽트용, I2C용 단자로도 사용

포트 핀	부가기능
PD7	T2(타이머/카운터2 클럭 입력)
PD6	T1(타이머/카운터1 클럭 입력)
PD5	XCK1(USART1 외부 클럭 입/출력)
PD4	IC1(타이머/카운터1 입력 캡처 트리거)
PD3	INT3/TXD1(외부 인터럽트3 입력 또는 USART1 전송 핀)
PD2	INT2/RXD1(외부 인터럽트2 입력 또는 USART1 수신 핀)
PD1	INT1/SDA(외부 인터럽트1 입력 또는 TWI 직렬 데이터)
PD0	INT0/SCL(외부 인터럽트0 입력 또는 TWI 직렬 클럭)

AVR 마이크로 컨트롤러의 입출력 포트

- ATmega128A의 범용 입출력 포트 : E 포트(PE7~PE0)
 - 내부 풀업 저항이 있는 8비트 양방향 입출력 단자
 - 타이머/카운터나 외부 인터럽트, 아날로그 비교기, USART용 단자로도 사용

포트핀	부가기능
PE7	INT7/IC3(외부 인터럽트 7 입력 또는 타이머/카운터3 입력 캡처 트리거)
PE6	INT6/T3 (외부 인터럽트 6 입력 또는 타이머/카운터3 클럭입력)
PE5	INT5/OC3C(외부 인터럽트 5 입력 또는 타이머/카운터3의 출력 캡처와 PWM 출력 C)
PE4	INT4/OC3B(외부 인터럽트 4 입력 또는 타이머/카운터3의 출력 캡처와 PWM 출력 B)
PE3	AIN1/OC3A(아날로그 비교 반대입력 또는 타이머/카운터3의 출력 비교와 PWM 출력A)
PE2	AIN0/XCK0(아날로그 비교 입력 또는 USART0 외부 클럭 입/출력)
PE1	PDO/TXD0(프로그램 데이터 출력 또는 UART0 전송 핀)
PE0	PDI/RXD0(프로그램 데이터 입력 또는 UART0 수신 핀)

AVR 마이크로 컨트롤러의 입출력 포트

- ATmega128A의 범용 입출력 포트 : F 포트(PF7~PF0)
 - 내부 풀업 저항이 있는 8비트 양방향 입출력 단자
 - **AD변환기(ADC)** 혹은 **JTAG 인터페이스용** 단자로도 사용

포트 핀	부가기능
PF7	ADC7/TDI(ADC 입력 채널 7 또는 JTAG Test Data Input)
PF6	ADC6/TDO(ADC 입력 채널 6 또는 JTAG Test Data Output)
PF5	ADC5/TMS(ADC 입력 채널 5 또는 JTAG Test Mode Select)
PF4	ADC4/TCK(ADC 입력 채널 4 또는 JTAG Test Clock)
PF3	ADC3 (ADC 입력 채널 3)
PF2	ADC2 (ADC 입력 채널 2)
PF1	ADC1 (ADC 입력 채널 1)
PF0	ADC0 (ADC 입력 채널 0)

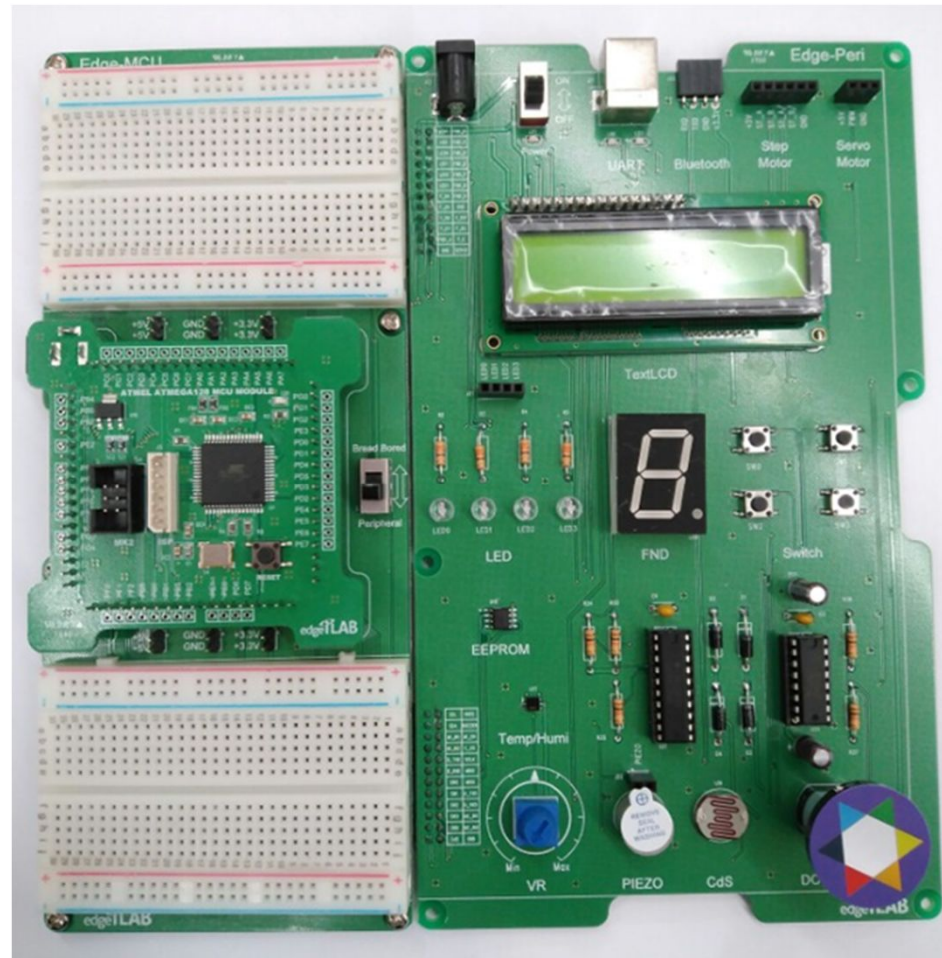
AVR 마이크로 컨트롤러의 입출력 포트

- ATmega128A의 범용 입출력 포트 : G 포트(PG4~PG0)
 - 내부 풀업 저항이 있는 5비트 양방향 입출력 단자
 - 외부 메모리 접속을 위한 스트로브 신호용, RTC(Real Time Counter) 타이머용 발진기 단자로도 사용

포트 핀	부가기능
PG4	TOSC1(RTC 오실레이터 타이머/카운터0)
PG3	TOSC2(RTC 오실레이터 타이머/카운터0)
PG2	ALE(외부메모리에 주소 래치 인에이블)
PG1	RD(외부메모리에 스트로브 읽기)
PG0	WR(외부메모리에 스트로브 쓰기)

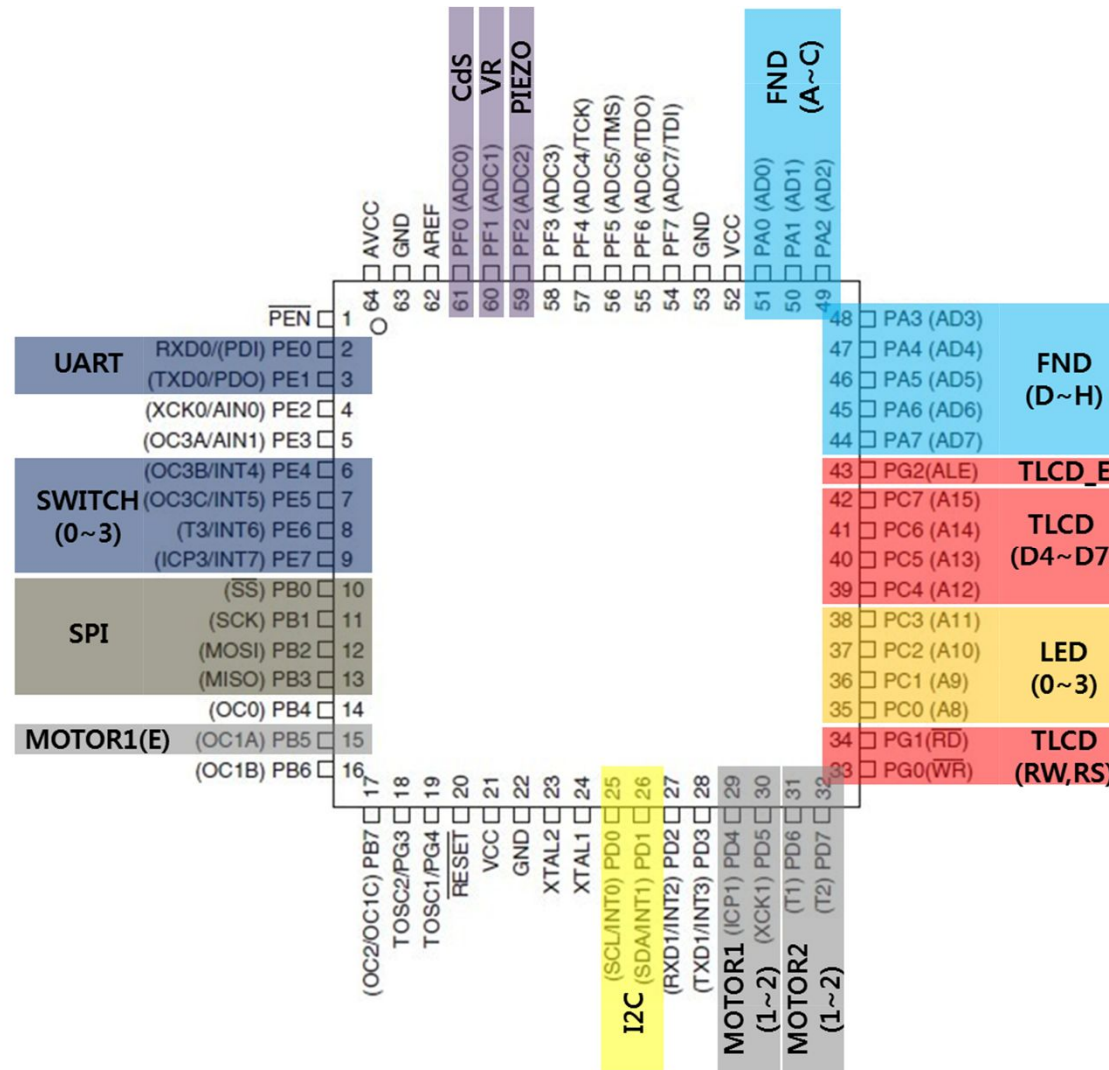
AVR 마이크로 컨트롤러의 입출력 포트

- Edge-MCU AVR
 - Edge-Peri는 센서 및 액추레이터가 연결되어 있음
 - Main Board와 Edge-Peri 결합시 별도의 연결없이 디바이스 사용 가능



AVR 마이크로 컨트롤러의 입출력 포트

- Edge-MCU AVR
 - 센서 및 액추레이터는 GPIO 핀과 1:1 매칭 또는 I2C, SPI 통신으로 연결



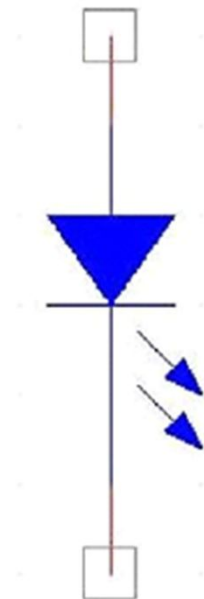
1)예제 : 입출력포트 제어용 레지스터 익히기

- 실습 개요
 - ATmega128A 마이크로컨트롤러의 입출력포트 제어용 레지스터 익히기
 - ATmega128A 마이크로컨트롤러의 GPIO를 이용하여 LED On/Off
 - 입출력 포트를 출력으로 설정하고, 그 포트를 이용하여 LED에 신호를 보내 점등
- 실습 목표
 - GPIO 입출력 포트의 방향 제어 및 출력 제어 방법 습득
 - LED 동작 원리 습득
 - 프로그램에서 시간지연 방법 습득

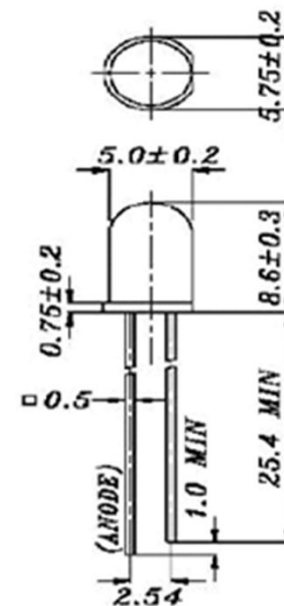
1)예제 : 입출력포트 제어용 레지스터 익히기

- LED 구조

- LED(Light-emitting diode) : 빛을 발산하는 반도체 소자(발광 다이오드)
- 순방향에 전류를 흘리는 것에 따라 전자와 정공이 재결합하여 발광
- 다리가 긴 부분이 양극 (Anode), 짧은 부분이 음극 (Cathode)



심볼



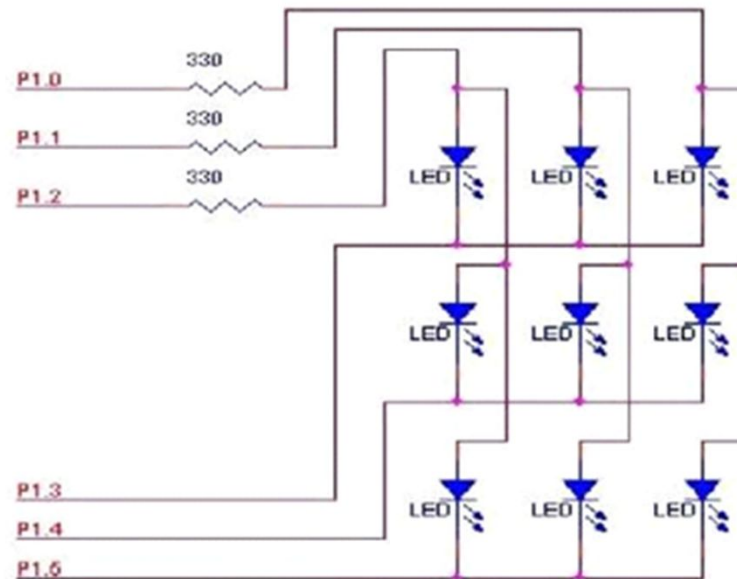
패키지 형상

1)예제 : 입출력포트 제어용 레지스터 익히기

- LED 구동방법
 - 정적 구동방식 : 각각의 LED를 독립해서 구동
 - 동적 구동방식 : 여러 개의 LED를 매트릭스 구조로 엮어서 함께 구동



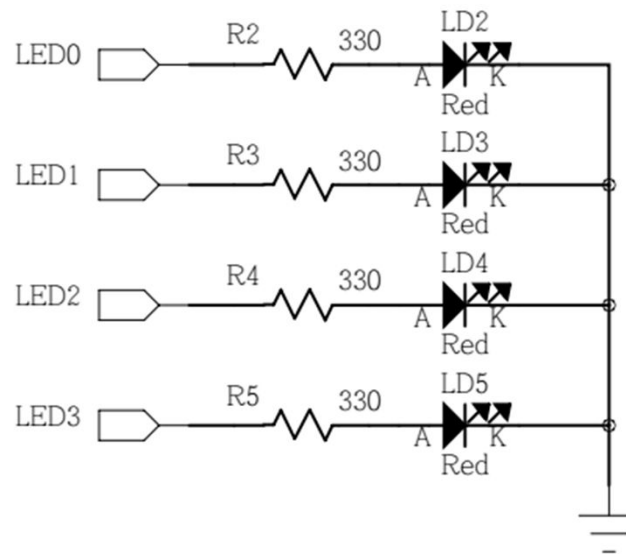
정적 구동



동적 구동

1)예제 : 입출력포트 제어용 레지스터 익히기

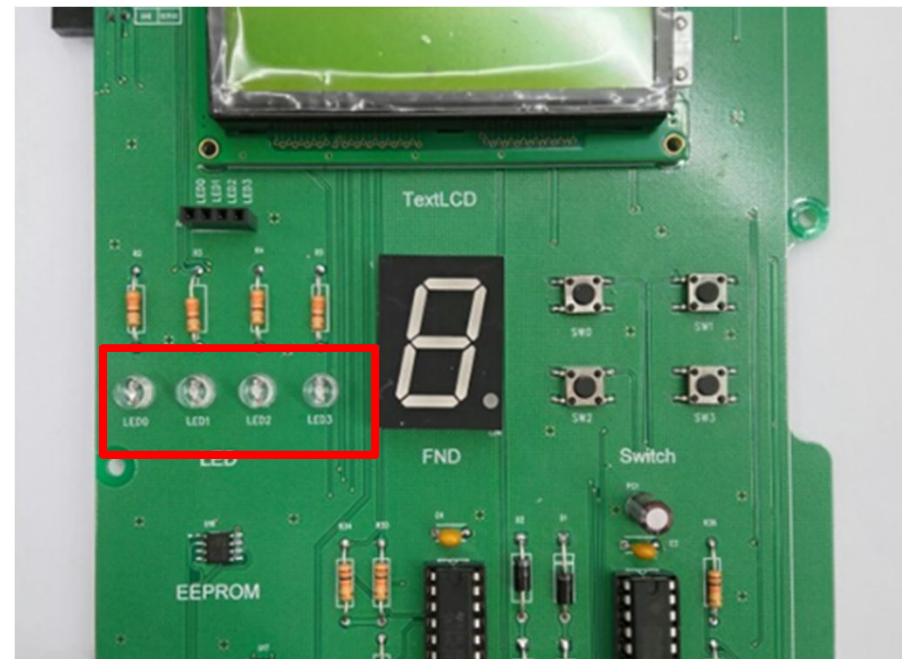
- 사용 모듈의 회로
 - LED 모듈



1)예제 : 입출력포트 제어용 레지스터 익히기

- Edge-MCU AVR
 - LED는 C 포트에 연결

Sensor	GPIO
LED0	PC0
LED1	PC1
LED2	PC2
LED3	PC3



1)예제 : 입출력포트 제어용 레지스터 익히기

- 구동 프로그램 : 사전 지식
 - LED를 점등하기 위해서는 LED 신호에 '1'을 인가해야 함
 - 즉, MCU C포트에서 '1'을 출력하도록 해야 함
 - MCU C 포트에 '1'을 출력하기 위해선
 - 입출력 포트 C의 GPIO 방향을 출력으로 설정
 - 입출력 포트를 출력으로 선언하려면 DDRx 레지스터 (여기서는 C 포트를 사용하므로 DDRC 레지스터)에 '1'로 설정
 - PORTx 레지스터(여기서는 PORTC 레지스터)에 '1'로 설정

1)예제 : 입출력포트 제어용 레지스터 익히기

- AVR 시스템 헤더 파일

헤더 파일명	설 명
<avr/interrupt.h>	ATmega128의 인터럽트에 관련된 내용을 정의
<avr/signal.h>	ATmega128에서 발생하는 신호에 관련된 내용을 정의
<avr/pgmspace.h>	ATmega128의 프로그램 공간에 관련된 내용을 정의
<avr/eeprom.h>	ATmega128의 EEPROM에 관련된 내용을 정의
<avr/wdt.h>	ATmega128의 워치독 타이머에 관련된 내용을 정의

1)예제 : 입출력포트 제어용 레지스터 익히기

- 마이크로 컨트롤러 구동시 시간지연 방법
 - 반복문에 의한 시간 지연
 - for문이나 while문을 사용하여 시간을 지연

```
void delay(unsigned char i) {  
    while(i--);  
}  
혹은  
void delay(unsigned char i) {  
    int k;  
    for(k=0;k<=i;k++) ;  
}  
  
void main(void) {  
    delay(0x0100);  
}
```

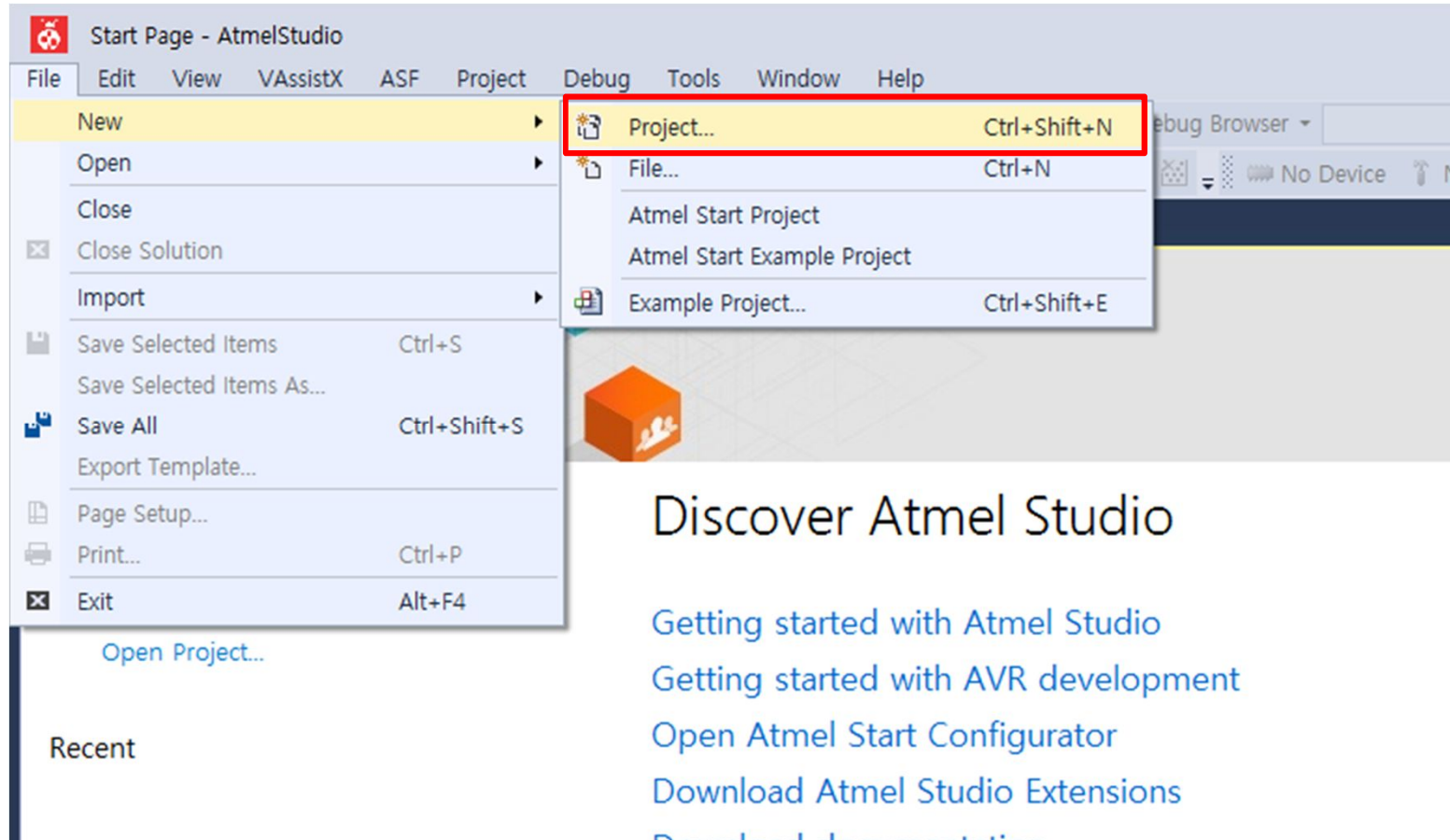
- 매우 부정확한 방법임(MCU상태, 클럭속도에 따라 달라짐)
- 그러나 가장 손쉬운 방법

1)예제 : 입출력포트 제어용 레지스터 익히기

- 마이크로 컨트롤러 구동시 시간지연 방법
 - 시스템 제공 함수를 이용하는 시간 지연
 - 시스템에서 소프트웨어적으로 제공하는 라이브러리 함수를 이용하는 방법
 - AVR 개발환경에서 제공하는 시간 지연용 함수들은 delay.h라는 헤더 파일에 정의되어 있음
 - `_delay_ms(unsigned int i), _delay_us(unsigned int i)`
 - 비교적 정확한 시간 지연을 얻을 수 있음
 - 인터럽트 등에 의해 지연 발생이 가능함
 - 하드웨어에 의한 시간 지연
 - 마이크로컨트롤러에서 하드웨어로 제공하는 내부 타이머/카운터를 사용하는 방법
 - 가장 정확한 방법

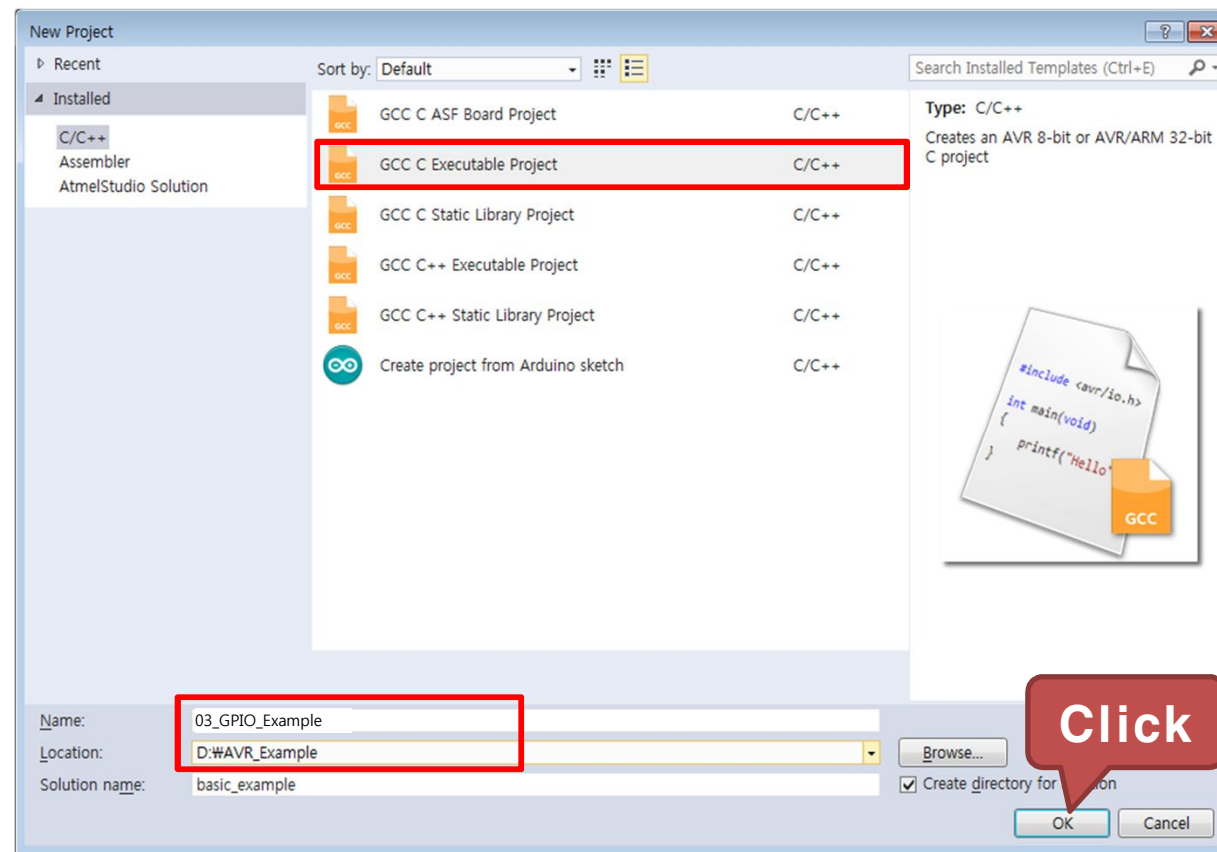
1)예제 : 입출력포트 제어용 레지스터 익히기

- File → New → Project... 을 클릭하여 새 프로젝트를 생성



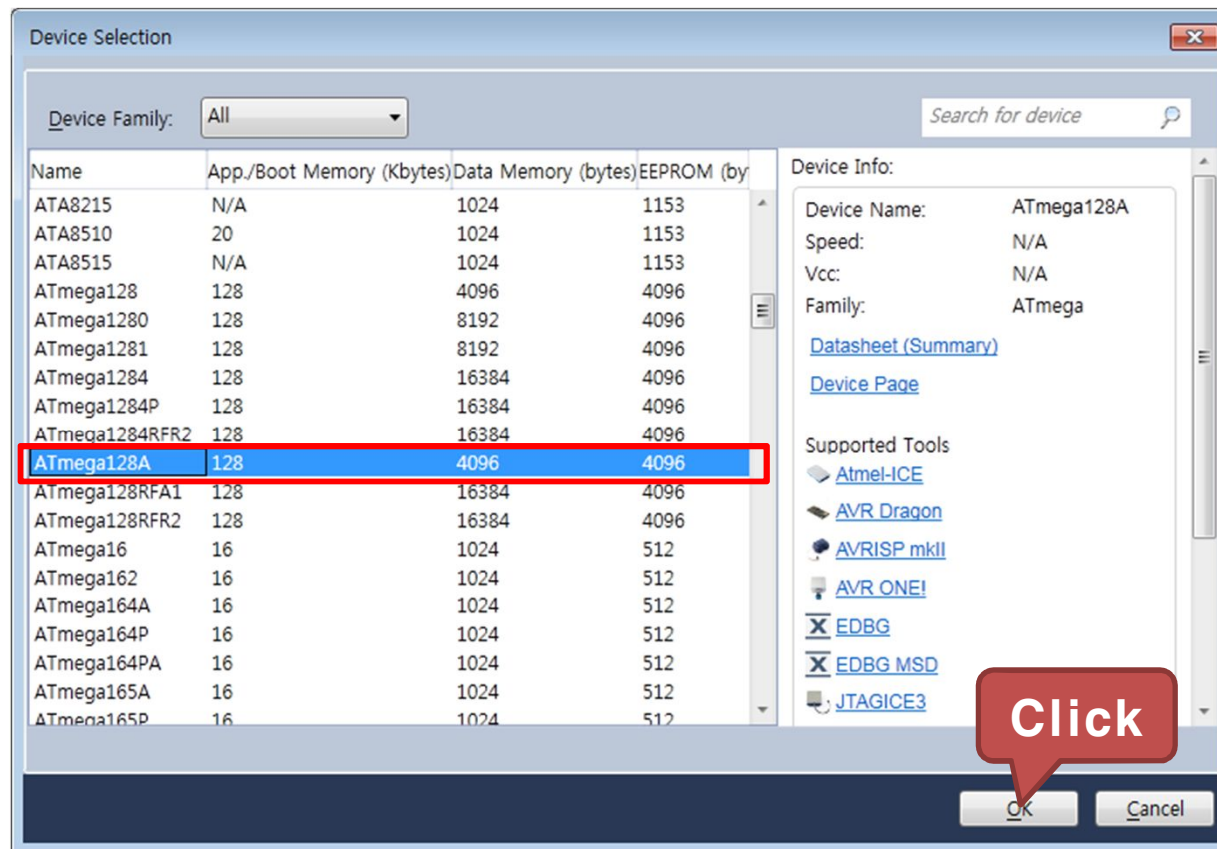
1)예제 : 입출력포트 제어용 레지스터 익히기

- GCC C Executable Project를 선택하고, 생성된 Project의 Name 과 Location을 설정
 - Name : 03_GPIO_Example
 - Location : D:\WAVR_Example



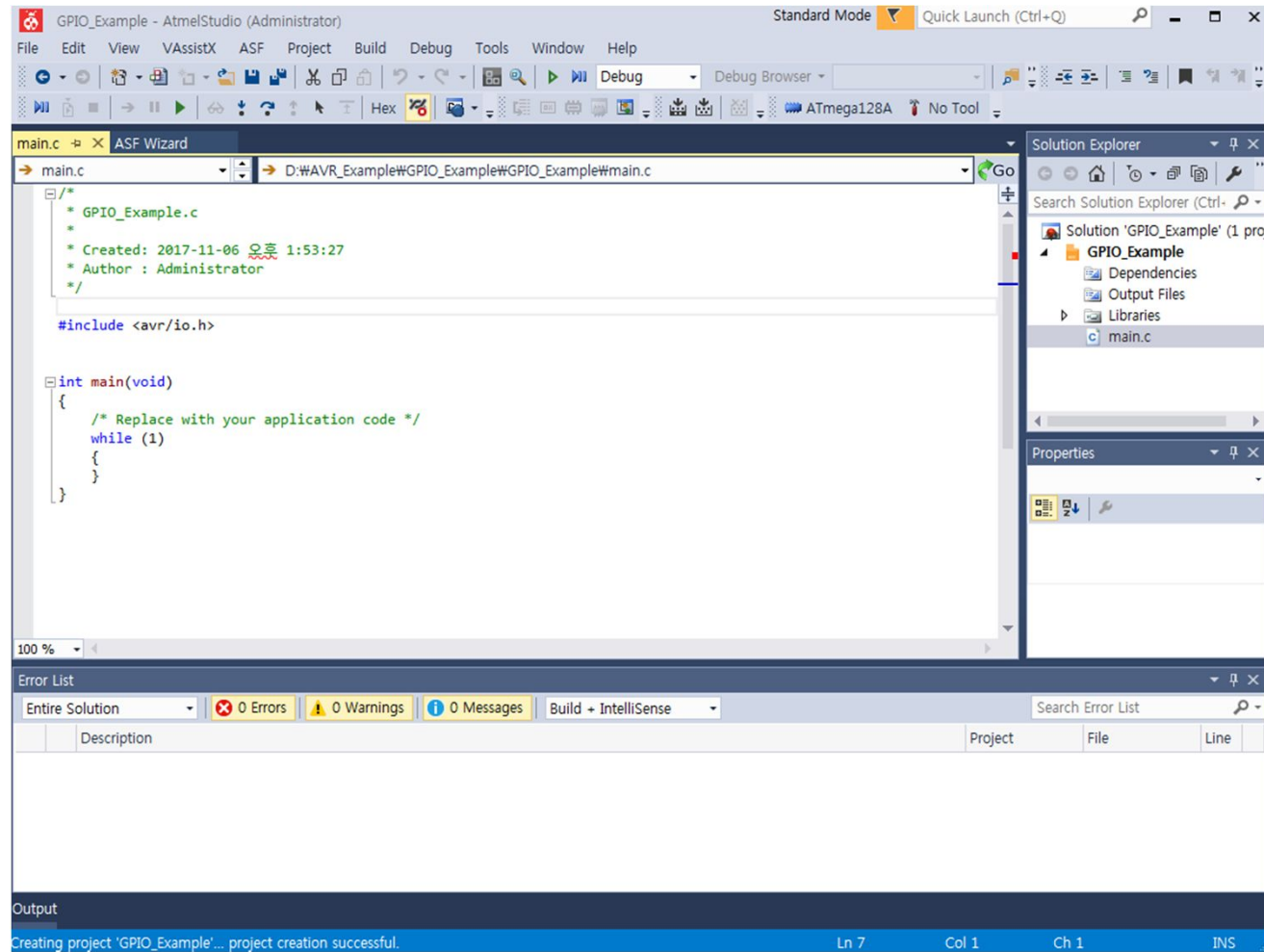
1)예제 : 입출력포트 제어용 레지스터 익히기

- ATmega128A를 선택하고 OK를 클릭



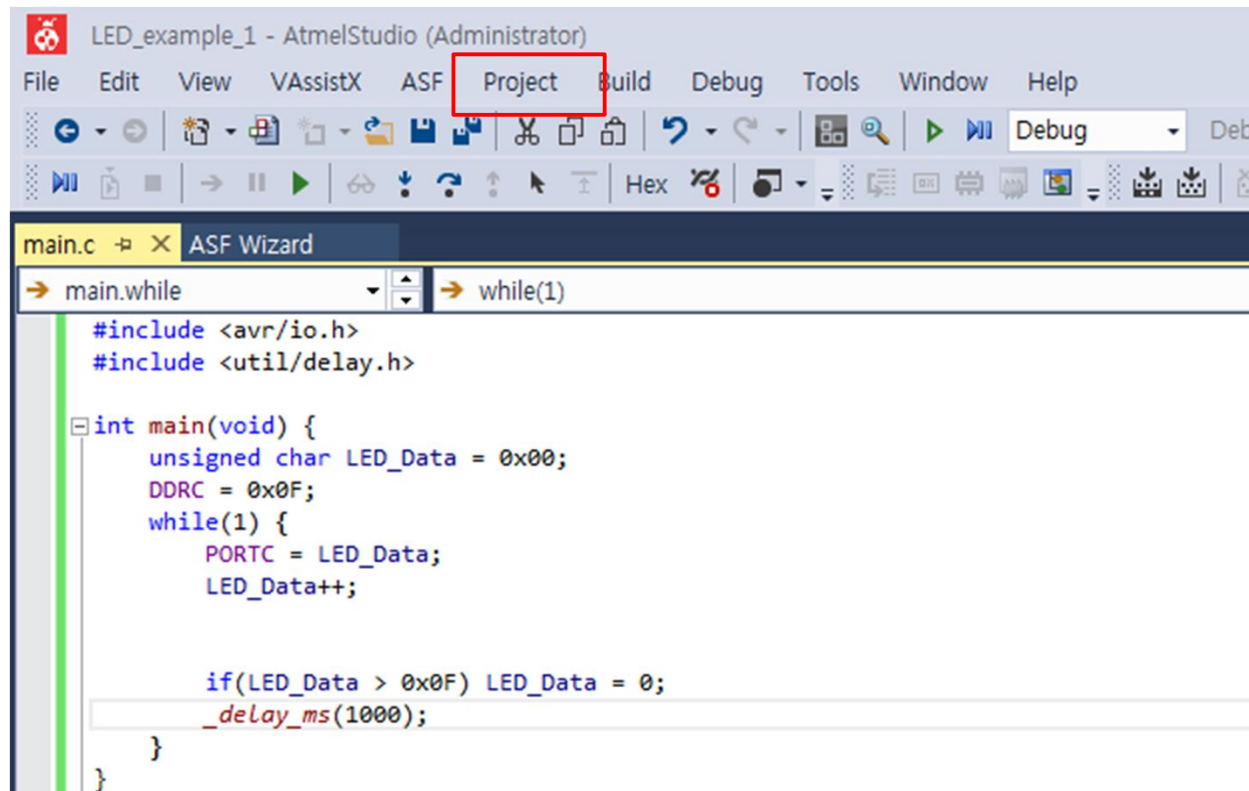
1)예제 : 입출력포트 제어용 레지스터 익히기

- 03_GPIO_Example 프로젝트 생성 완료



1)예제 : 입출력포트 제어용 레지스터 익히기

- 프로젝트 설정
 - Project 탭에서 "03_GPIO_Example Properties..." 를 선택



The screenshot shows the Atmel Studio (Administrator) interface. The 'Project' menu item in the top toolbar is highlighted with a red rectangle. Below the toolbar, the 'main.c' file is open, and the 'ASF Wizard' is visible. The code in the editor is as follows:

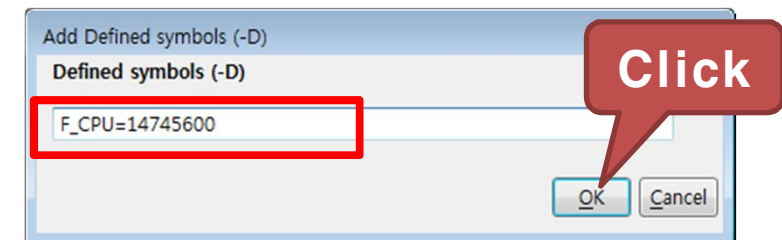
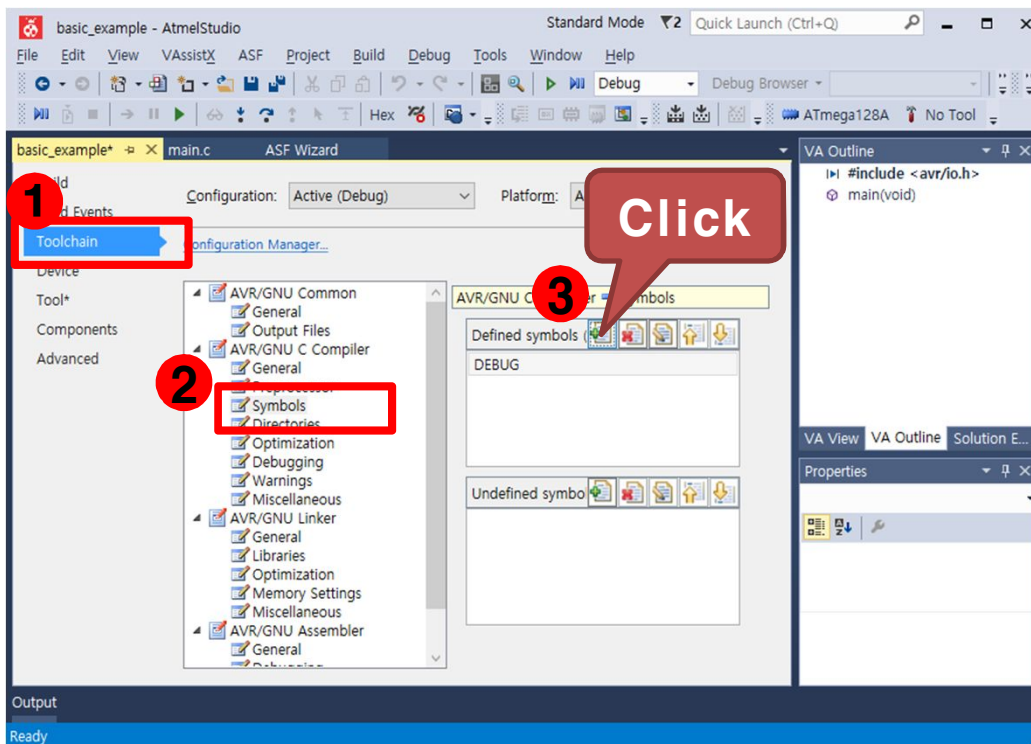
```
#include <avr/io.h>
#include <util/delay.h>

int main(void) {
    unsigned char LED_Data = 0x00;
    DDRC = 0x0F;
    while(1) {
        PORTC = LED_Data;
        LED_Data++;

        if(LED_Data > 0x0F) LED_Data = 0;
        _delay_ms(1000);
    }
}
```

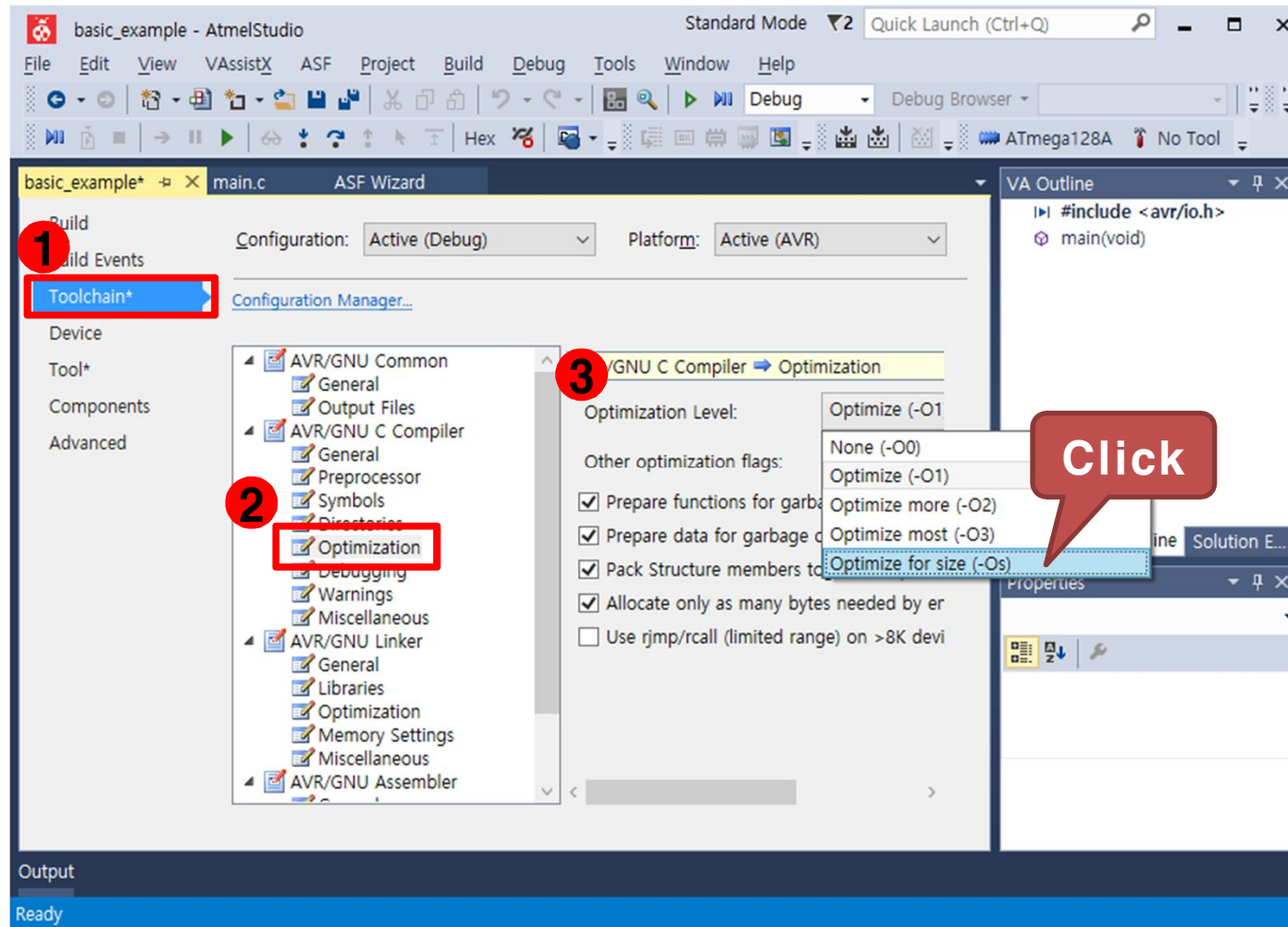
1)예제 : 입출력포트 제어용 레지스터 익히기

- Toolchain → AVR/GNU C Compiler → Symbols를 선택한 후 우측의 Defined symbols에서 "Add Item" 을 선택
- "F_CPU=14745600"을 입력하고 OK를 클릭. AVR의 외부 클럭 14.7456Mhz



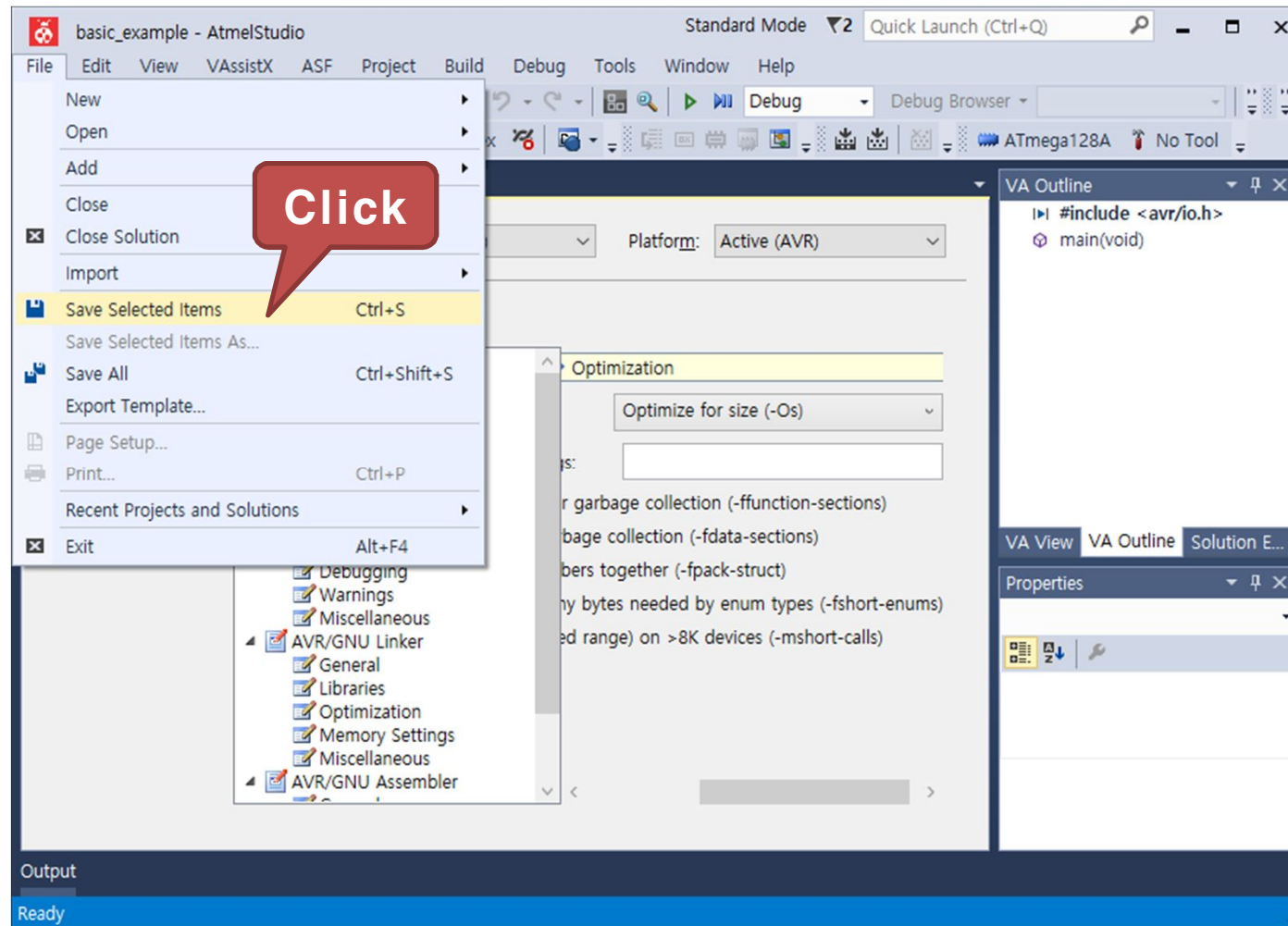
1)예제 : 입출력포트 제어용 레지스터 익히기

- Toolchain → AVR/GNU C Compiler → Optimization 을 선택한 후 우측의 Optimization Level 을 변경, "Optimize for size (-OS)" 선택



1)예제 : 입출력포트 제어용 레지스터 익히기

- File 탭에서 "Save Selected Items"를 클릭하여 설정값을 저장



1)예제 : 입출력포트 제어용 레지스터 익히기

- 구동 프로그램
 - main.c 코드 작성

```
#include <avr/io.h>      // AVR 입출력에 대한 헤더 파일
#include <util/delay.h>  // delay 함수사용을 위한 헤더파일

int main(void) {

    while(1) {
        DDRC = 0x03;      // LED0, 1 출력모드, LED2, 3 입력모드로 설정
        PORTC = 0x0F;     // LED0~3 모두 1로 출력
        _delay_ms(500);    // 500ms 시간 지연

        PORTC = 0x00;     // LED0~3 모두 0으로 출력
        _delay_ms(500);    // 500ms 시간 지연
    }
}
```

1)예제 : 입출력포트 제어용 레지스터 익히기

- 구동 프로그램
 - main.c 코드 작성

이어서

...

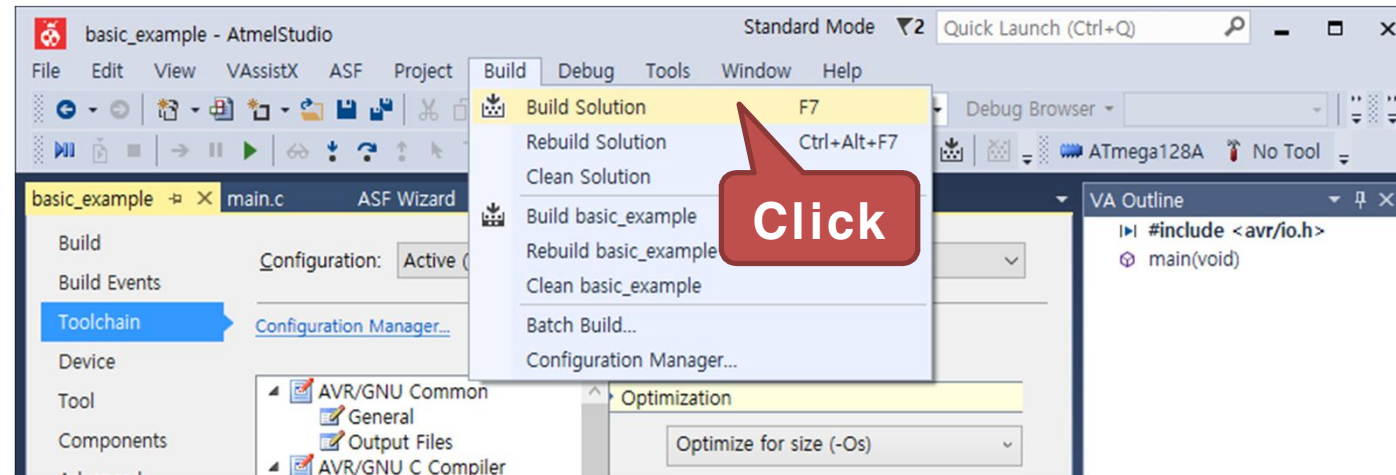
...

```
DDRC = 0x0C;           // LED0, 1 입력모드, LED2, 3 출력모드로 설정
PORTC = 0x0F;           // LED0~3 모두 1로 출력
_delay_ms(500);         // 500ms 시간 지연

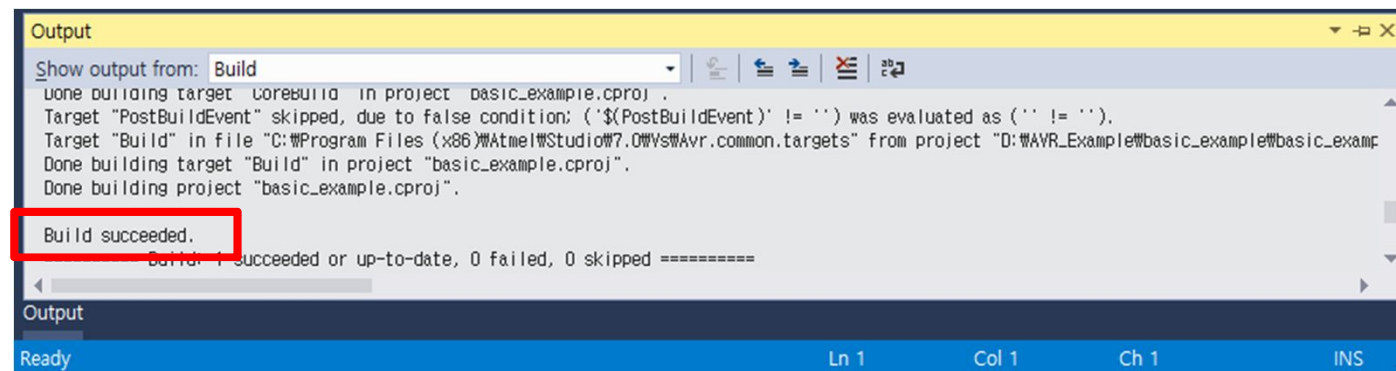
PORTC = 0x00;           // LED0~3 모두 0으로 출력
_delay_ms(500);         // 500ms 시간 지연
}
```


1) 예제 : 입출력포트 제어용 레지스터 익히기

- Build 탭에서 "Build Solution" 을 클릭하여 컴파일을 실행

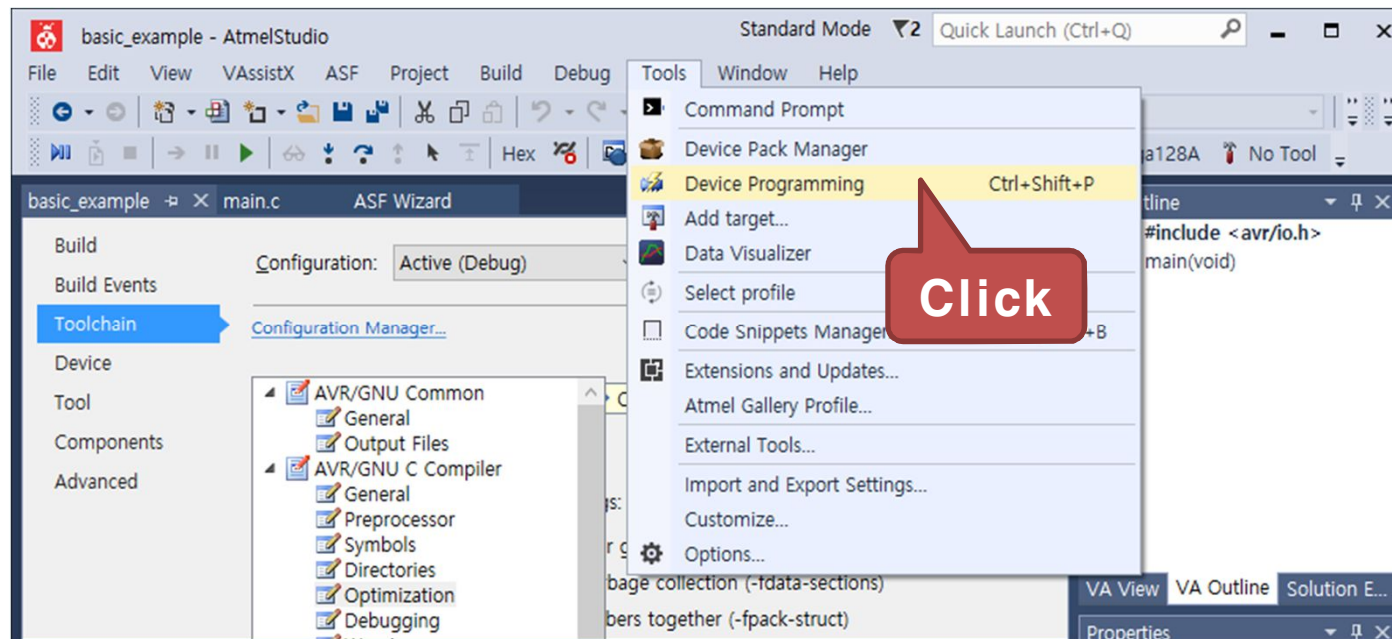


- 다음과 같이 Output 창에 "Build succeeded" 가 출력되면 컴파일이 완료



1)예제 : 입출력포트 제어용 레지스터 익히기

- Tools 탭에서 "Device Programming" 을 클릭



1)예제 : 입출력포트 제어용 레지스터 익히기

- Device Programming 창이 나타나면 Tool 에서 AVRISP mkII를 선택



- Device를 ATmega128A로 선택하고 "Apply" 버튼을 클릭

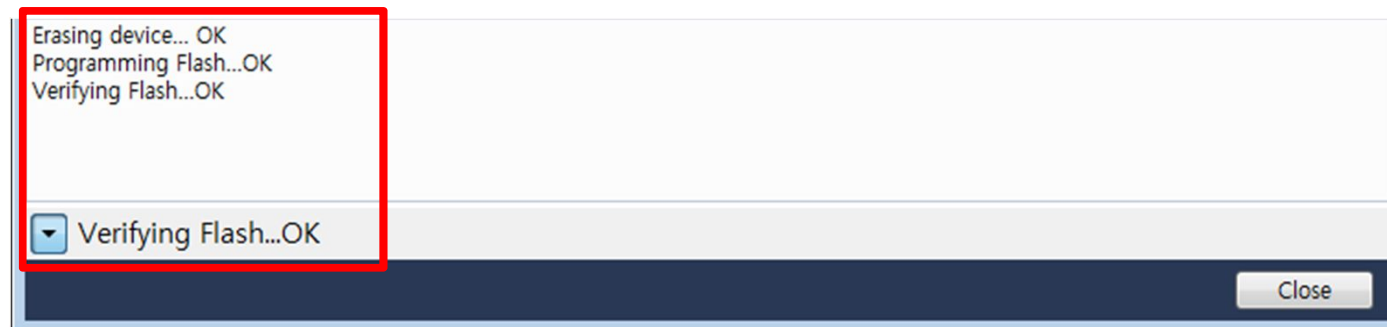


1)예제 : 입출력포트 제어용 레지스터 익히기

- Device 인식이 완료되면 Memories 탭을 선택하고, "Program" 버튼을 클릭

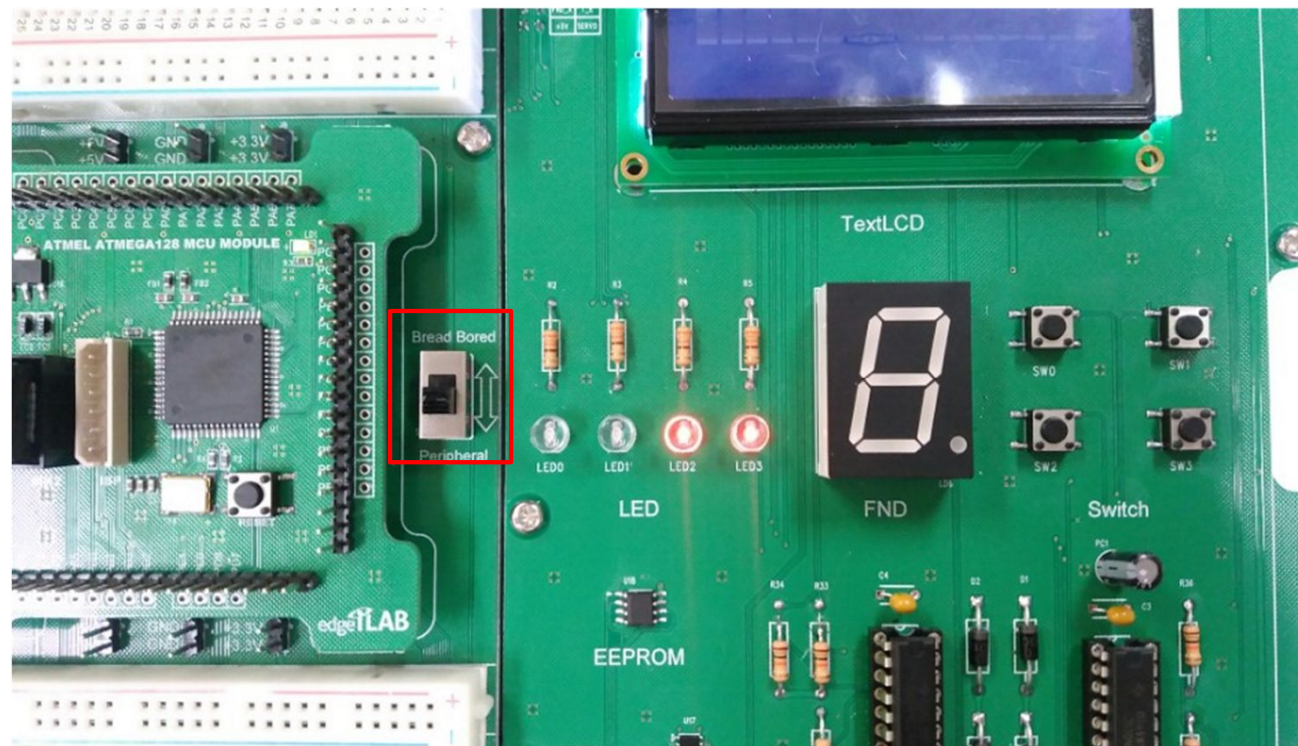


- 프로그램 다운로드가 성공하면 다음과 같은 메시지가 출력된다.



1)예제 : 입출력포트 제어용 레지스터 익히기

- 실행 결과
 - LED0,1 불이 켜지고, LED2,3의 불이 순차적으로 점등
 - 보드 옆 스위치를 Peripheral(주변장치)로 이동

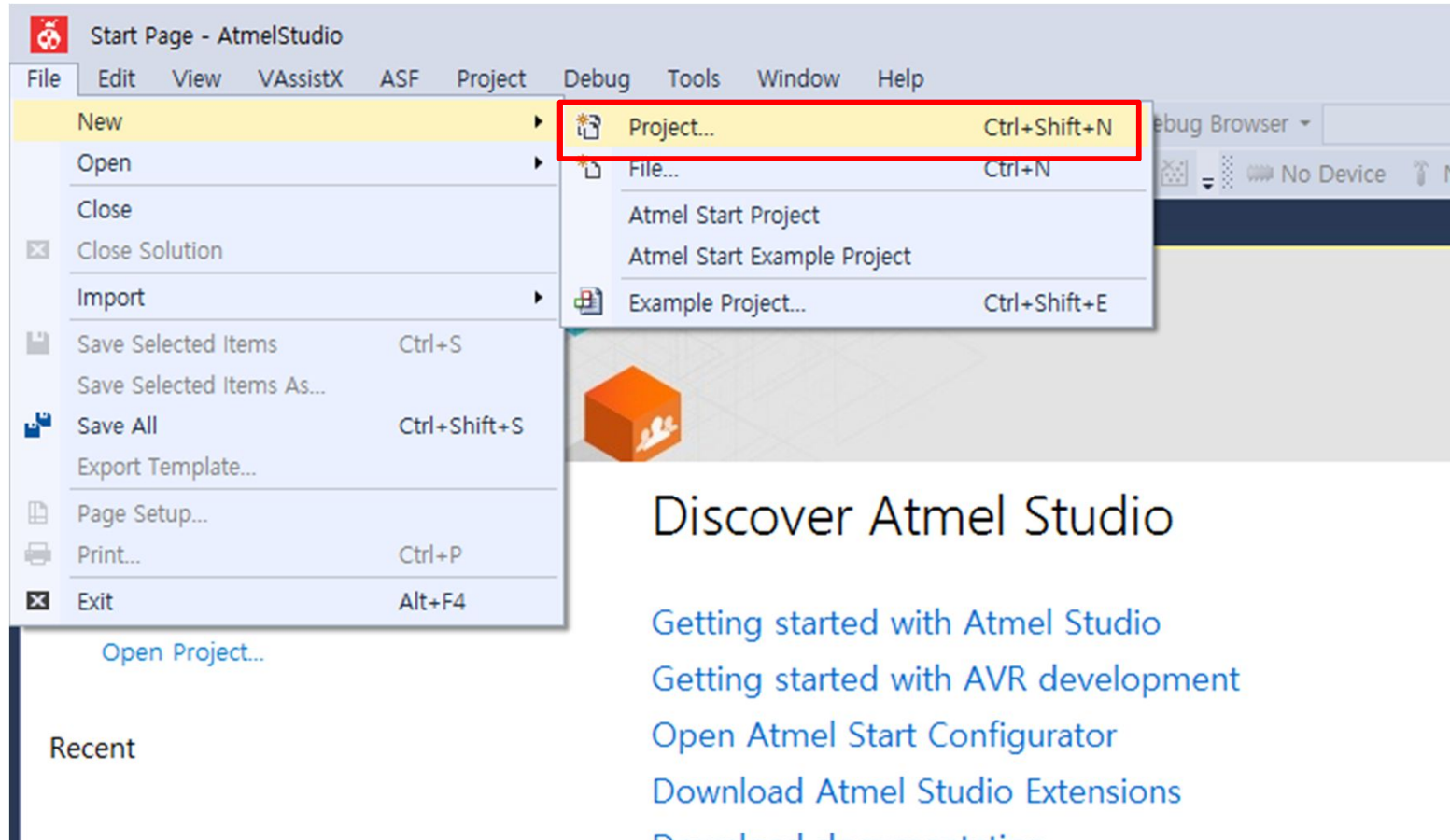


2)예제 : GPIO를 이용하여 LED 켜기

- 실습 개요
 - ATmega128A 마이크로컨트롤러의 GPIO를 이용하여 LED를 켜기
 - 입출력 포트를 출력으로 설정하고, 그 포트를 이용하여 LED에 신호를 보내 점등
 - 프로그램이 시작하면 1초마다 LED 점등
- 실습 목표
 - GPIO 입출력 포트의 방향 제어 및 출력 제어 방법 습득
 - LED 동작 원리 습득

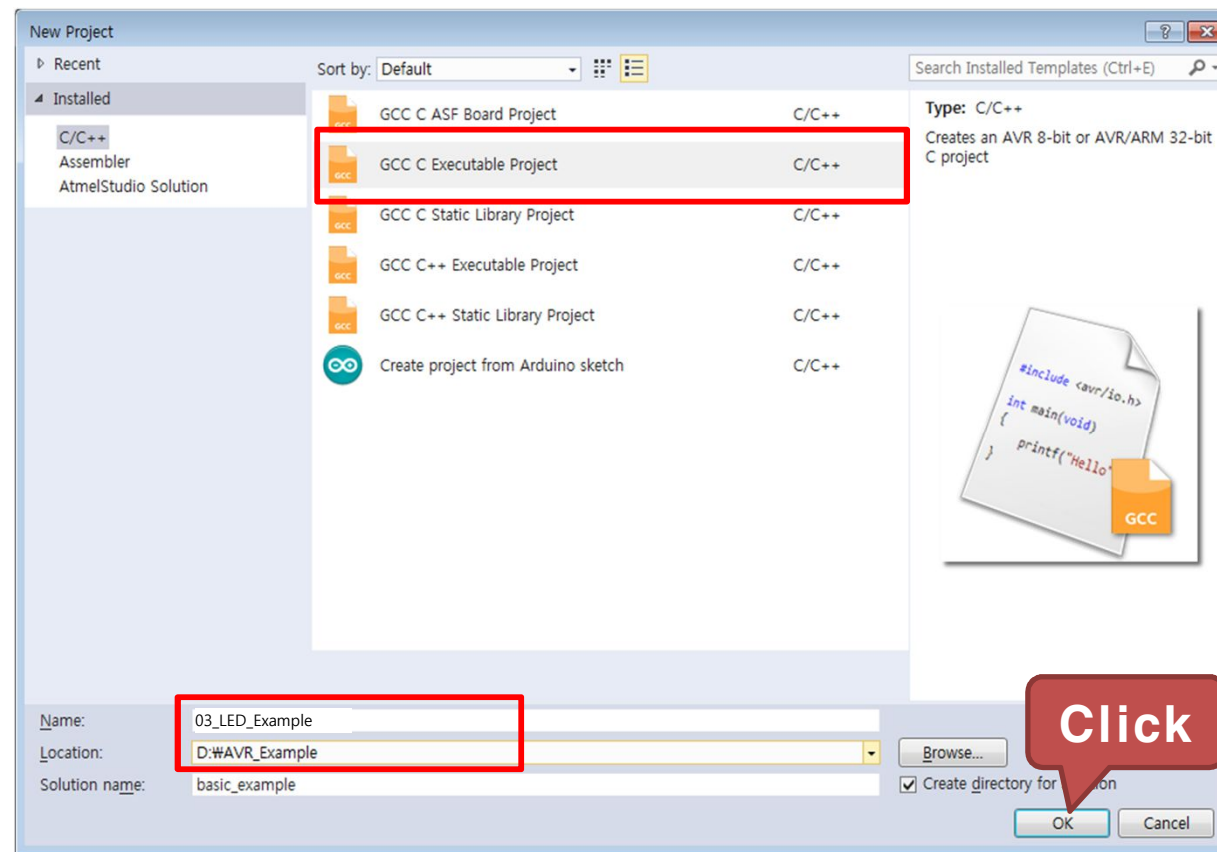
2)예제 : GPIO를 이용하여 LED 켜기

- File → New → Project... 을 클릭하여 새 프로젝트를 생성



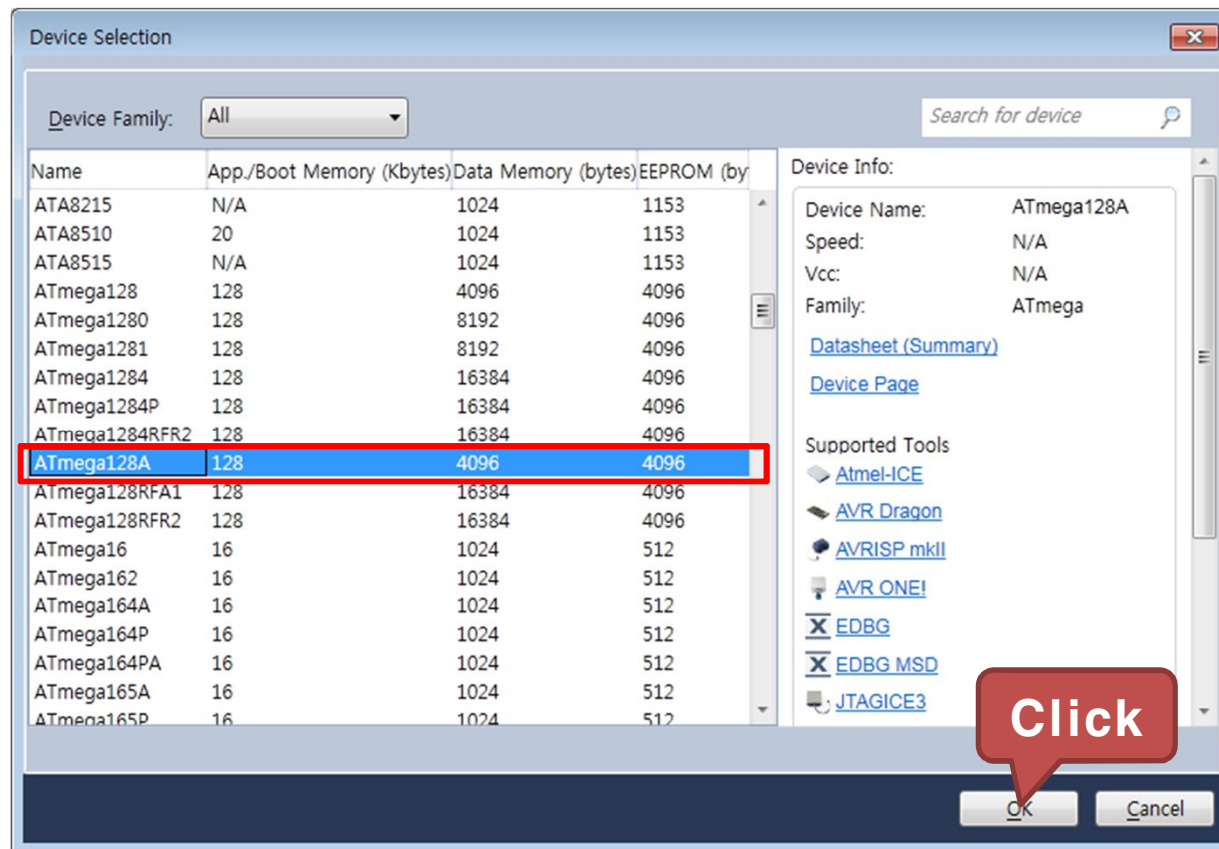
2)예제 : GPIO를 이용하여 LED 켜기

- GCC C Executable Project를 선택하고, 생성된 Project의 Name 과 Location을 설정
 - Name : 03_LED_Example
 - Location : D:\WAVR_Example



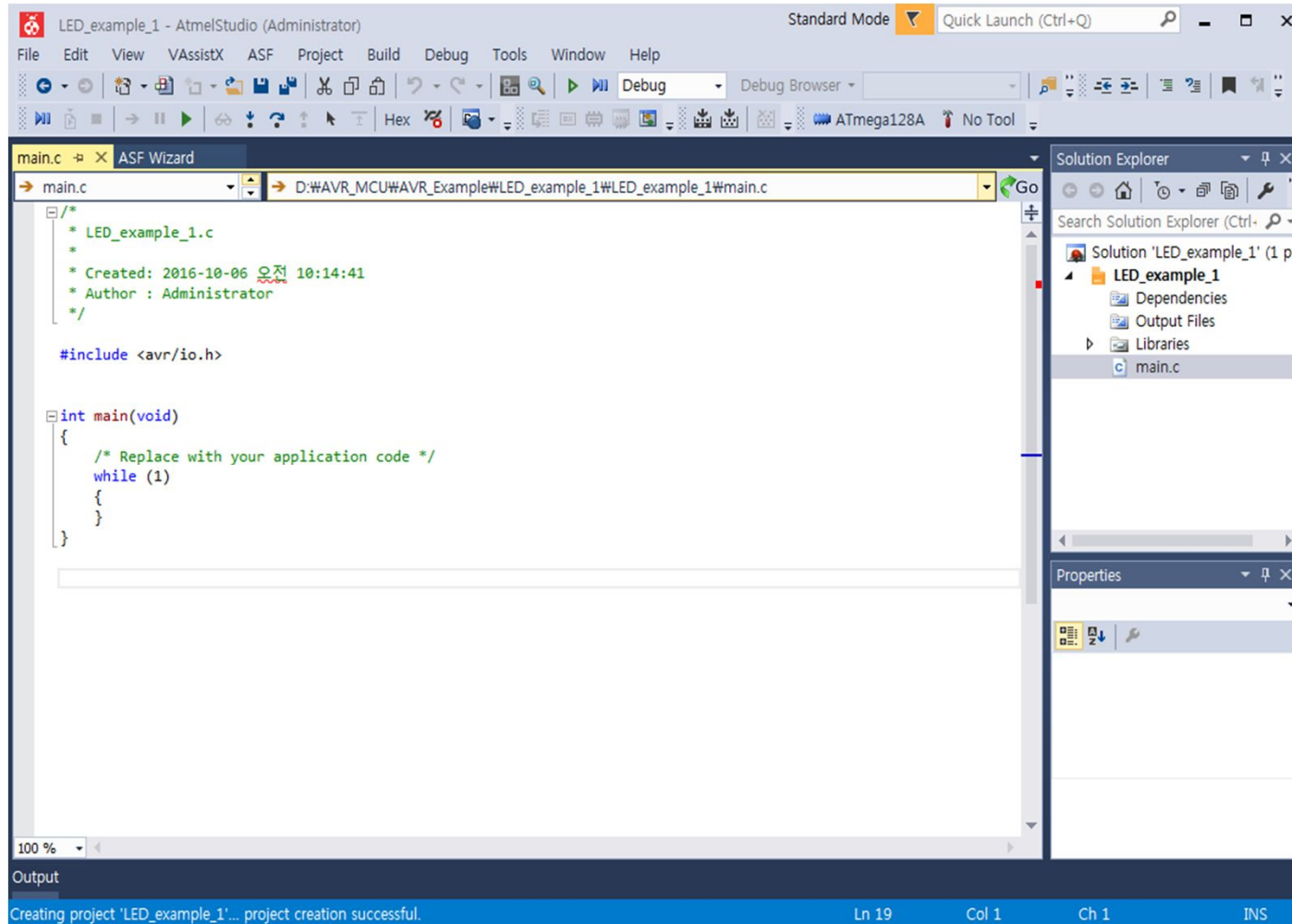
2)예제 : GPIO를 이용하여 LED 켜기

- ATmega128A를 선택하고 OK를 클릭



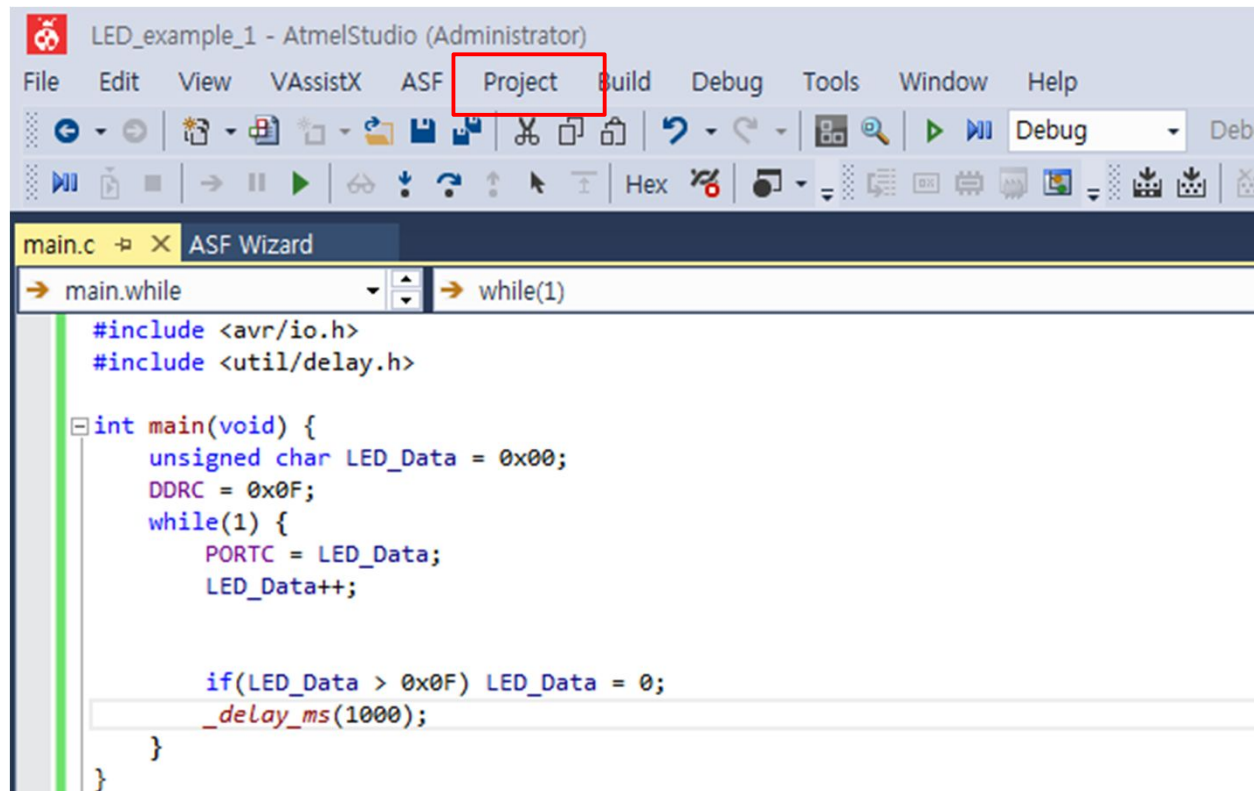
2)예제 : GPIO를 이용하여 LED 켜기

- 03_LED_Example 프로젝트 생성 완료



2)예제 : GPIO를 이용하여 LED 켜기

- 프로젝트 설정
 - Project 탭에서 "03_LED_Example Properties..." 를 선택



The screenshot shows the Atmel Studio (Administrator) interface. The 'Project' menu item in the top toolbar is highlighted with a red rectangle. Below the toolbar, the 'main.c' file is open, and the 'ASF Wizard' is visible. The code in the editor is as follows:

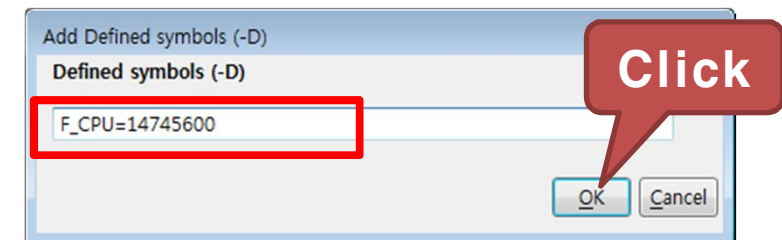
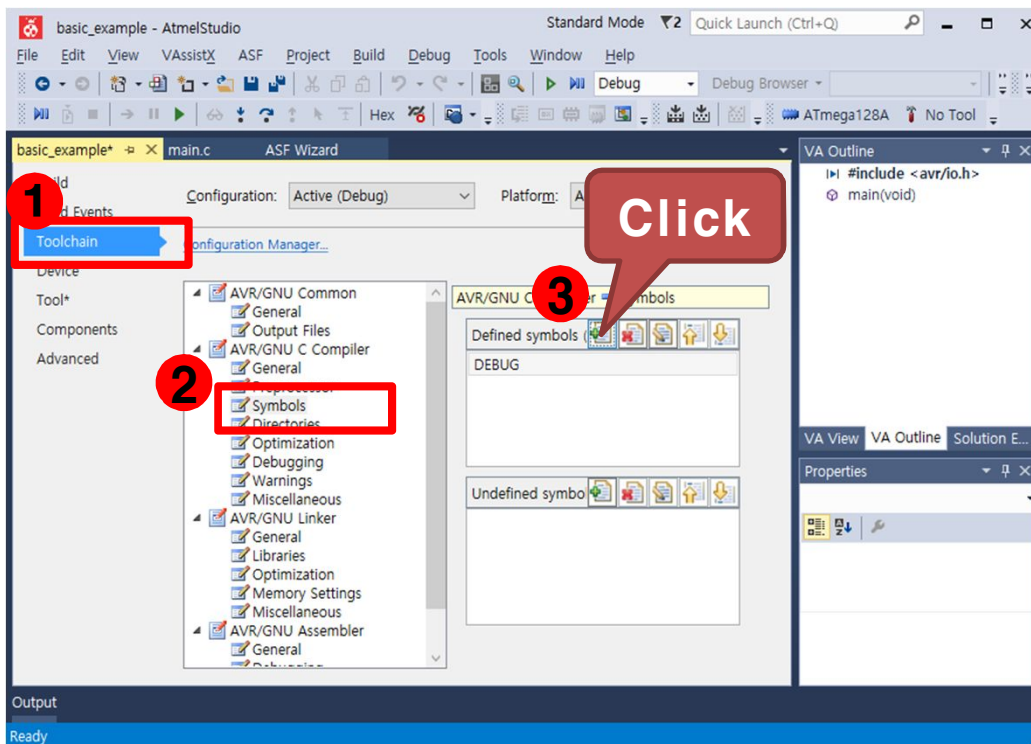
```
main.c x ASF Wizard
main.while while(1)
#include <avr/io.h>
#include <util/delay.h>

int main(void) {
    unsigned char LED_Data = 0x00;
    DDRC = 0x0F;
    while(1) {
        PORTC = LED_Data;
        LED_Data++;

        if(LED_Data > 0x0F) LED_Data = 0;
        _delay_ms(1000);
    }
}
```

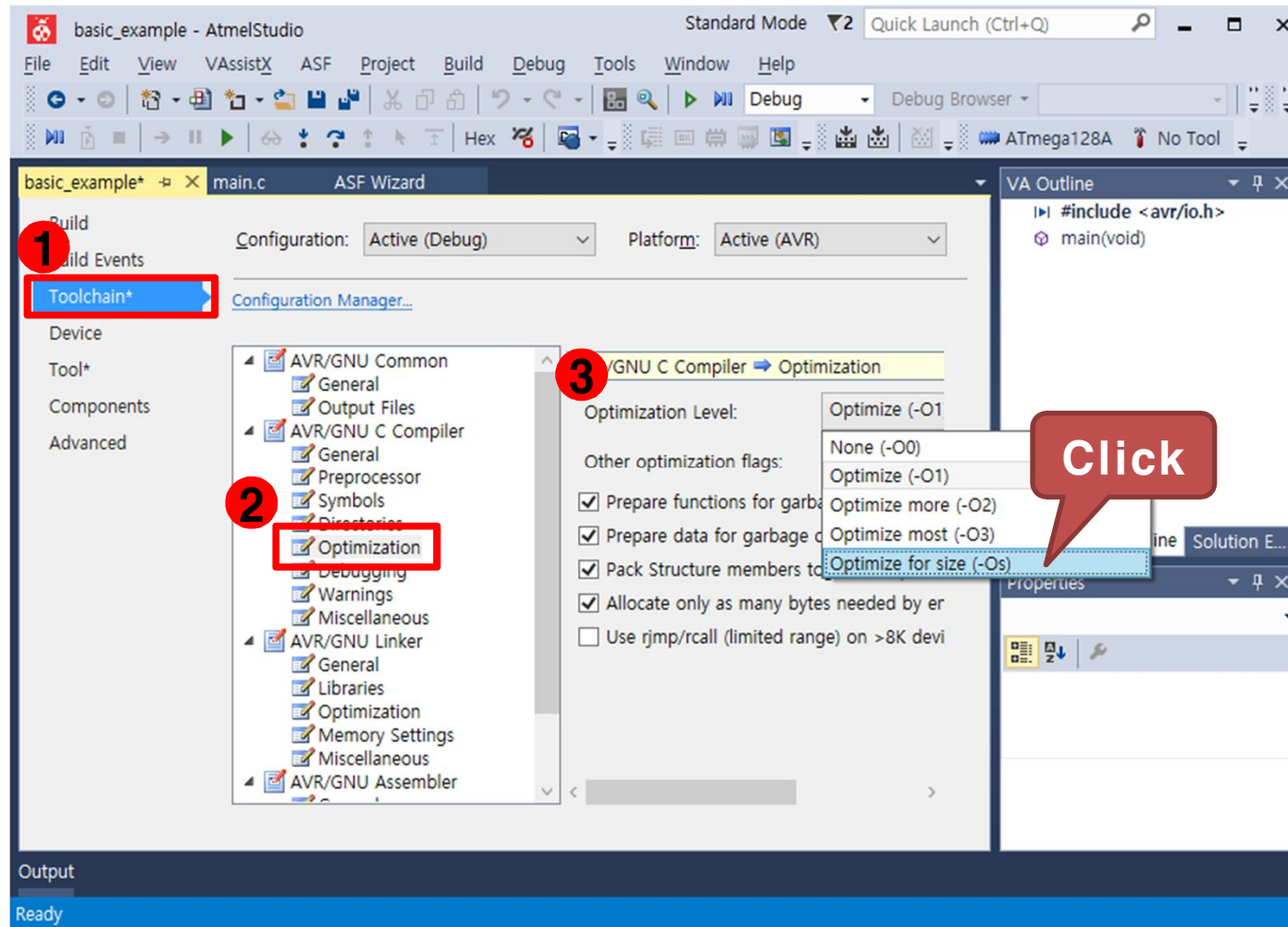
2)예제 : GPIO를 이용하여 LED 켜기

- Toolchain → AVR/GNU C Compiler → Symbols를 선택한 후 우측의 Defined symbols에서 "Add Item" 을 선택
- "F_CPU=14745600"을 입력하고 OK를 클릭. AVR의 외부 클럭 14.7456Mhz



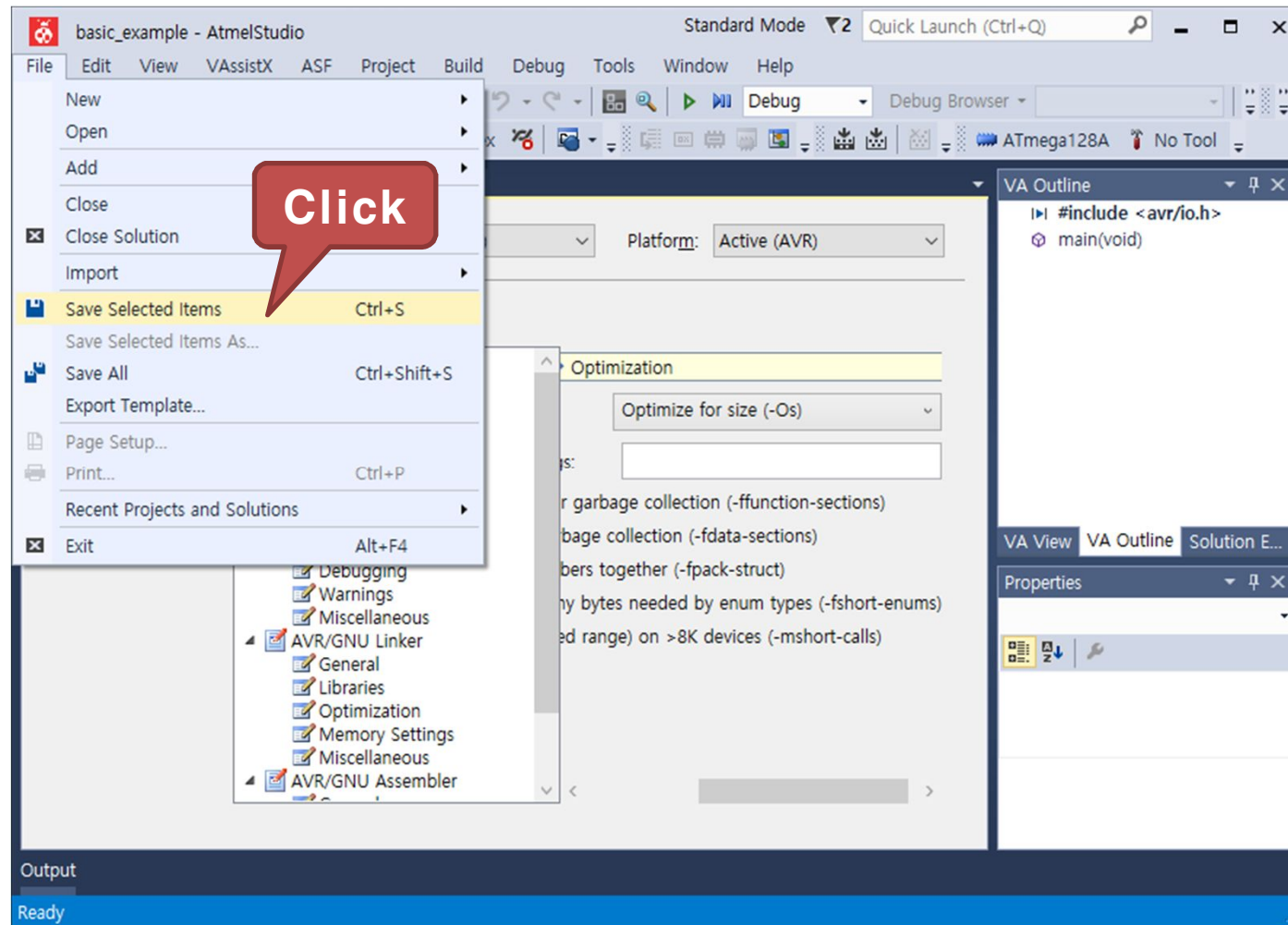
2)예제 : GPIO를 이용하여 LED 켜기

- Toolchain → AVR/GNU C Compiler → Optimization 을 선택한 후 우측의 Optimization Level 을 변경, "Optimize for size (-Os)" 선택



2)예제 : GPIO를 이용하여 LED 켜기

- File 탭에서 "Save Selected Items"를 클릭하여 설정값을 저장



2)예제 : GPIO를 이용하여 LED 켜기

- 구동 프로그램
 - main.c 코드 작성

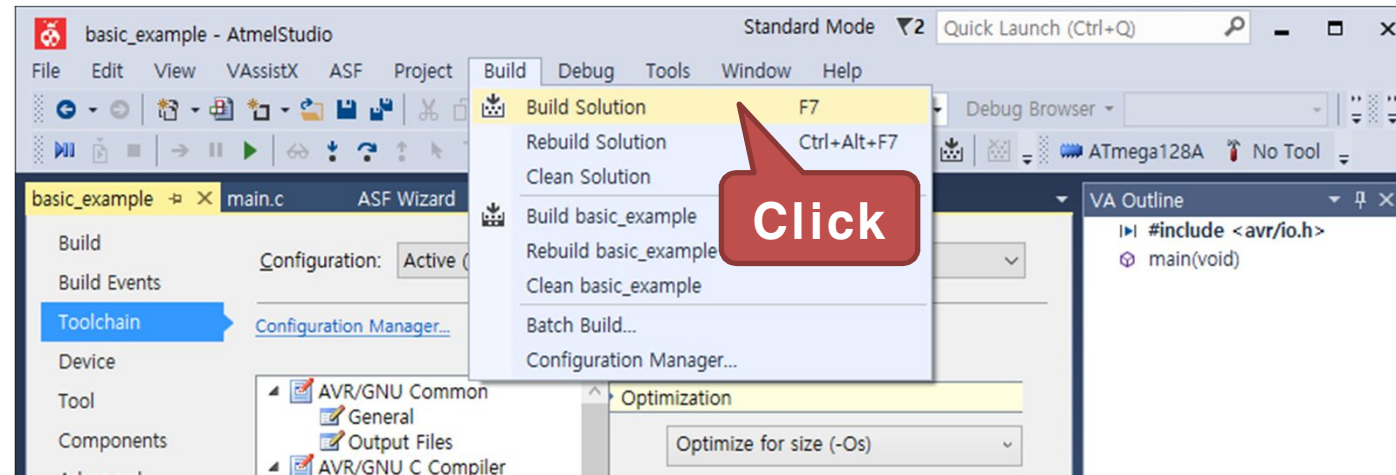
```
#include <avr/io.h>      // AVR 입출력에 대한 헤더 파일
#include <util/delay.h>  // delay 함수사용을 위한 헤더파일

int main(void) {
    unsigned char LED_Data = 0x00;
    DDRC = 0x0F;          // 포트C를 출력포트로 설정
    while(1) {
        PORTC = LED_Data; // 포트C로 변수 LED_Data에 있는 데이터 출력
        LED_Data++;       // LED_Data값을 1씩 증가

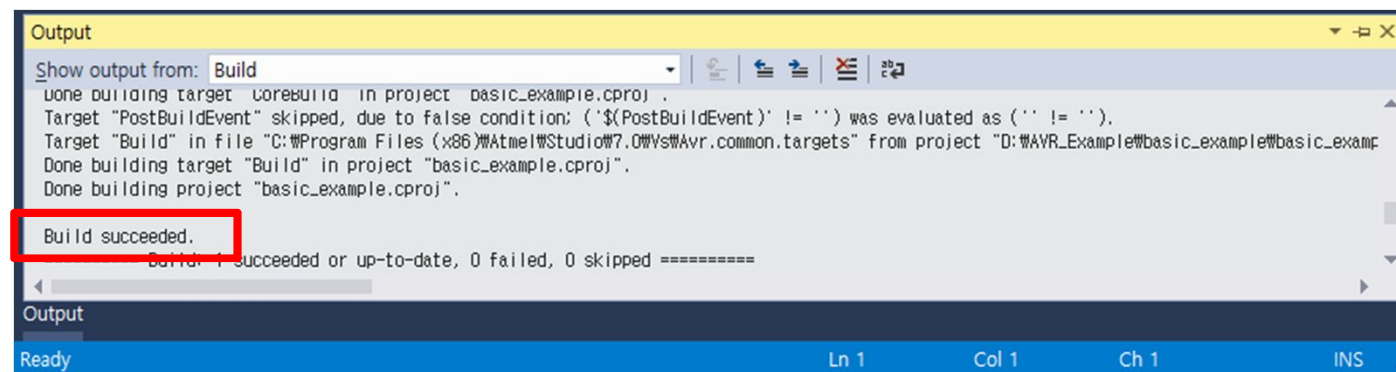
        // LED_Data값이 0x0F 보다 크면 0으로 초기화
        if(LED_Data > 0x0F) LED_Data = 0;
        _delay_ms(1000);    // ms 단위의 딜레이 함수
    }
}
```

2)예제 : GPIO를 이용하여 LED 켜기

- Build 탭에서 "Build Solution" 을 클릭하여 컴파일을 실행

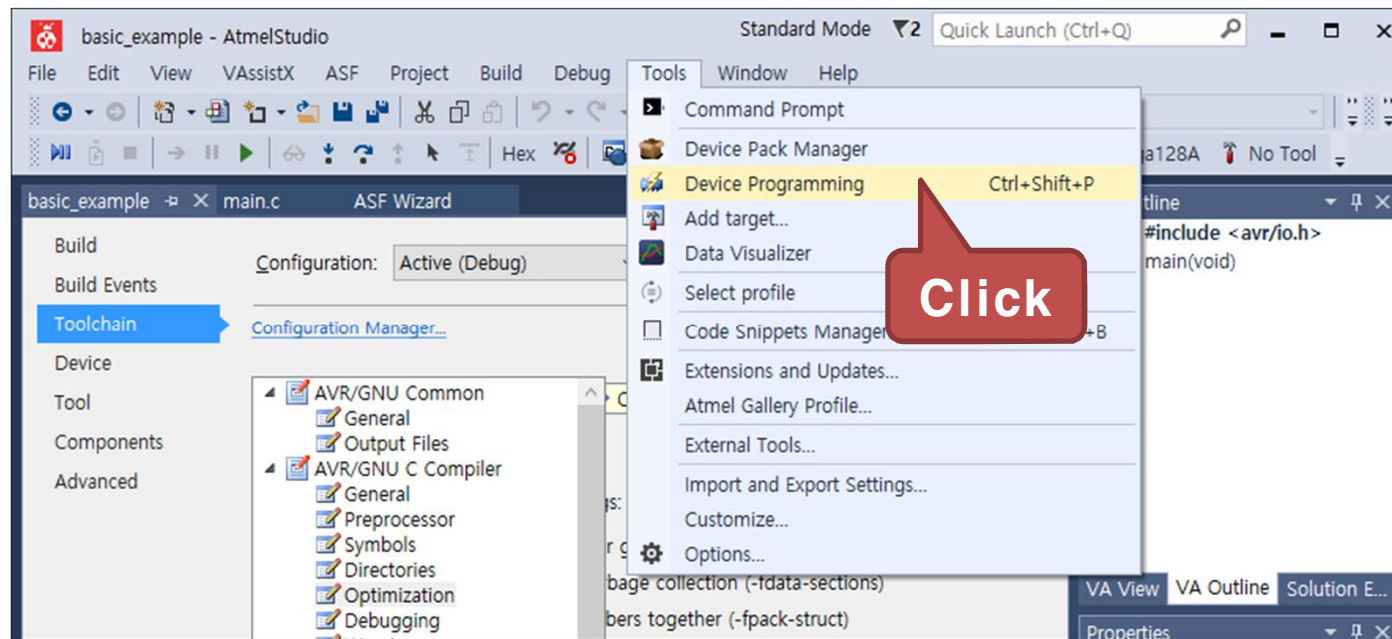


- 다음과 같이 Output 창에 "Build succeeded" 가 출력되면 컴파일이 완료



2)예제 : GPIO를 이용하여 LED 켜기

- Tools 탭에서 "Device Programming" 을 클릭



2)예제 : GPIO를 이용하여 LED 켜기

- Device Programming 창이 나타나면 Tool 에서 AVRISP mkII를 선택



- Device를 ATmega128A로 선택하고 "Apply" 버튼을 클릭

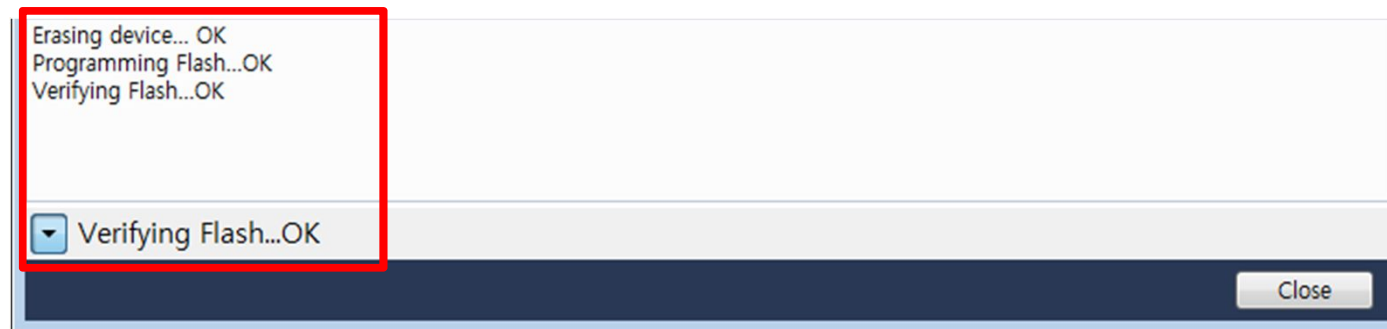


2)예제 : GPIO를 이용하여 LED 켜기

- Device 인식이 완료되면 Memories 탭을 선택하고, "Program" 버튼을 클릭

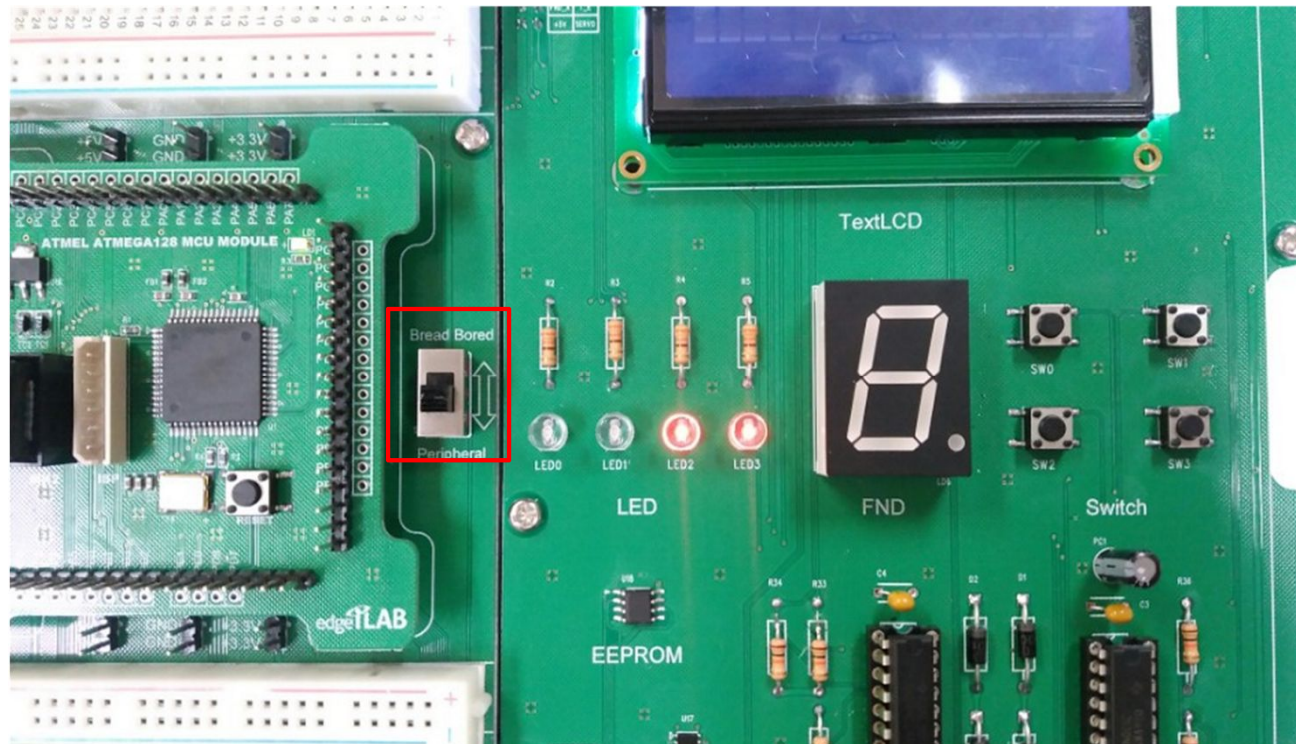


- 프로그램 다운로드가 성공하면 다음과 같은 메시지가 출력된다.



2)예제 : GPIO를 이용하여 LED 켜기

- 실행 결과
 - LED의 불이 순차적으로 점멸
 - 보드 옆 스위치를 Peripheral(주변장치)로 이동

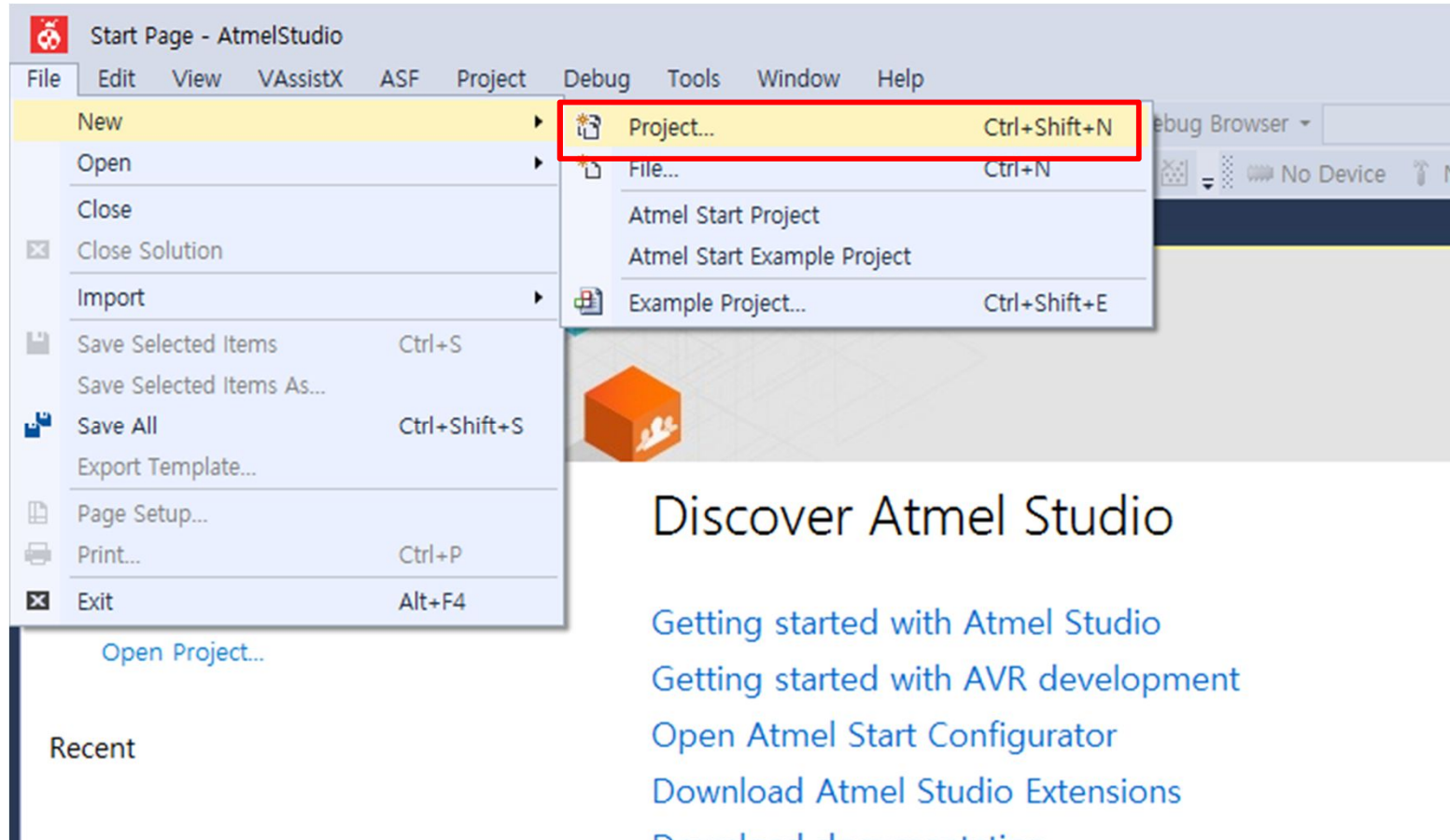


3) 응용 : LED를 좌우로 이동시키기

- 실습 개요
 - LED의 빛을 좌우로 이동
 - 프로그램이 시작하면 1초마다 LED의 빛을 오른쪽으로 이동시키고, 끝에 닿으면 다시 왼쪽으로 이동
- 실습 목표
 - GPIO 입출력 포트의 방향 제어 및 출력 제어 방법 습득

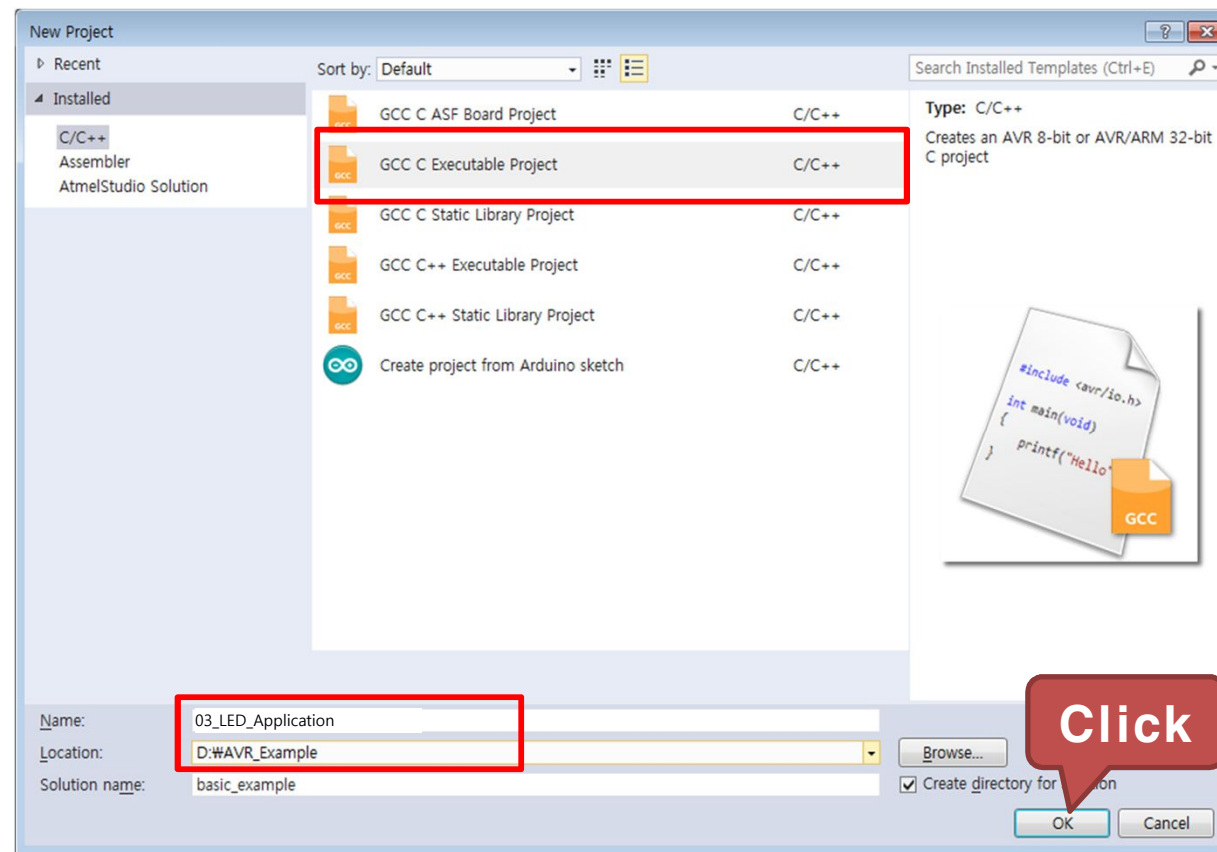
3) 응용 : LED를 좌우로 이동시키기

- File → New → Project... 을 클릭하여 새 프로젝트를 생성



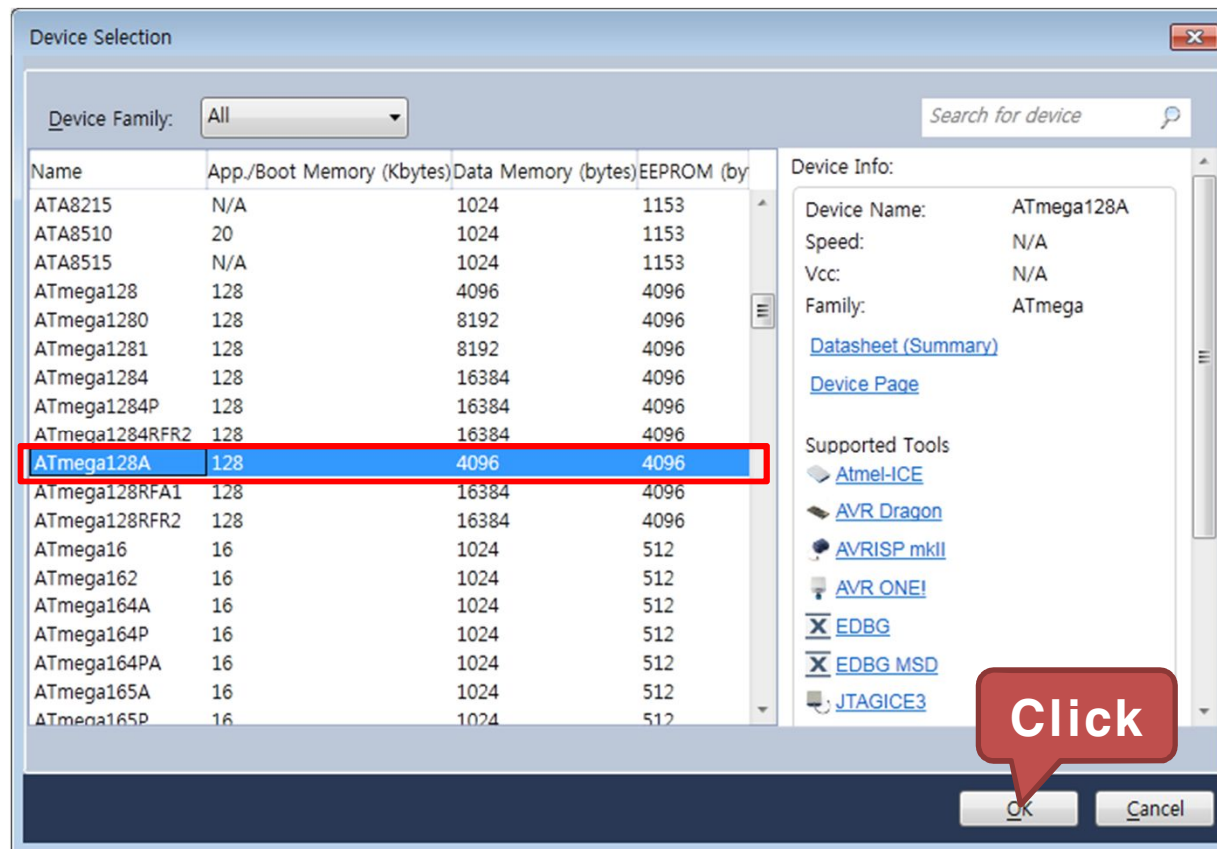
3) 응용 : LED를 좌우로 이동시키기

- GCC C Executable Project를 선택하고, 생성된 Project의 Name 과 Location을 설정
 - Name : 03_LED_Application
 - Location : D:\WAVR_Example



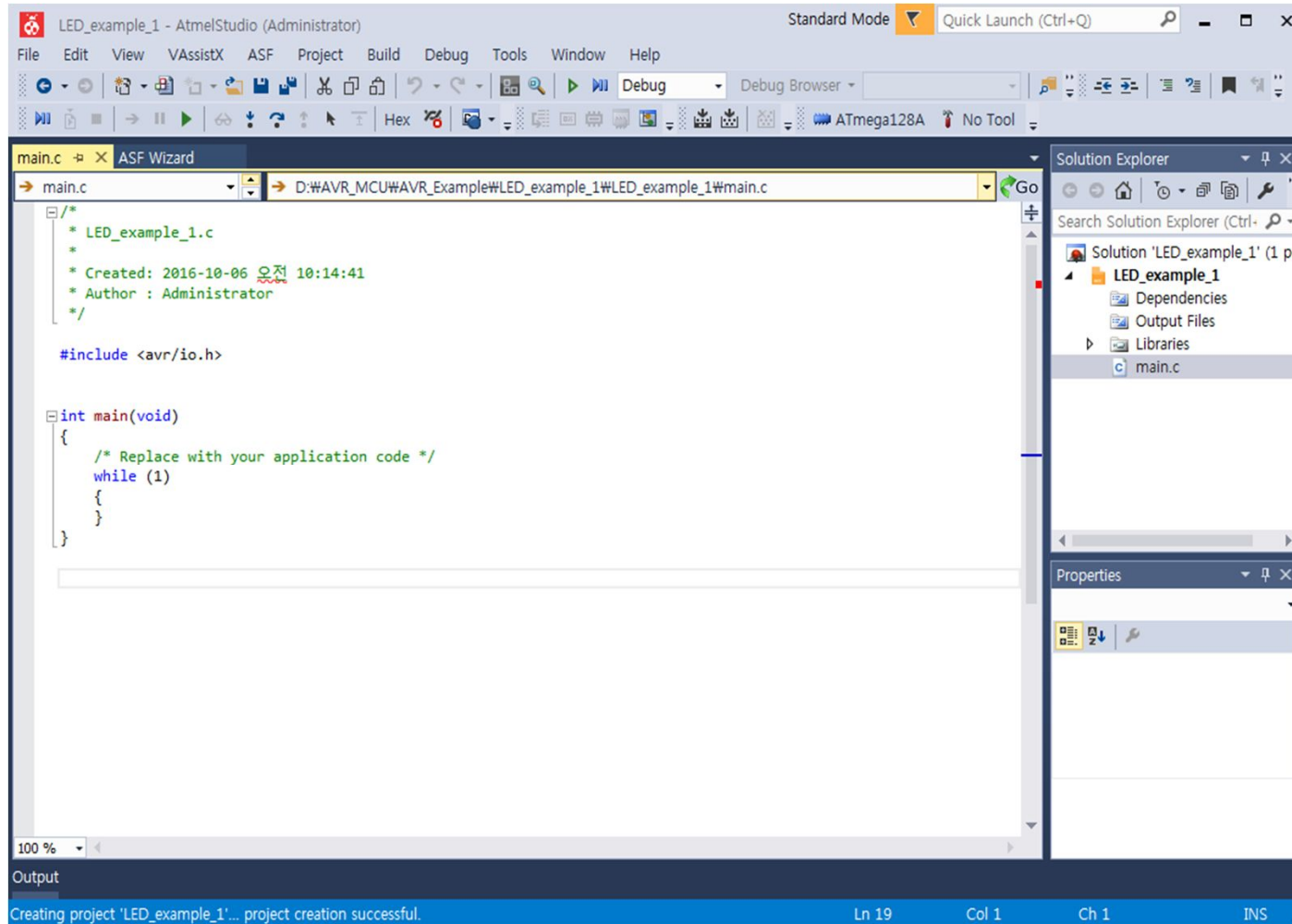
3) 응용 : LED를 좌우로 이동시키기

- ATmega128A를 선택하고 OK를 클릭



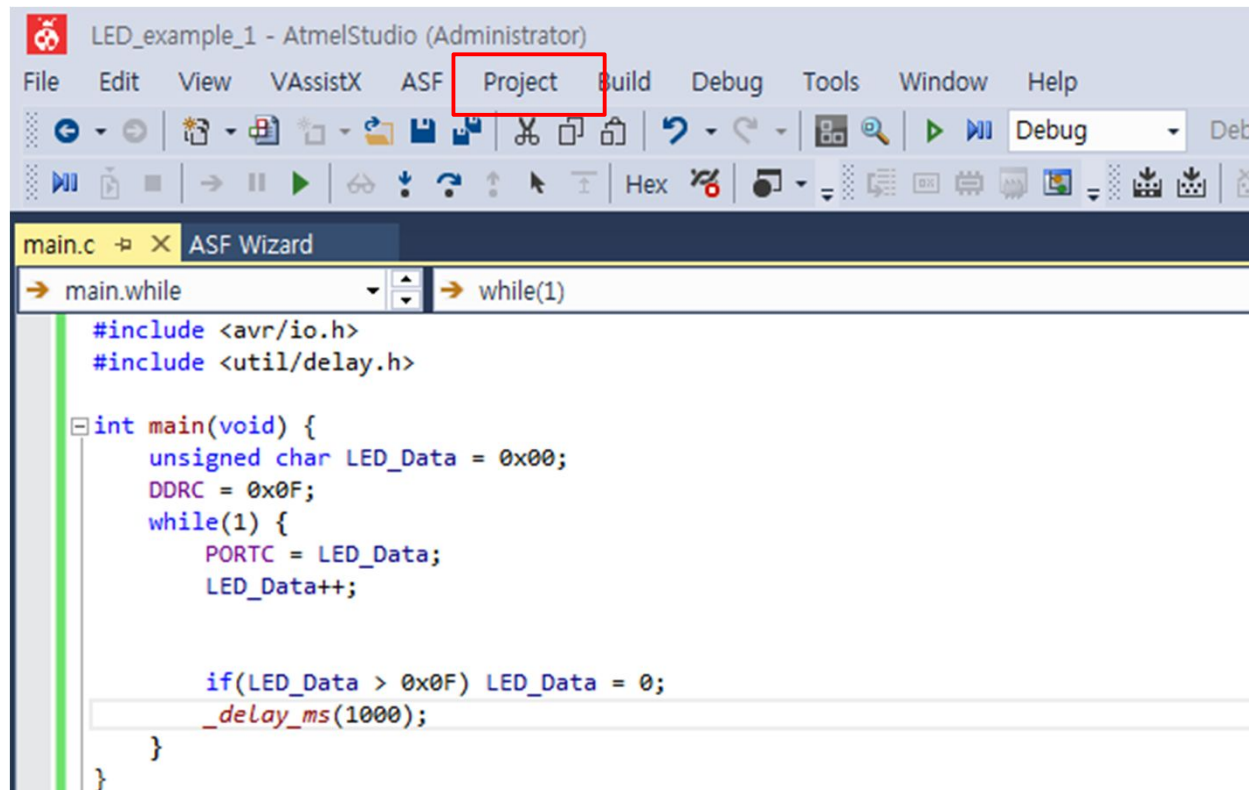
3) 응용 : LED를 좌우로 이동시키기

- 03_LED_Application 프로젝트 생성 완료



3) 응용 : LED를 좌우로 이동시키기

- 프로젝트 설정
 - Project 탭에서 "03_LED_Application Properties..." 를 선택



The screenshot shows the Atmel Studio (Administrator) interface. The 'Project' menu item in the top toolbar is highlighted with a red rectangle. Below the toolbar, the 'main.c' file is open, and the 'ASF Wizard' is visible. The code in the editor is as follows:

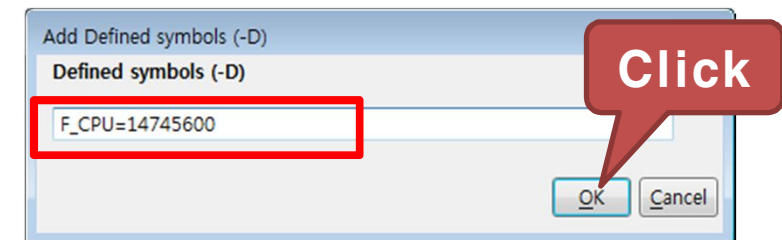
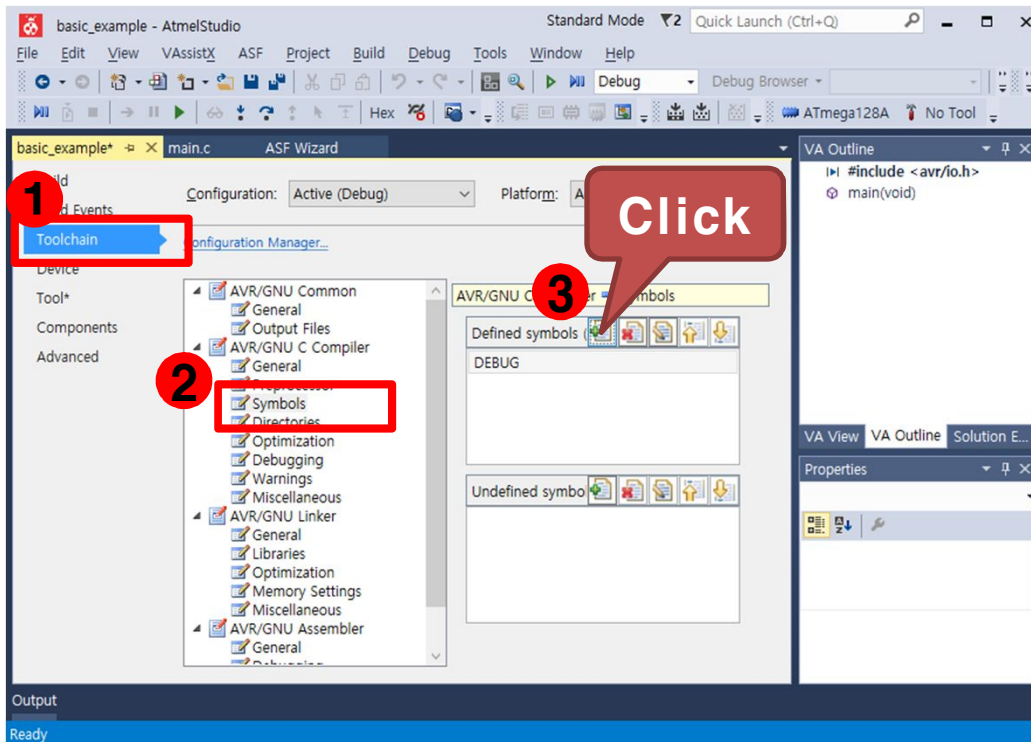
```
main.c x ASF Wizard
main.while while(1)
#include <avr/io.h>
#include <util/delay.h>

int main(void) {
    unsigned char LED_Data = 0x00;
    DDRC = 0x0F;
    while(1) {
        PORTC = LED_Data;
        LED_Data++;

        if(LED_Data > 0x0F) LED_Data = 0;
        _delay_ms(1000);
    }
}
```

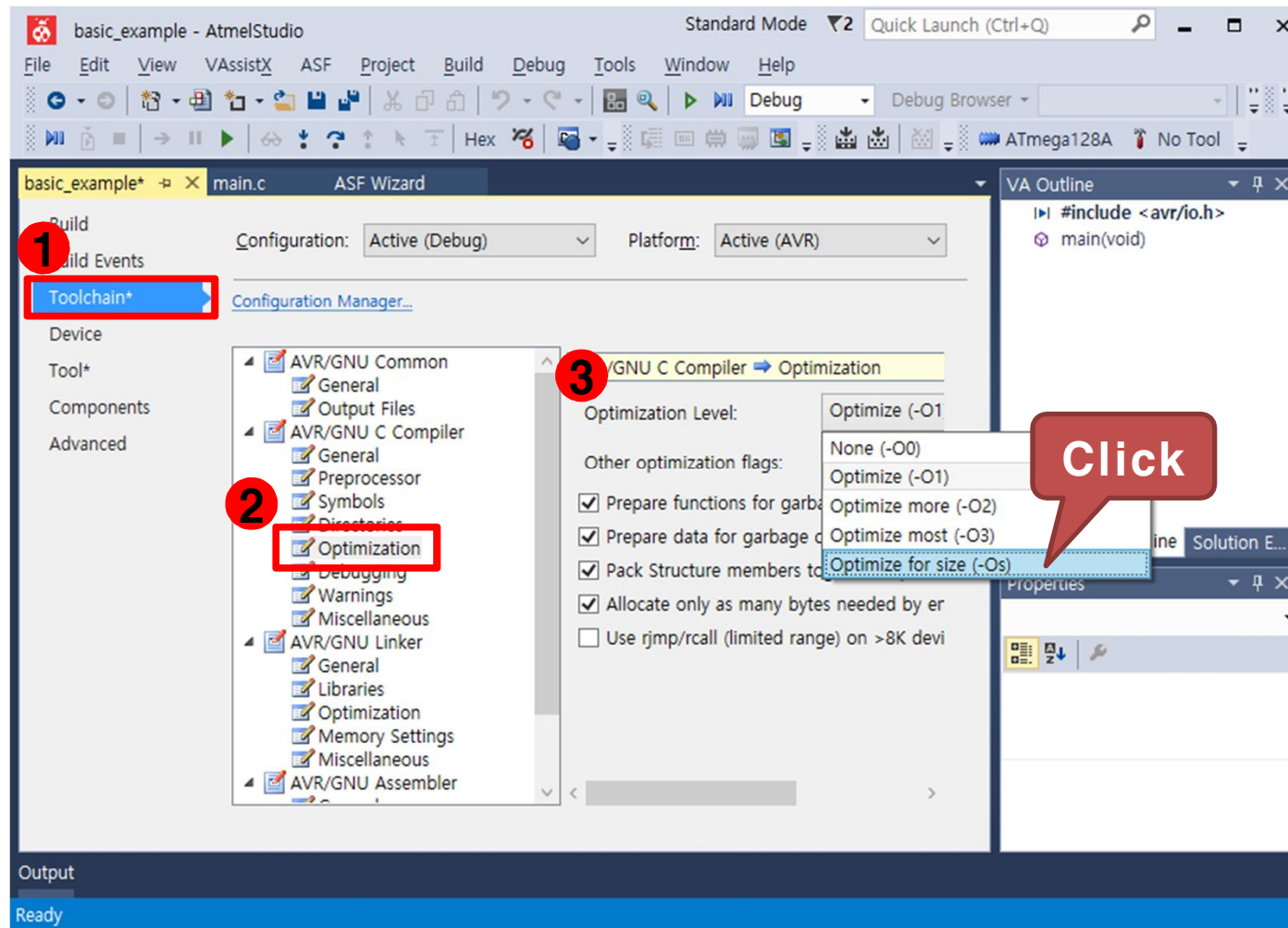
3) 응용 : LED를 좌우로 이동시키기

- Toolchain → AVR/GNU C Compiler → Symbols를 선택한 후 우측의 Defined symbols에서 "Add Item" 을 선택
- "F_CPU=14745600"을 입력하고 OK를 클릭. AVR의 외부 클럭 14.7456Mhz



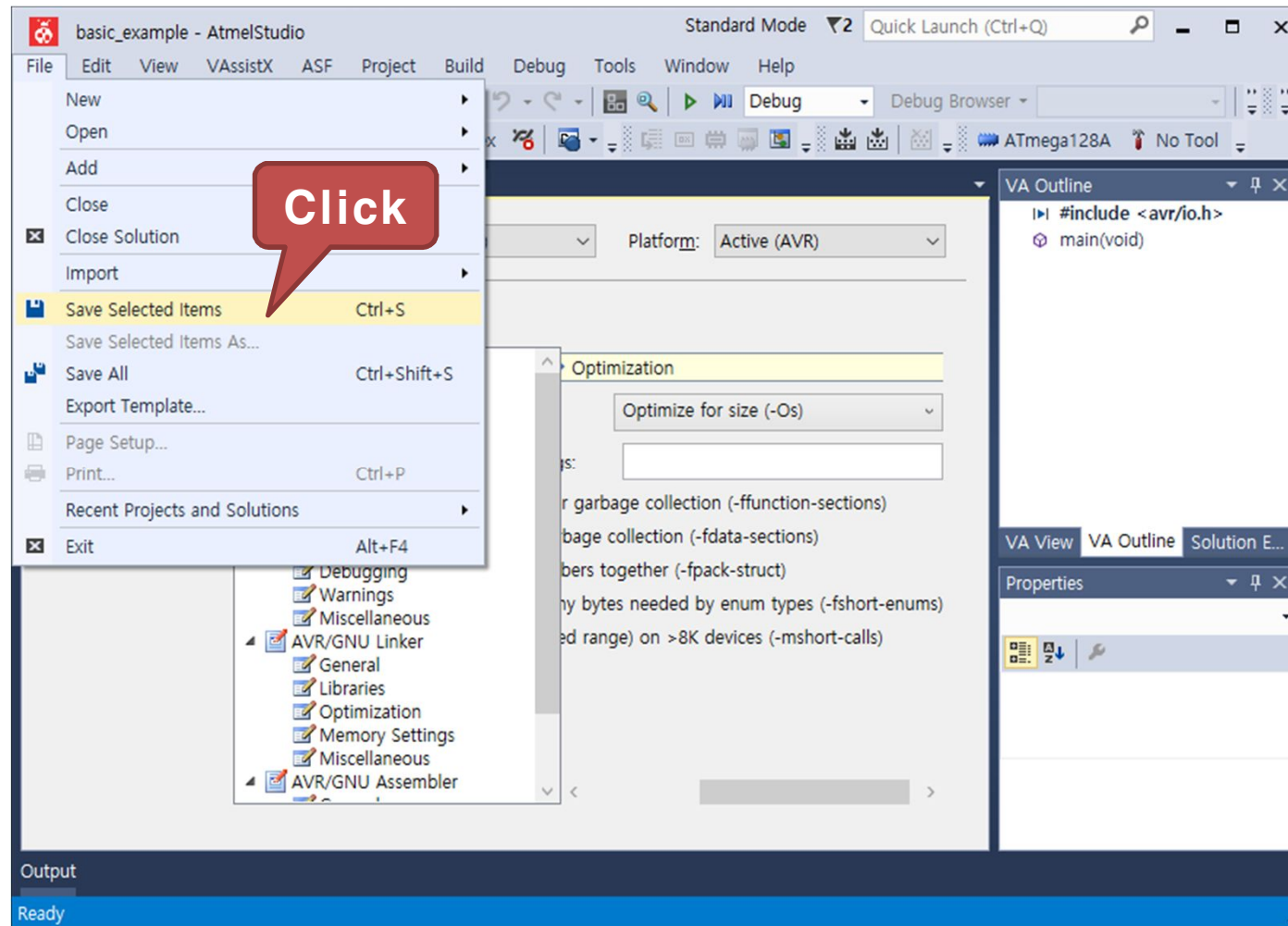
3) 응용 : LED를 좌우로 이동시키기

- Toolchain → AVR/GNU C Compiler → Optimization 을 선택한 후 우측의 Optimization Level 을 변경, "Optimize for size (-Os)" 선택



3) 응용 : LED를 좌우로 이동시키기

- File 탭에서 "Save Selected Items"를 클릭하여 설정값을 저장



3) 응용 : LED를 좌우로 이동시키기

- 구동 프로그램
 - main.c 코드 작성

```
#include <avr/io.h>      // AVR 입출력에 대한 헤더 파일
#include <util/delay.h>  // delay 함수사용을 위한 헤더파일

int main(void) {
    unsigned char LED_Data = 0x01, i;
    DDRC = 0x0F;          // 포트C를 출력포트로 설정
    while(1) {
        LED_Data = 0x01;  // 변수 LED_Data에 초기값 0x01로 저장

        // LED0~LED3으로 이동
        for(i=0; i<4; i++)
        {
            PORTC = LED_Data; // 포트C로 변수 LED_Data에 있는 데이터 출력
            LED_Data <<= 1;    // 좌측 쉬프트 연산자를 이용
            _delay_ms(1000);   // ms 단위의 딜레이 함수
        }
    }
}
```

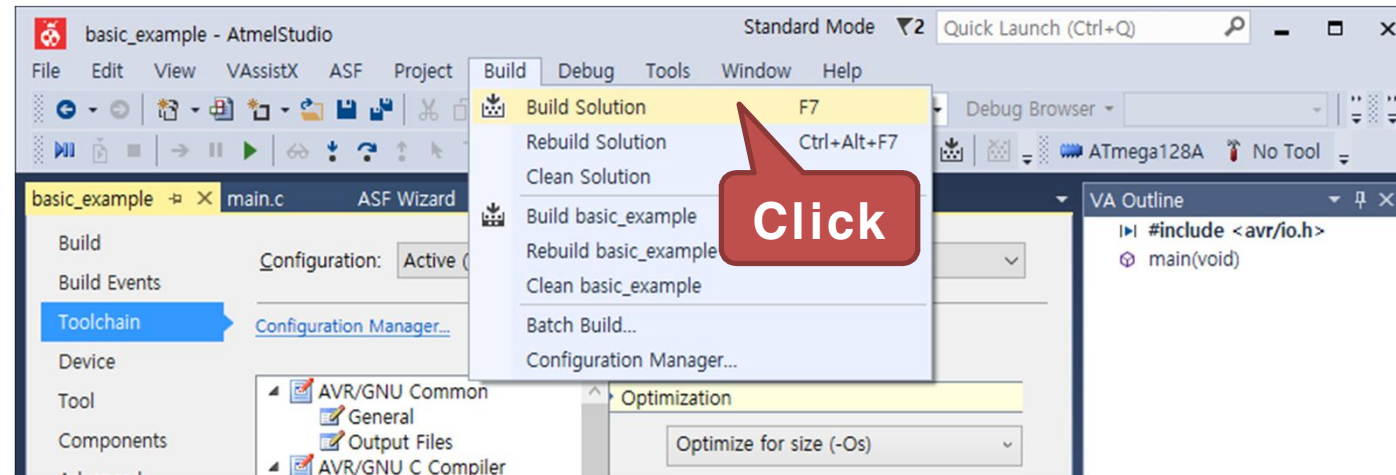
3) 응용 : LED를 좌우로 이동시키기

- 구동 프로그램
 - main.c 코드 작성

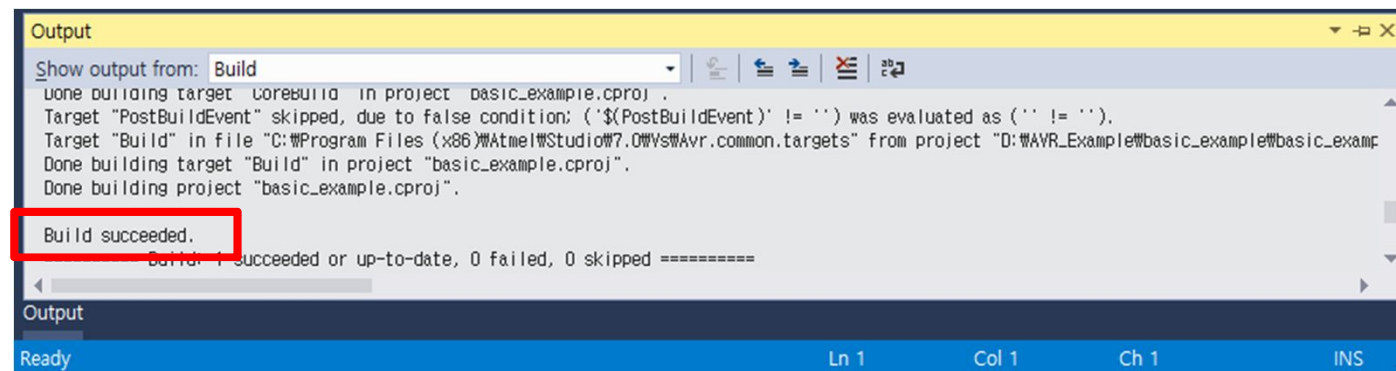
```
// LED3~LED0으로 이동
for(i=0; i<4; i++)
{
    PORTC = LED_Data; // 포트C로 변수 LED_Data에 있는 데이터 출력
    LED_Data >>= 1;    // 우측 쉬프트 연산자를 이용
    _delay_ms(1000);   // ms 단위의 딜레이 함수
}
}
```


3) 응용 : LED를 좌우로 이동시키기

- Build 탭에서 "Build Solution" 을 클릭하여 컴파일을 실행

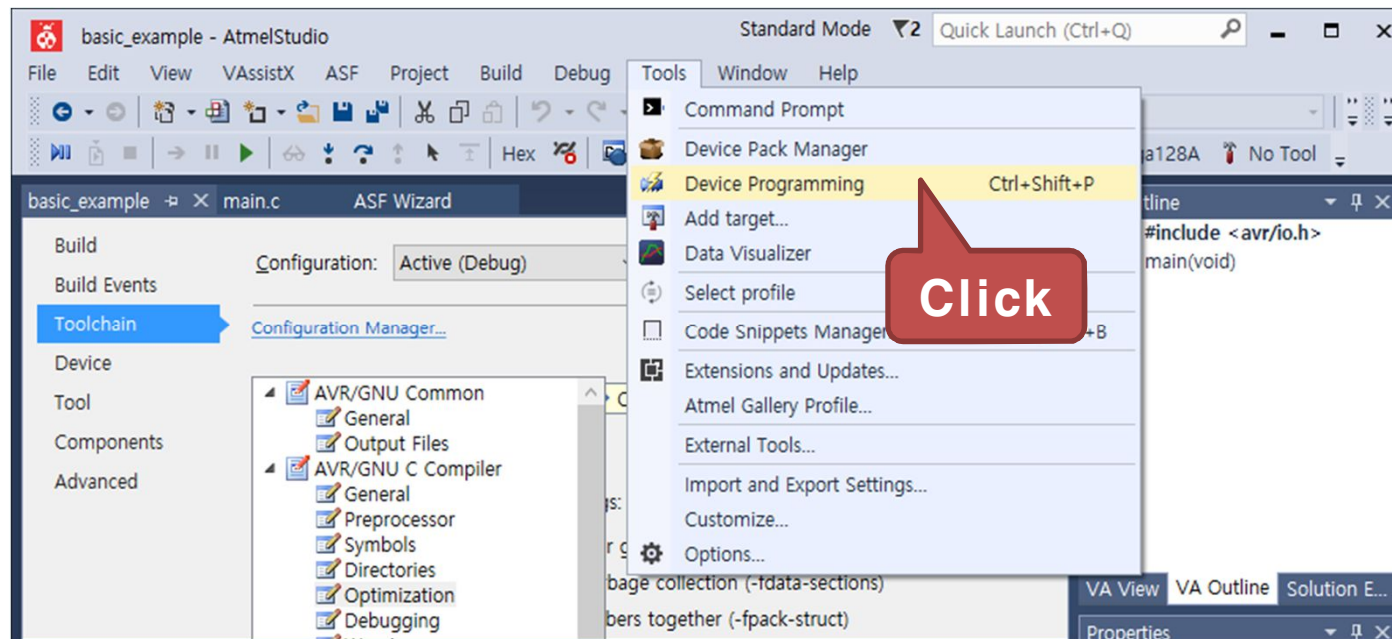


- 다음과 같이 Output 창에 "Build succeeded" 가 출력되면 컴파일이 완료



3) 응용 : LED를 좌우로 이동시키기

- Tools 탭에서 "Device Programming" 을 클릭



3) 응용 : LED를 좌우로 이동시키기

- Device Programming 창이 나타나면 Tool 에서 AVRISP mkII를 선택



- Device를 ATmega128A로 선택하고 "Apply" 버튼을 클릭

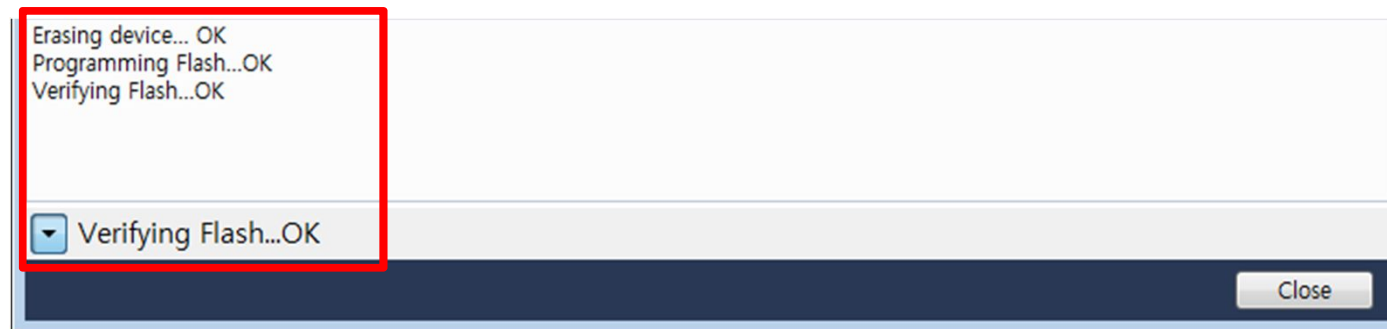


3) 응용 : LED를 좌우로 이동시키기

- Device 인식이 완료되면 Memories 탭을 선택하고, "Program" 버튼을 클릭

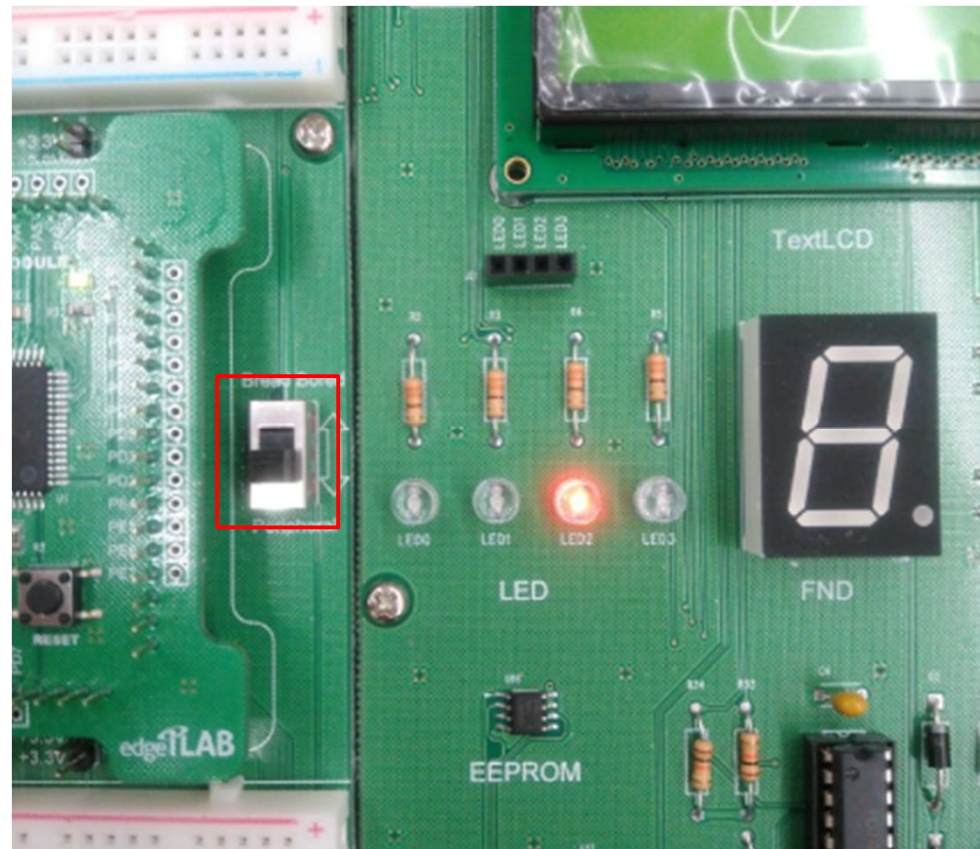


- 프로그램 다운로드가 성공하면 다음과 같은 메시지가 출력된다.



3) 응용 : LED를 좌우로 이동시키기

- 실행 결과
 - LED의 불이 좌우로 이동
 - 보드 옆 스위치를 Peripheral(주변장치)로 이동

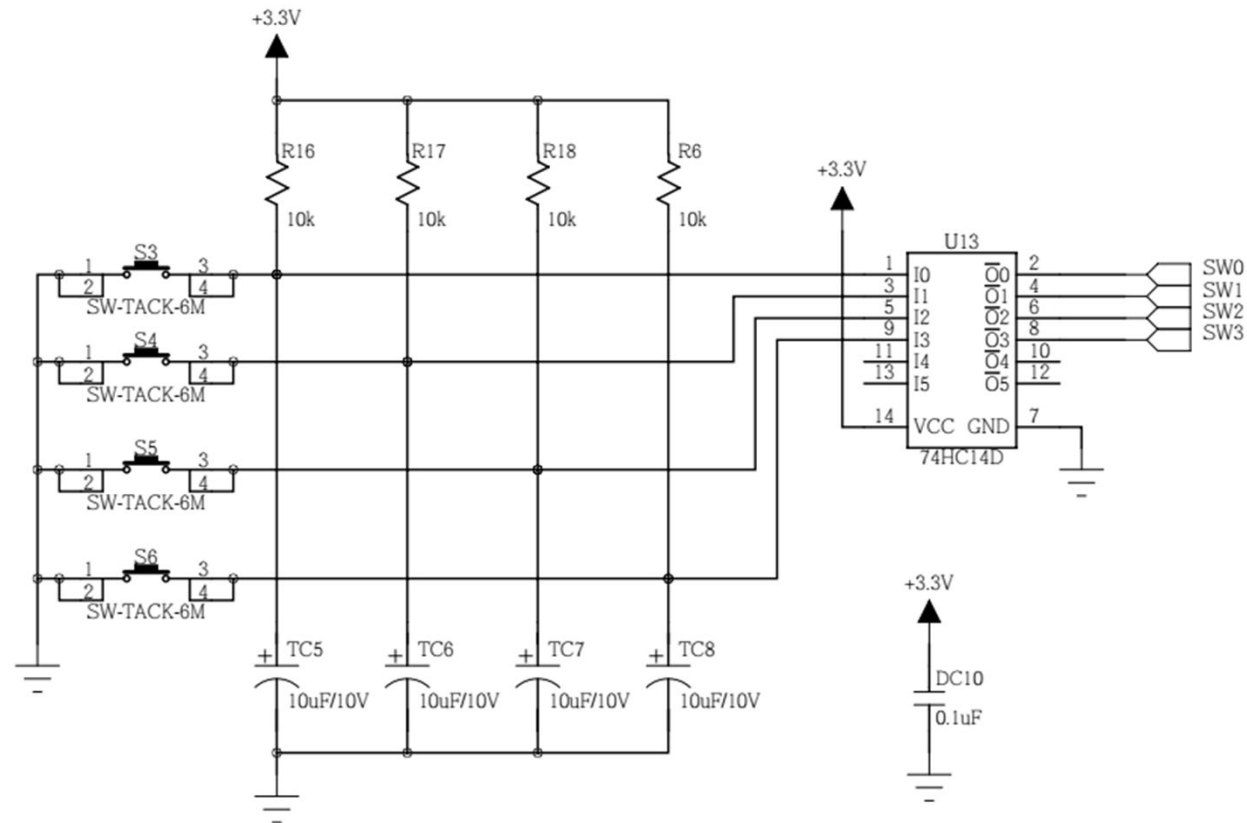


4)예제 : GPIO를 이용하여 스위치 입력 받기

- 실습 개요
 - GPIO 포트를 통해 신호를 입력하여 그 신호에 따라 LED 점등
 - 스위치 모듈의 스위치를 누르면 해당되는 LED 모듈의 LED가 점등되도록 함
 - 입출력 포트를 스위치쪽은 입력으로 LED쪽은 출력으로 설정하도록 함
- 실습 목표
 - GPIO 입출력 포트의 방향 제어 및 입력 제어 방법 습득
 - 스위치 동작원리 습득

4)예제 : GPIO를 이용하여 스위치 입력 받기

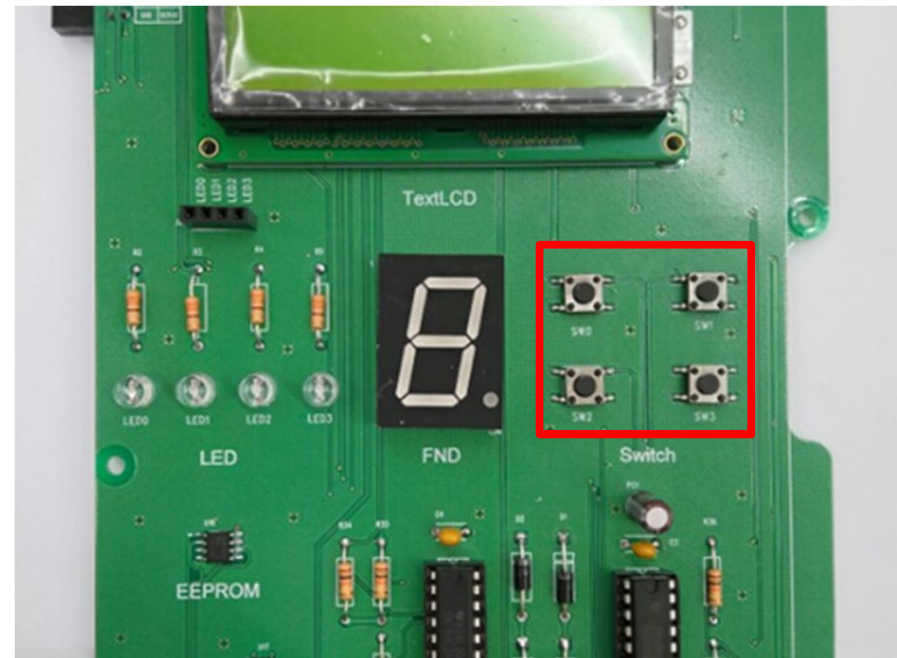
- 사용 모듈의 회로
 - Push Button 모듈



4)예제 : GPIO를 이용하여 스위치 입력 받기

- Edge-MCU AVR
 - SWITCH는 E 포트에 연결

Sensor	GPIO
SW0	PE4
SW1	PE5
SW2	PE6
SW3	PE7

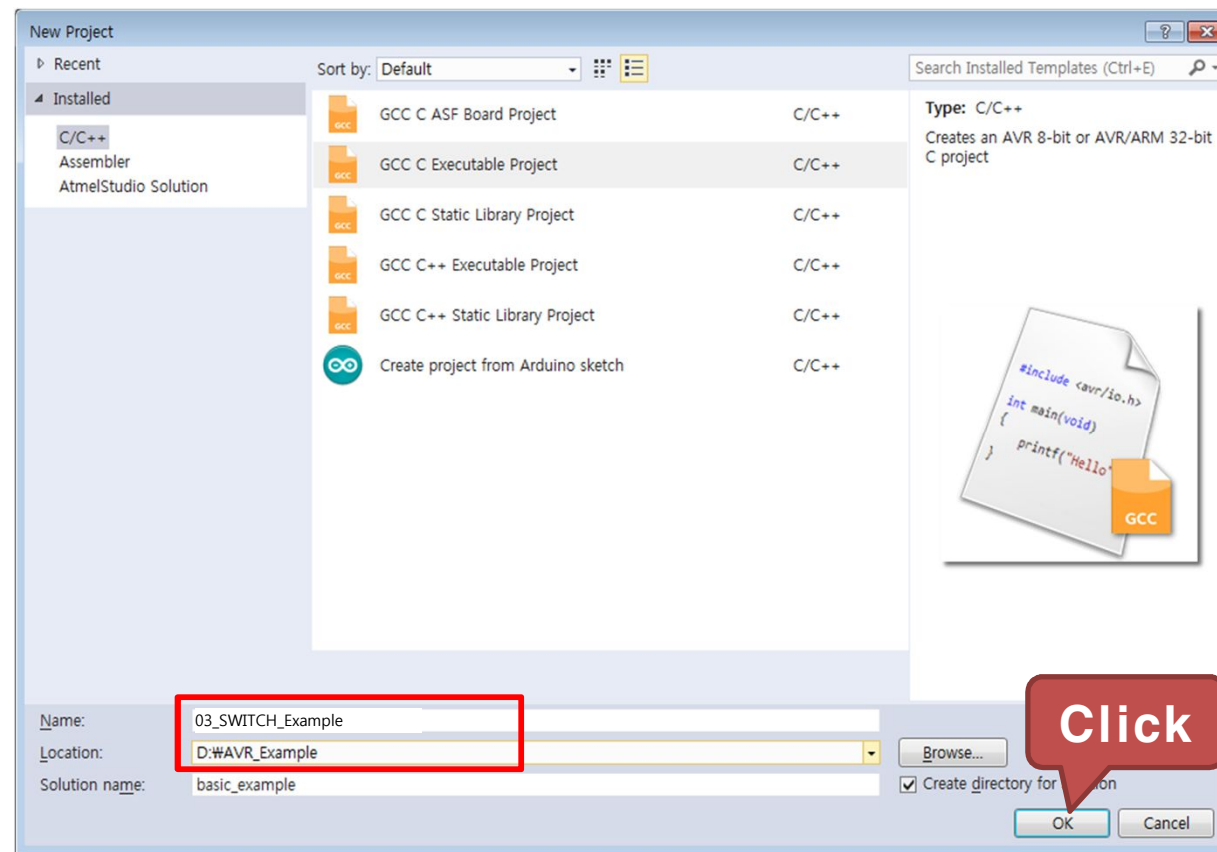


4)예제 : GPIO를 이용하여 스위치 입력 받기

- 구동 프로그램 : 사전 지식
 - 스위치를 누르면 '1' 신호가 나오고, 놓으면 '0' 신호가 나옴
 - 이 신호를 입력 받기 위해서는
 - MCU의 입출력 포트를 입력으로 선언해야 함
 - 즉, 입력으로 사용하기로 한 MCU E 포트를 입력으로 선언해야 함
 - 입출력 포트를 입력으로 선언하려면 DDRx 레지스터 (여기서는 E 포트를 사용하므로 DDRE 레지스터)에 '0'으로 설정
 - 스위치 모듈의 버튼을 누른다면 PINx 레지스터(여기서는 E 포트를 사용하므로 PINE 레지스터)에 '1'이라는 값이 입력되어 들어옴
 - LED 출력 방법은 앞의 예제와 동일

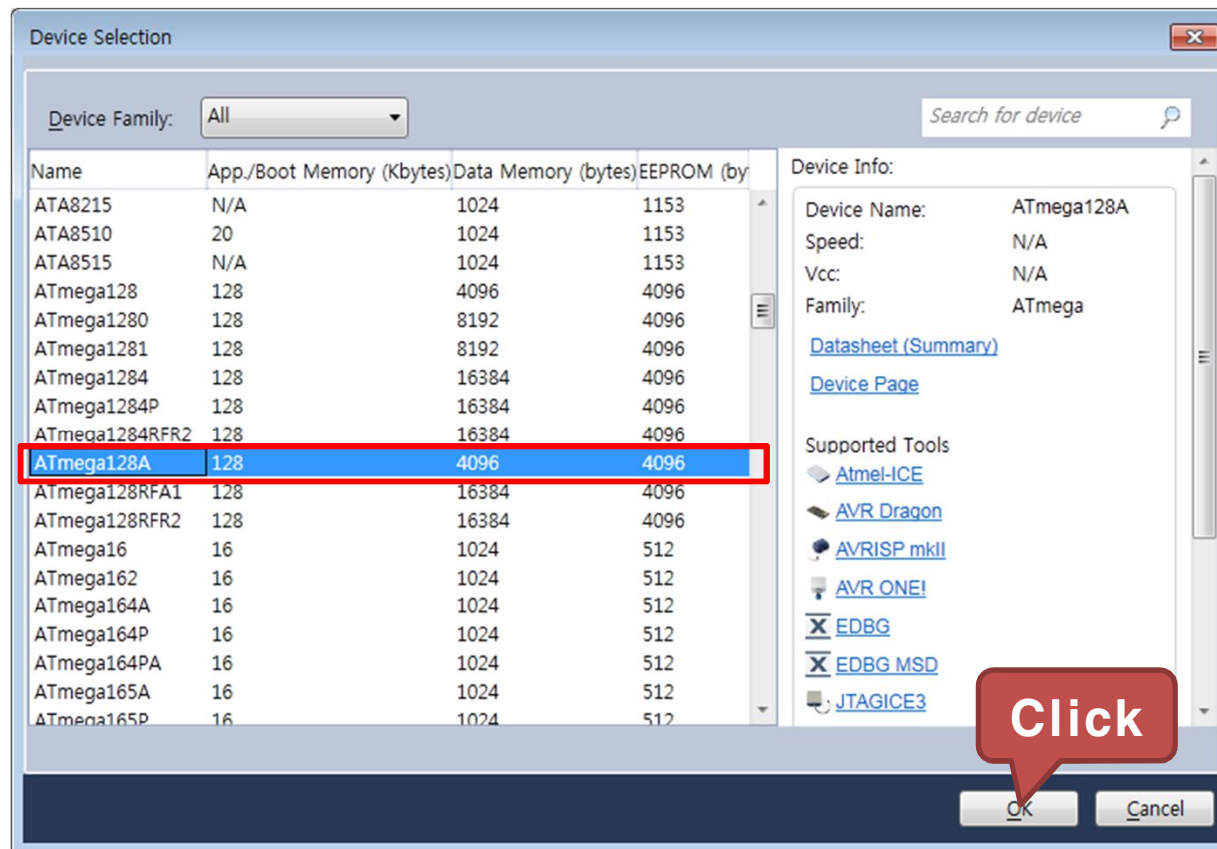
4)예제 : GPIO를 이용하여 스위치 입력 받기

- GCC C Executable Project를 선택하고, 생성된 Project의 Name 과 Location을 설정
 - Name : 03_SWITCH_Example
 - Location : D:\AVR_Example



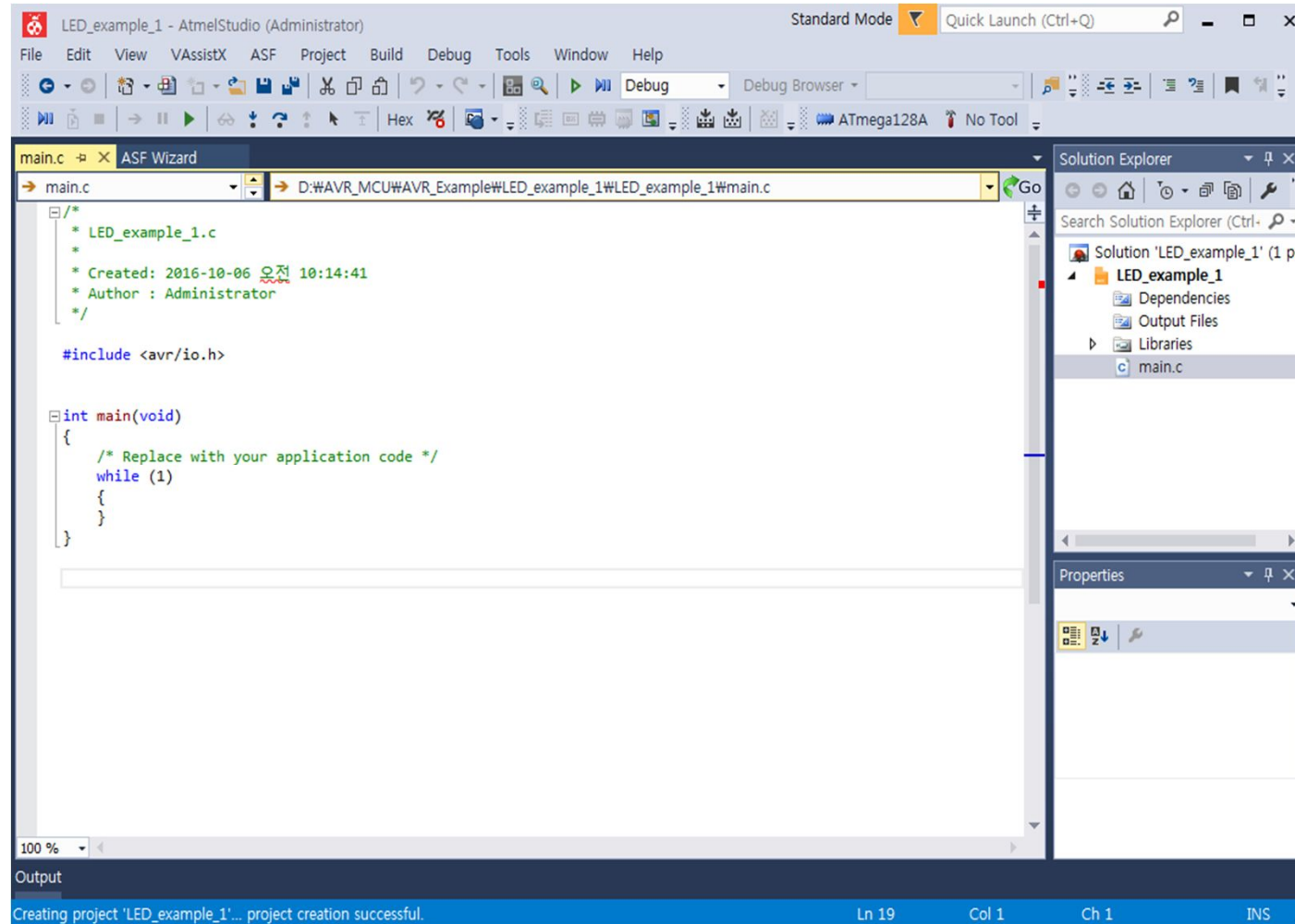
4)예제 : GPIO를 이용하여 스위치 입력 받기

- ATmega128A를 선택하고 OK를 클릭



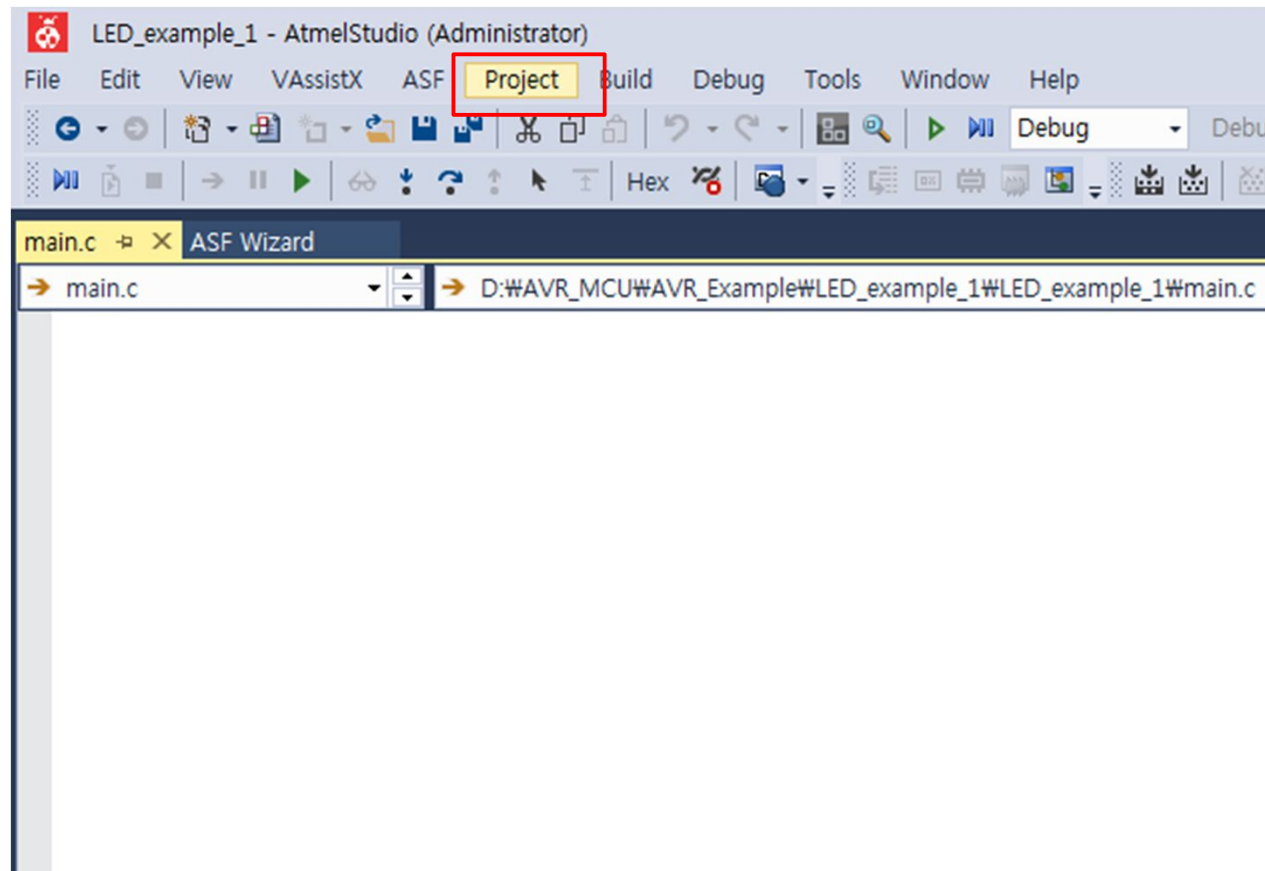
4)예제 : GPIO를 이용하여 스위치 입력 받기

- 03_SWITCH_Example 프로젝트 생성 완료



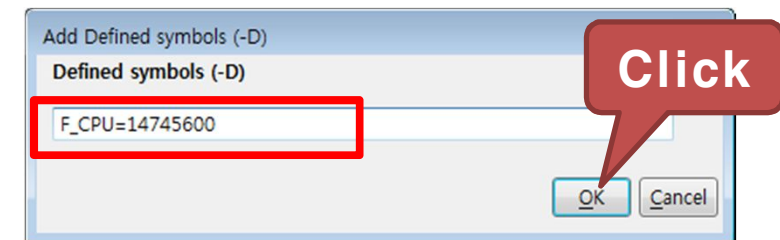
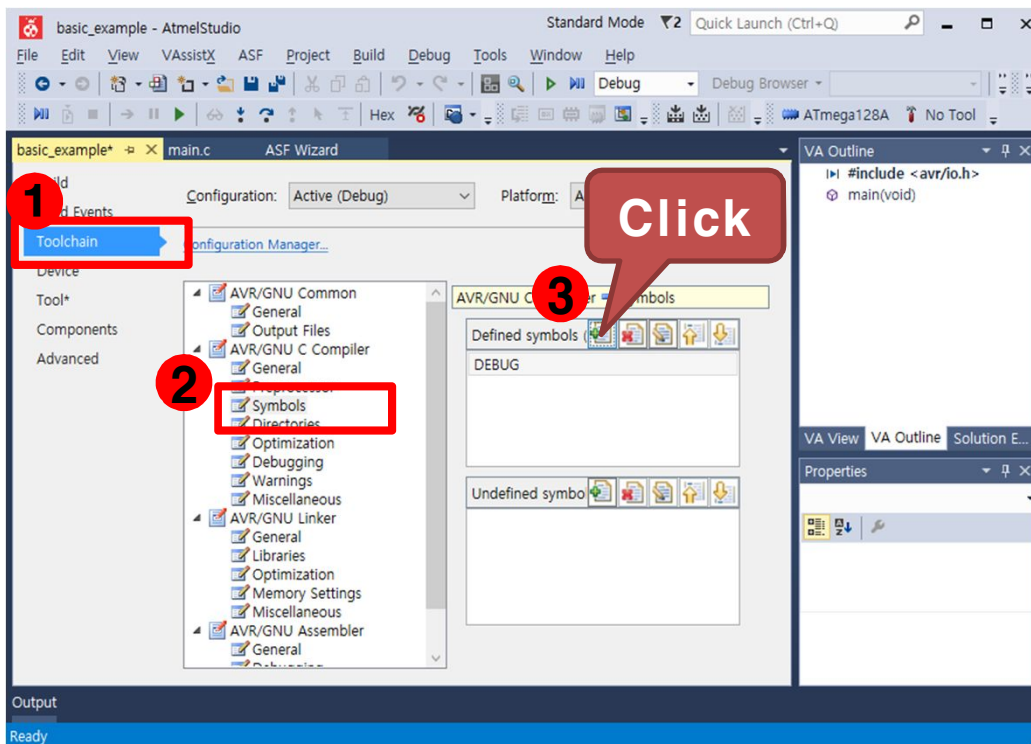
4)예제 : GPIO를 이용하여 스위치 입력 받기

- 프로젝트 설정
 - Project 탭에서 "03_SWITCH_Example Properties..." 를 선택



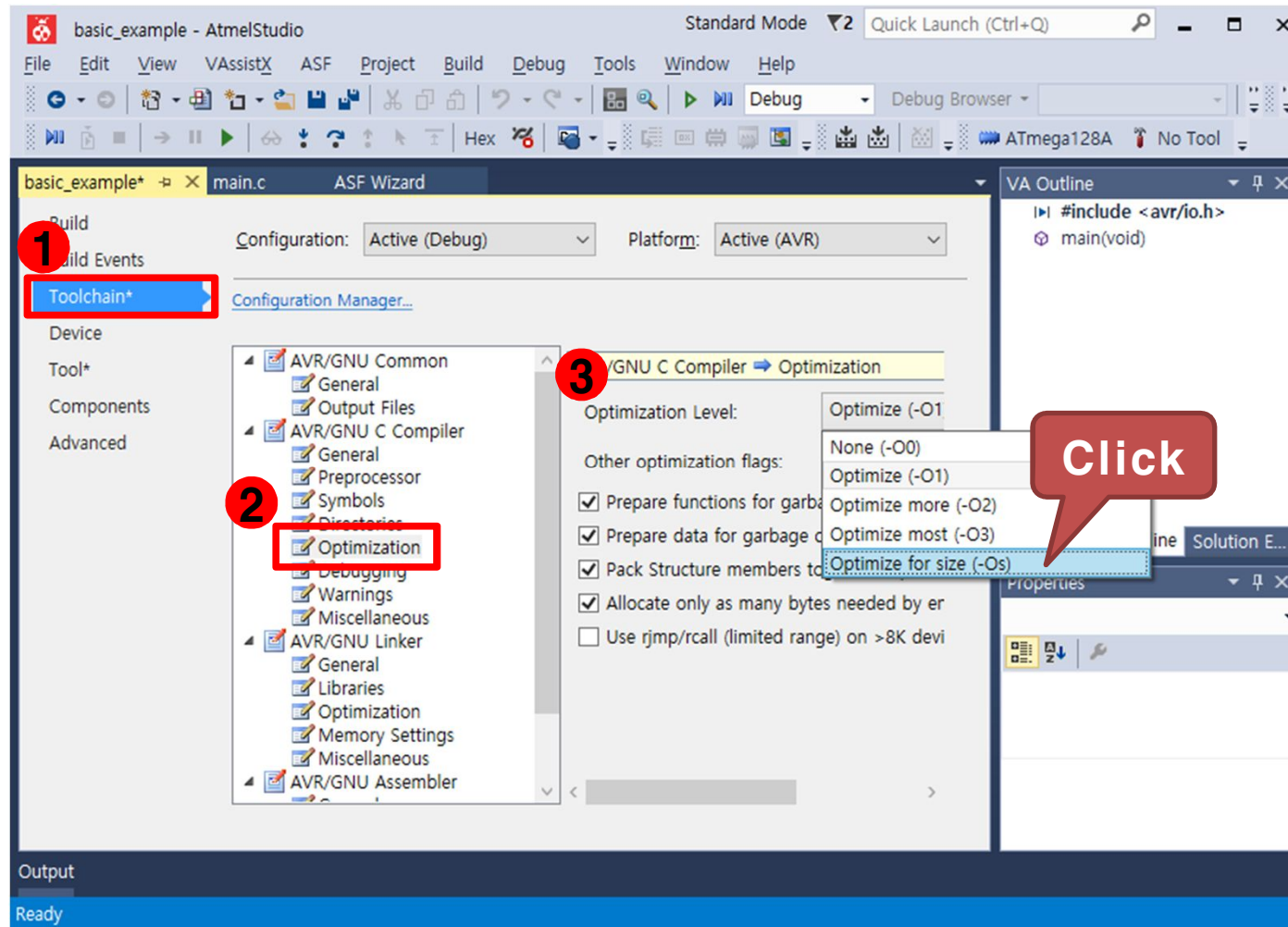
4)예제 : GPIO를 이용하여 스위치 입력 받기

- Toolchain → AVR/GNU C Compiler → Symbols를 선택한 후 우측의 Defined symbols에서 "Add Item" 을 선택
- "F_CPU=14745600"을 입력하고 OK를 클릭. AVR의 외부 클럭 14.7456Mhz



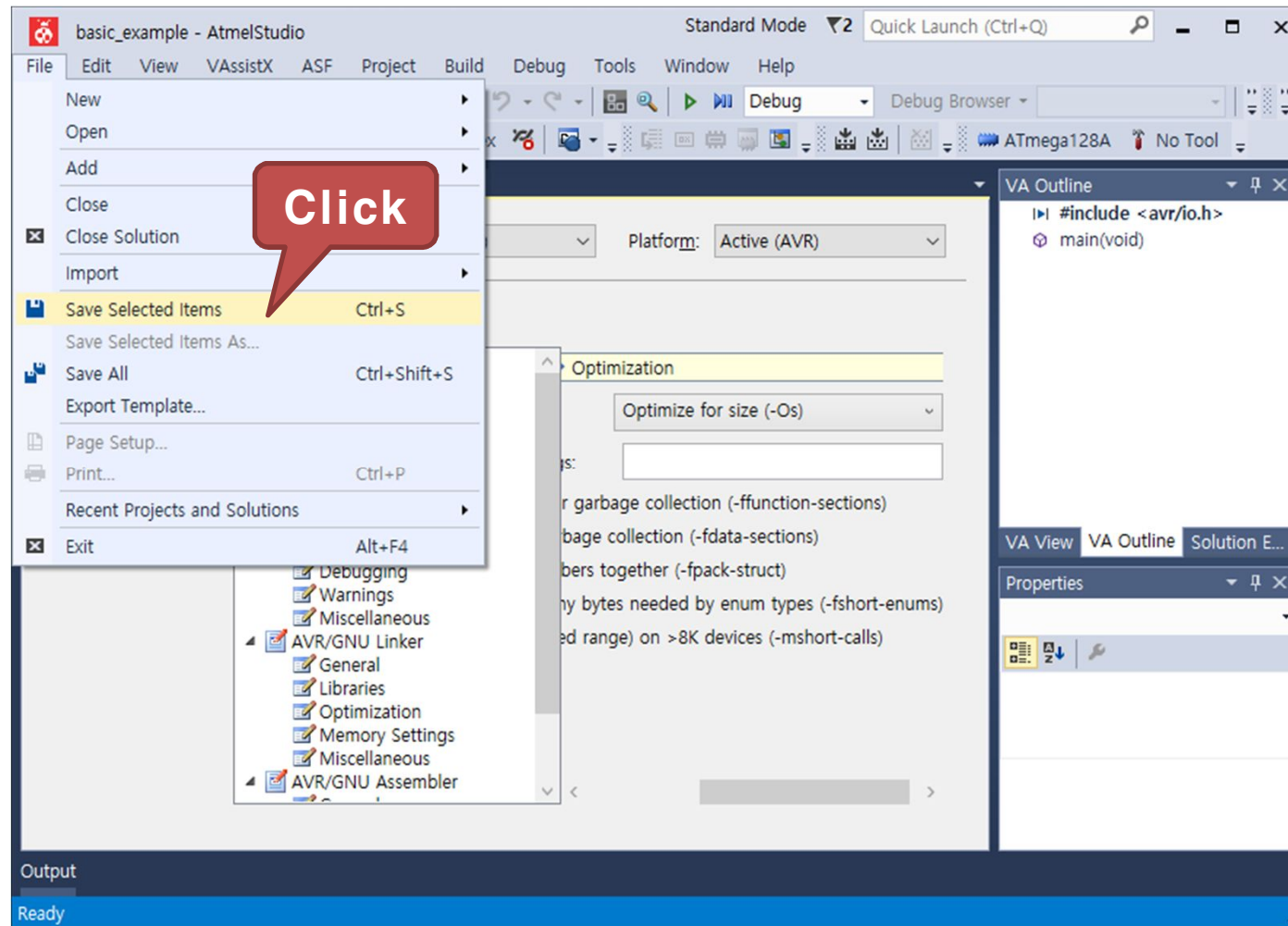
4)예제 : GPIO를 이용하여 스위치 입력 받기

- Toolchain → AVR/GNU C Compiler → Optimization 을 선택한 후 우측의 Optimization Level 을 변경, "Optimize for size (-Os)" 선택



4)예제 : GPIO를 이용하여 스위치 입력 받기

- File 탭에서 "Save Selected Items"를 클릭하여 설정값을 저장



4)예제 : GPIO를 이용하여 스위치 입력 받기

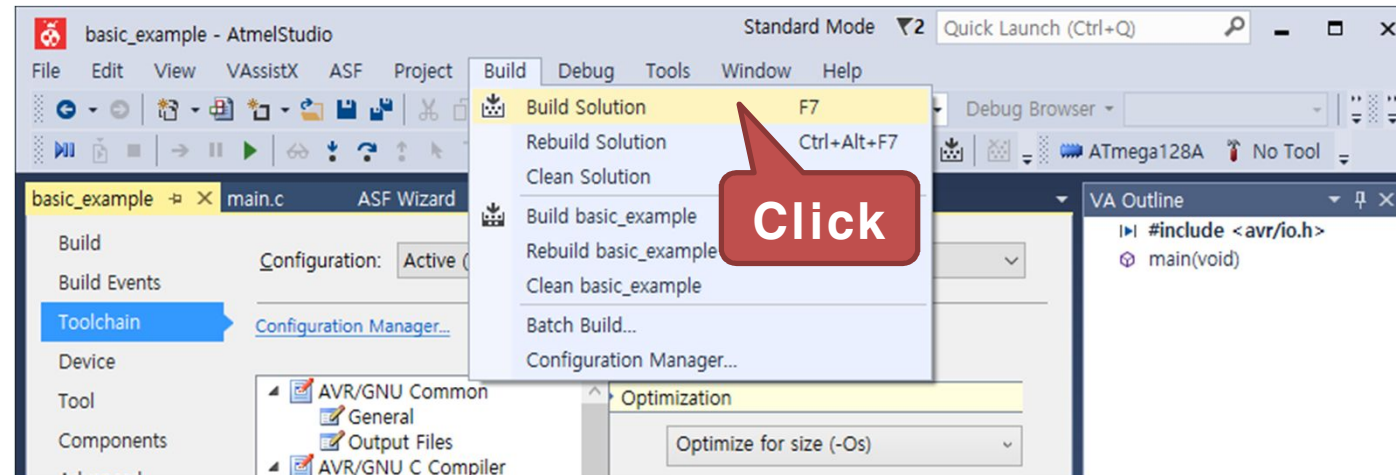
- 구동 프로그램
 - main.c 코드 작성

```
#include <avr/io.h>      // AVR 입출력에 대한 헤더 파일

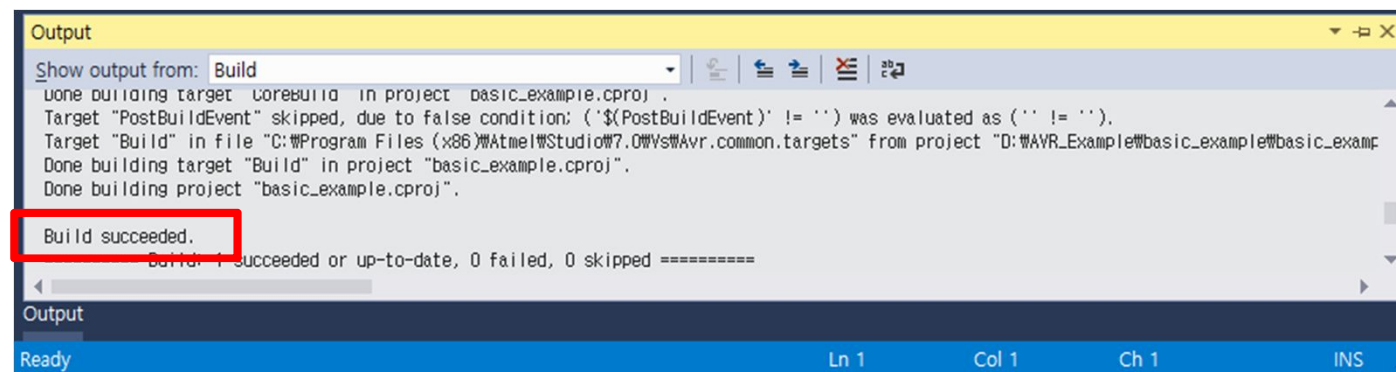
int main(void)
{
    DDRC = 0x0F;          // 포트C 를 출력포트로 설정
    DDRE = 0x00;          // 포트E 를 입력포트로 설정
    while (1)
    {
        PORTC = PINE >> 4; // 포트 E로 입력받은 값을 포트C로 출력
        // 입력핀이 포트 E의 상위 비트이므로 우측으로 4비트 쉬프트
    }
}
```

4)예제 : GPIO를 이용하여 스위치 입력 받기

- Build 탭에서 "Build Solution" 을 클릭하여 컴파일을 실행

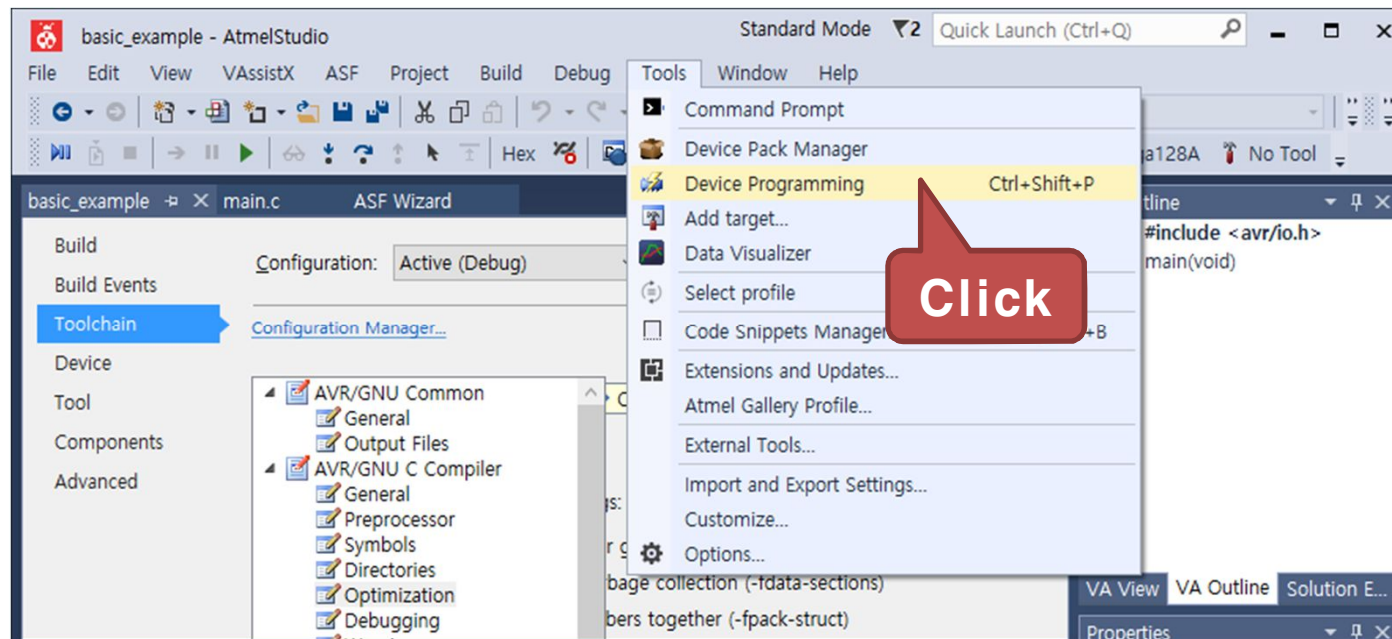


- 다음과 같이 Output 창에 "Build succeeded" 가 출력되면 컴파일이 완료



4)예제 : GPIO를 이용하여 스위치 입력 받기

- Tools 탭에서 "Device Programming" 을 클릭



4)예제 : GPIO를 이용하여 스위치 입력 받기

- Device Programming 창이 나타나면 Tool 에서 AVRISP mkII를 선택



- Device를 ATmega128A로 선택하고 "Apply" 버튼을 클릭

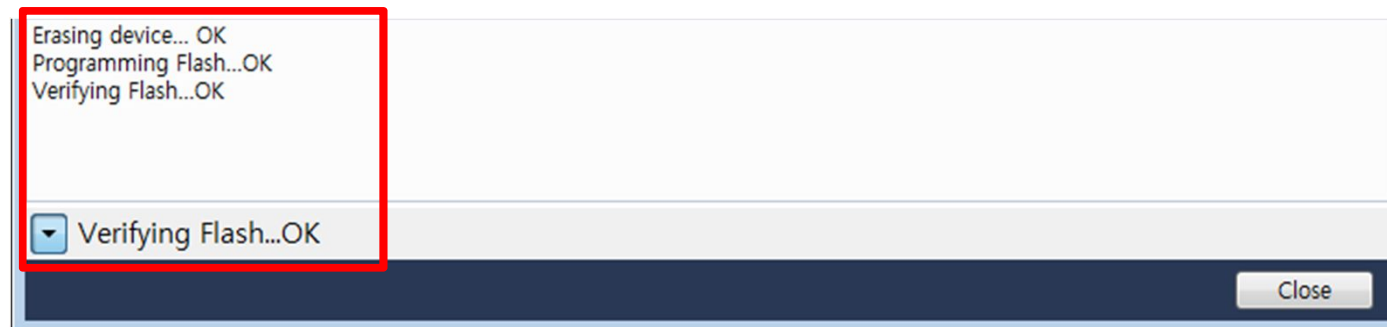


4)예제 : GPIO를 이용하여 스위치 입력 받기

- Device 인식이 완료되면 Memories 탭을 선택하고, "Program" 버튼을 클릭

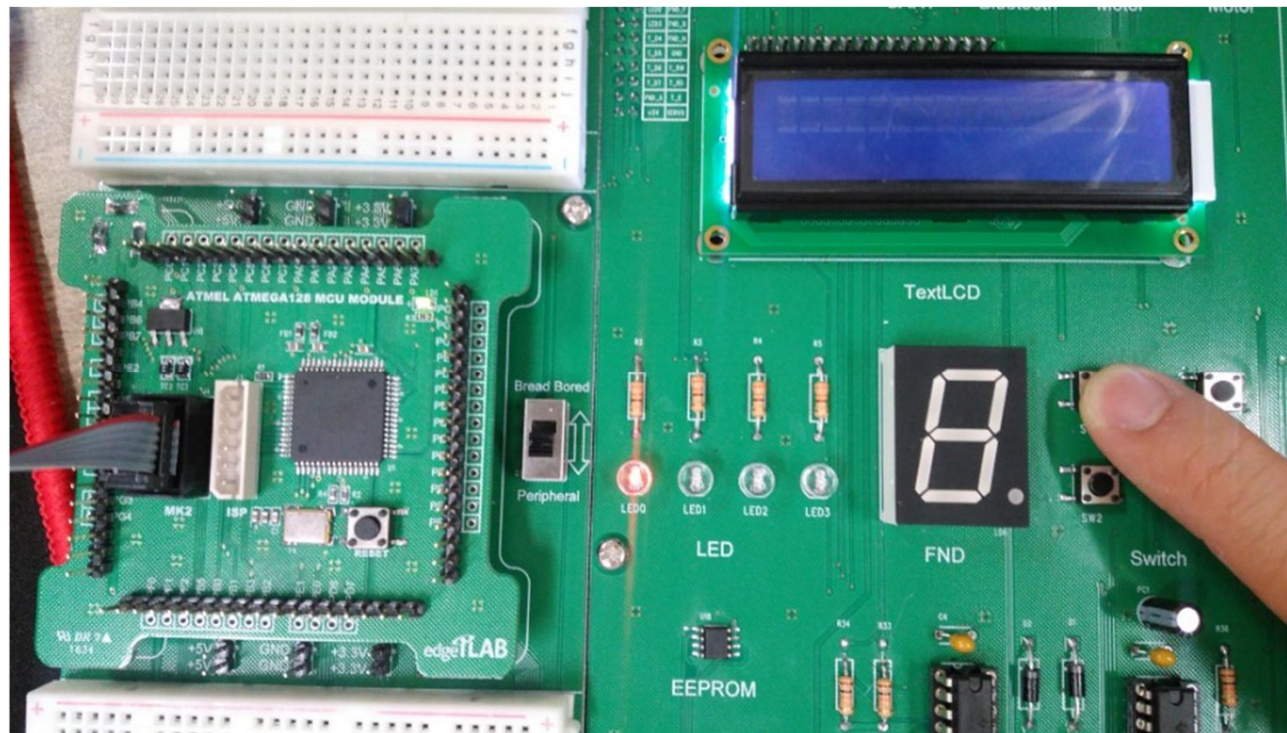


- 프로그램 다운로드가 성공하면 다음과 같은 메시지가 출력된다.



4)예제 : GPIO를 이용하여 스위치 입력 받기

- 실행 결과
 - Switch모듈의 눌려진 버튼과 같은 LED의 불이 점등
 - 각 버튼을 누르면서 확인

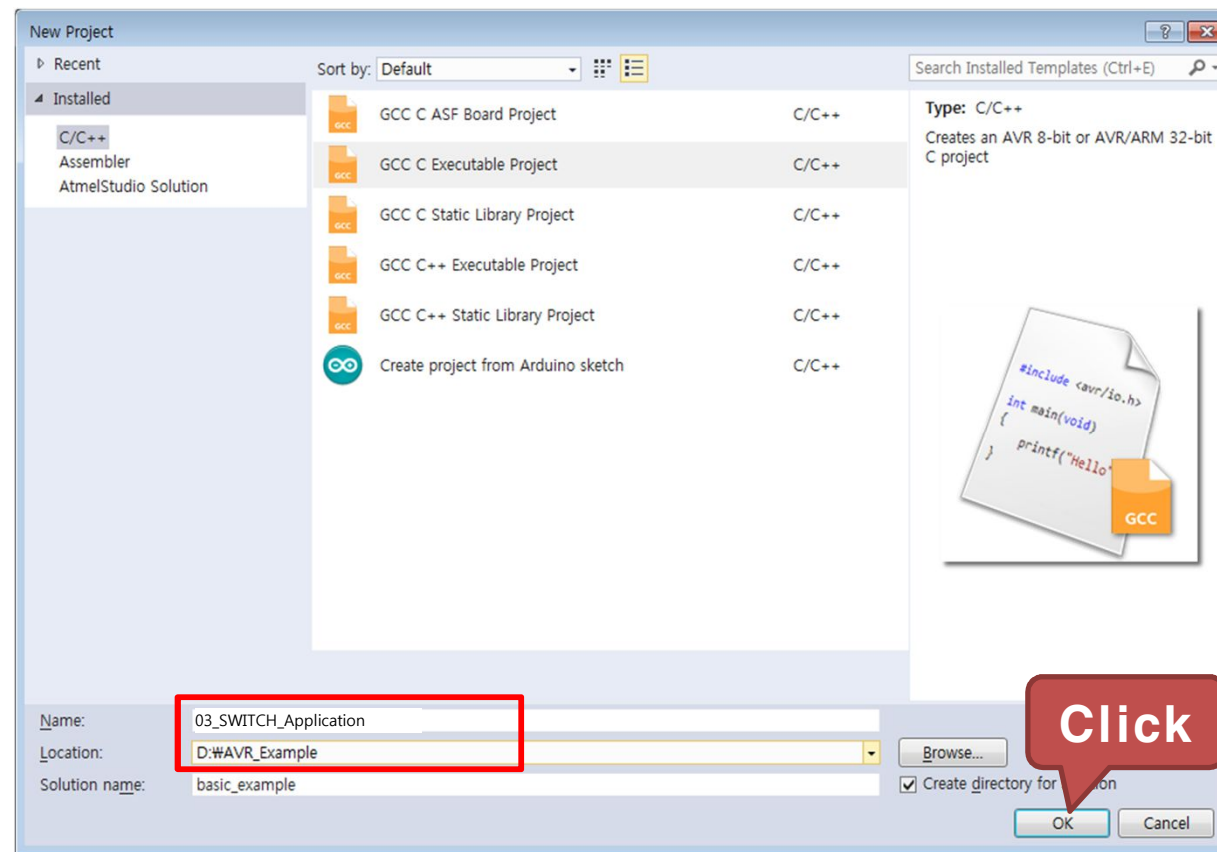


5)응용 : 스위치의 입력값에 따라 LED 점멸시간 변경하기

- 실습 개요
 - GPIO 포트를 통해 신호를 입력하여 그 신호에 따라 LED 점멸시간 변경
 - 입출력 포트를 스위치쪽은 입력으로 LED쪽은 출력으로 설정하도록 함
- 실습 목표
 - GPIO 입출력 포트의 방향 제어 및 입력 제어 방법 습득
 - 스위치 동작원리 습득

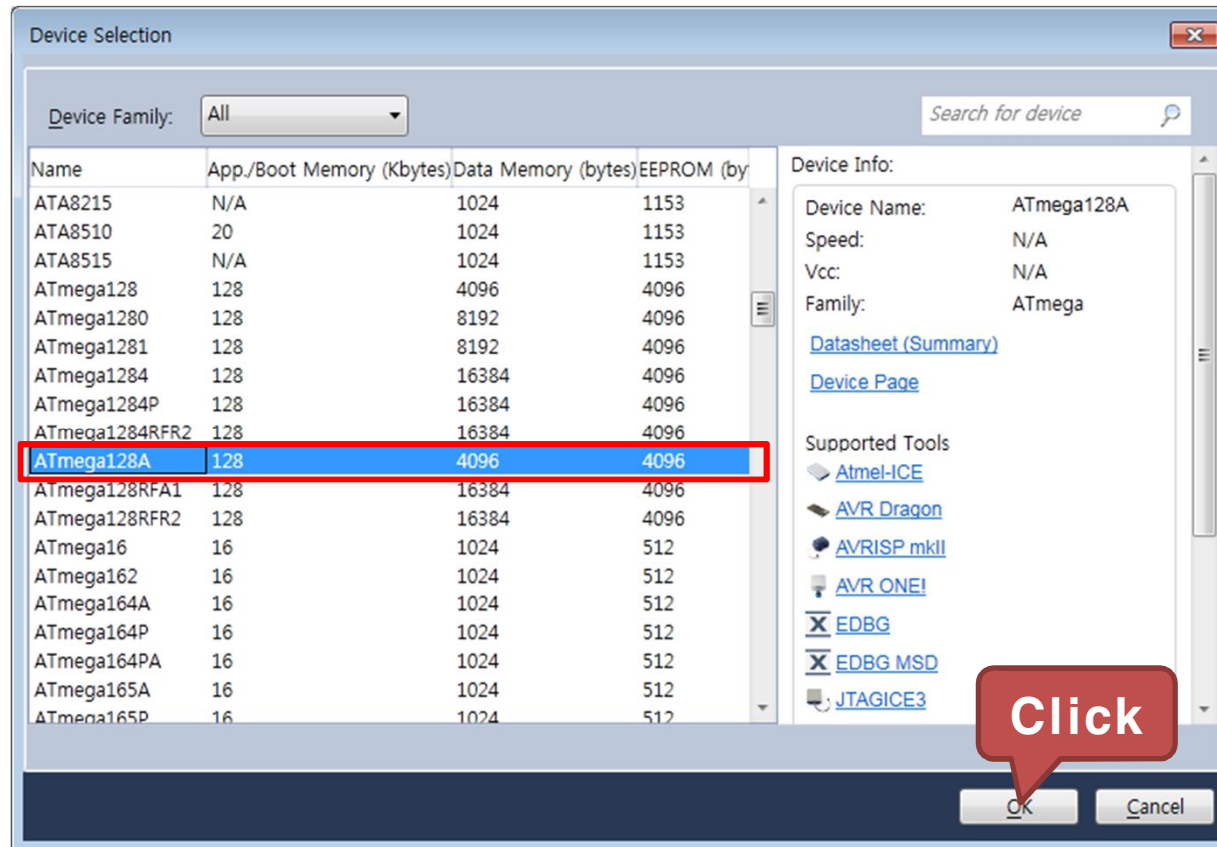
5) 응용 : 스위치의 입력값에 따라 LED 점멸시간 변경하기

- GCC C Executable Project를 선택하고, 생성된 Project의 Name 과 Location을 설정
 - Name : 03_SWITCH_Application
 - Location : D:\WAVR_Example



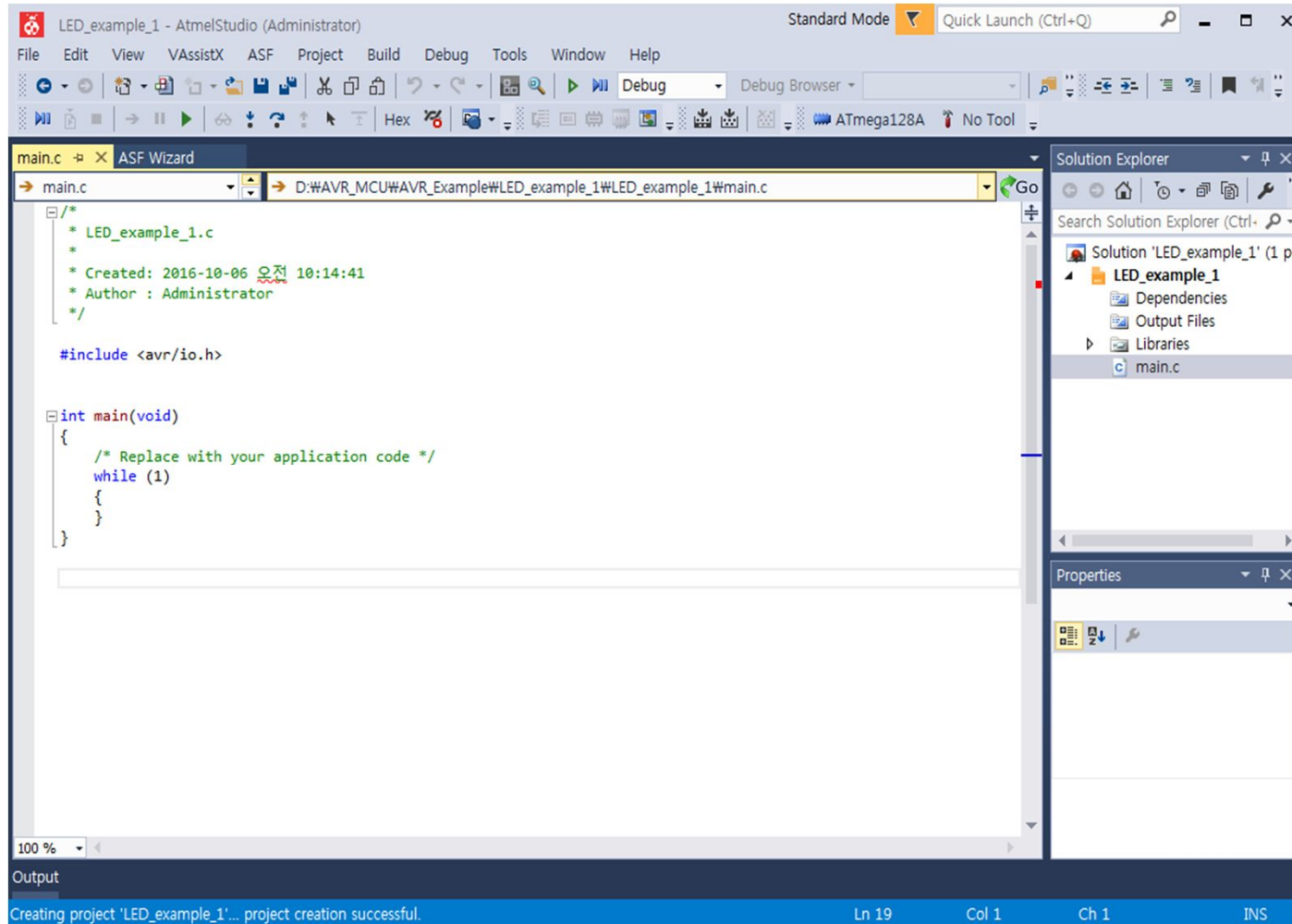
5) 응용 : 스위치의 입력값에 따라 LED 점멸시간 변경하기

- ATmega128A를 선택하고 OK를 클릭



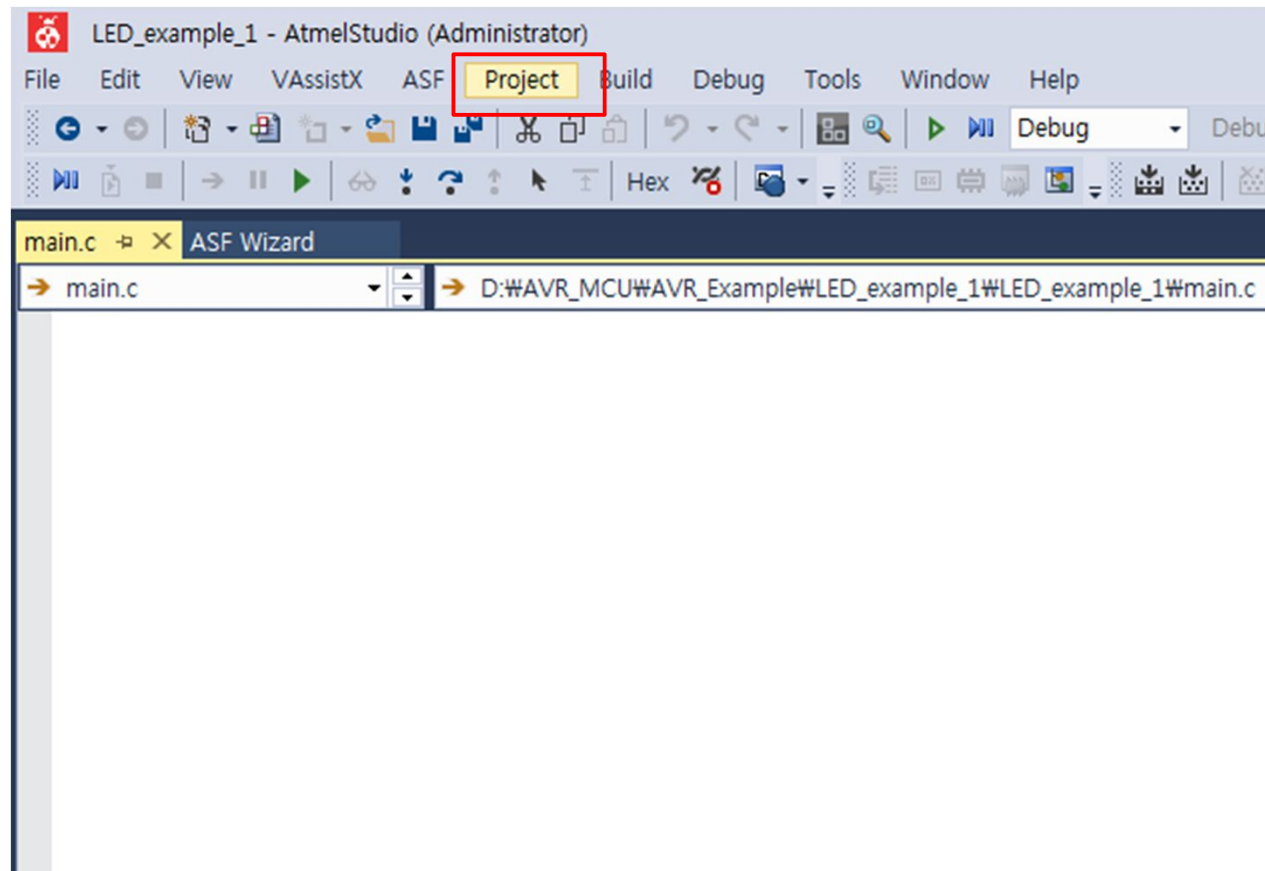
5) 응용 : 스위치의 입력값에 따라 LED 점멸시간 변경하기

- 03_SWITCH_Application 프로젝트 생성 완료



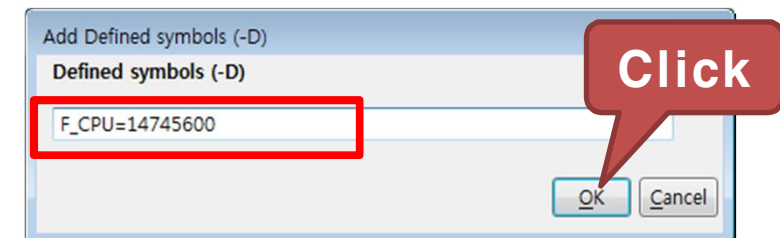
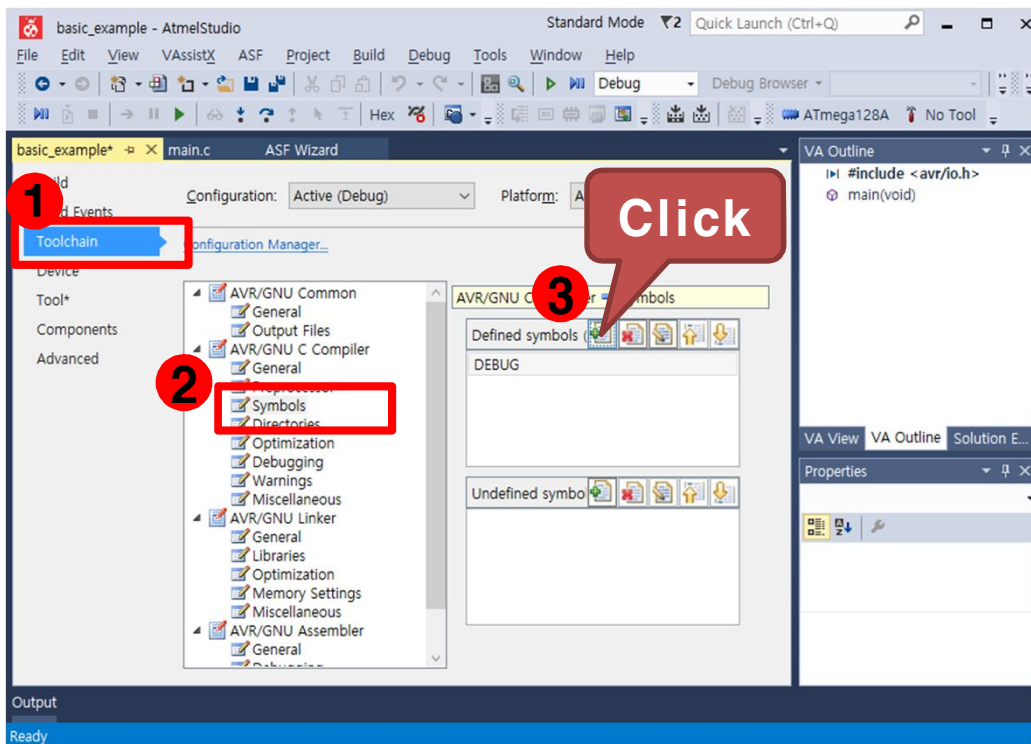
5) 응용 : 스위치의 입력값에 따라 LED 점멸시간 변경하기

- 프로젝트 설정
 - Project 탭에서 "03_SWITCH_Application Properties..." 를 선택



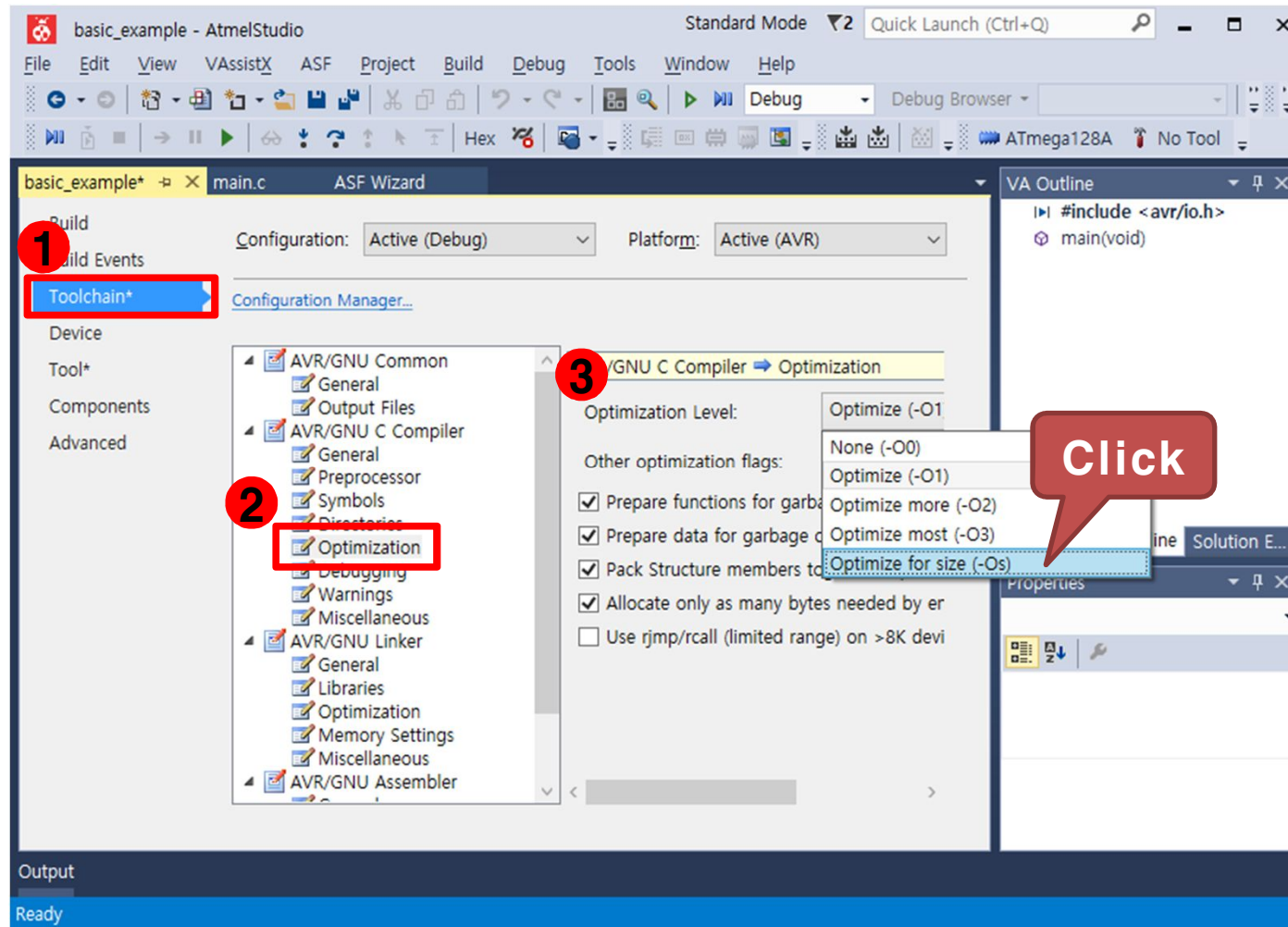
5) 응용 : 스위치의 입력값에 따라 LED 점멸시간 변경하기

- Toolchain → AVR/GNU C Compiler → Symbols를 선택한 후 우측의 Defined symbols에서 "Add Item" 을 선택
- "F_CPU=14745600"을 입력하고 OK를 클릭. AVR의 외부 클럭 14.7456Mhz



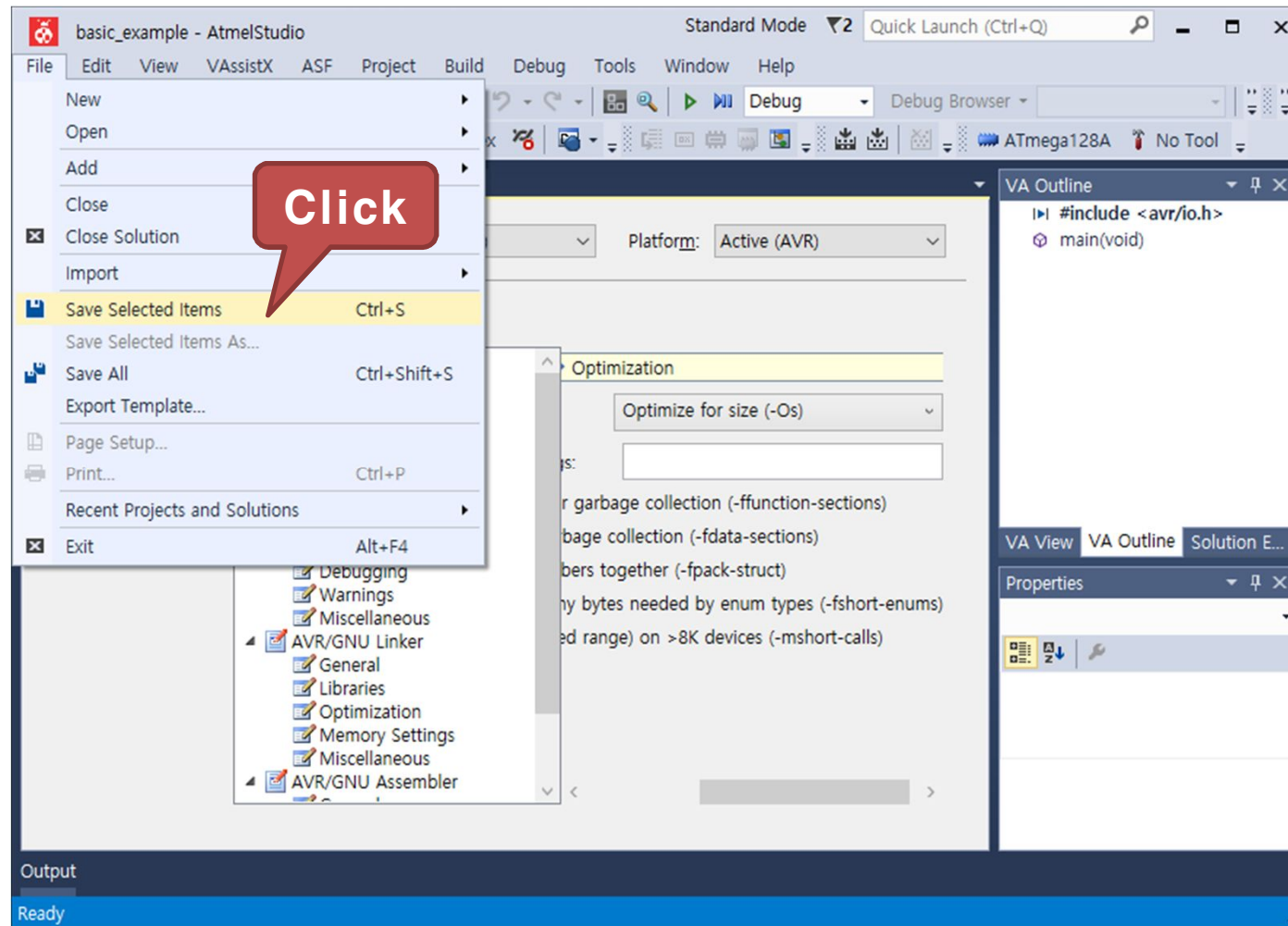
5) 응용 : 스위치의 입력값에 따라 LED 점멸시간 변경하기

- Toolchain → AVR/GNU C Compiler → Optimization 을 선택한 후 우측의 Optimization Level 을 변경, "Optimize for size (-Os)" 선택



5) 응용 : 스위치의 입력값에 따라 LED 점멸시간 변경하기

- File 탭에서 "Save Selected Items"를 클릭하여 설정값을 저장



5)응용 : 스위치의 입력값에 따라 LED 점멸시간 변경하기

- 구동 프로그램
 - main.c 코드 작성

```
#include <avr/io.h> //AVR 입출력에 대한 헤더 파일
#include <util/delay.h> //delay 함수사용을 위한 헤더파일

int main(void)
{
    unsigned char Switch_flag = 0, Switch_flag_pre = 0x01;

    DDRC = 0x0F;           // 포트C 를 출력포트로 설정
    DDRE = 0x00;           // 포트E 를 입력포트로 설정
    PORTC = 0x00;          // LED0 ~ 3을 OFF

    while (1)
    {
        // 포트 E로 입력받은 값을 변수 Switch_flag에 저장
        // 입력핀이 포트 E의 상위 비트이므로 우측으로 4비트 쉬프트
        Switch_flag = PINE >> 4;
```

5)응용 : 스위치의 입력값에 따라 LED 점멸시간 변경하기

- 구동 프로그램
 - main.c 코드 작성

```
// SW0 ~ 4 스위치중 하나만 눌렀을때
if((Switch_flag == 0x01) || (Switch_flag == 0x02) ||
   (Switch_flag == 0x04) || (Switch_flag == 0x08))
{
    // 눌린 상태 Switch_flag 값을 변수 Switch_flag_pre에 저장
    Switch_flag_pre = Switch_flag;
}

// PORTC(LED가 연결된 포트)의 하위 4비트를 반전
PORTC ^= 0x0F;
```

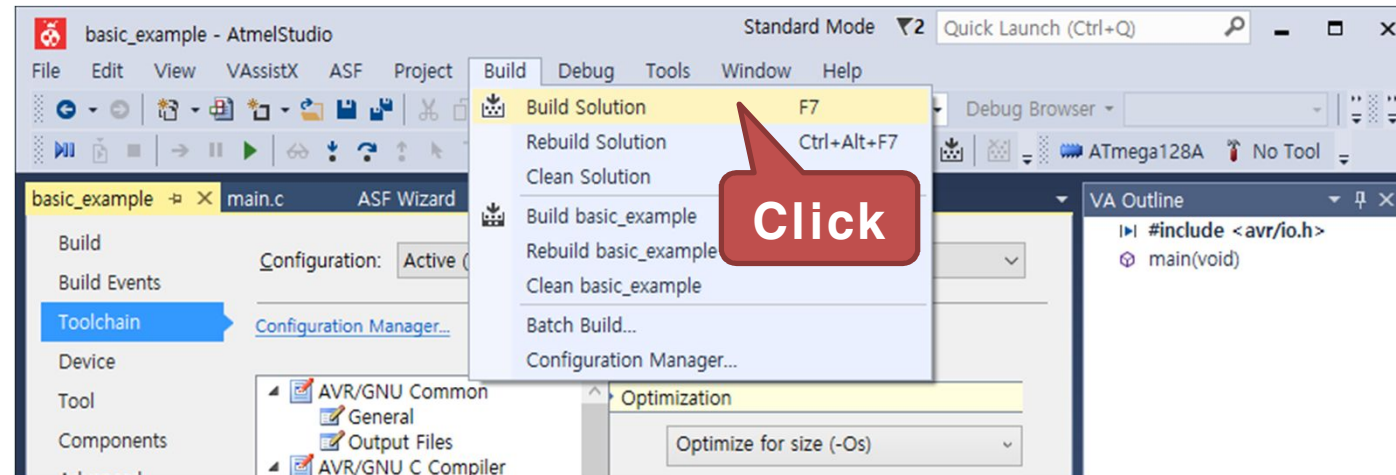
5) 응용 : 스위치의 입력값에 따라 LED 점멸시간 변경하기

- 구동 프로그램
 - main.c 코드 작성

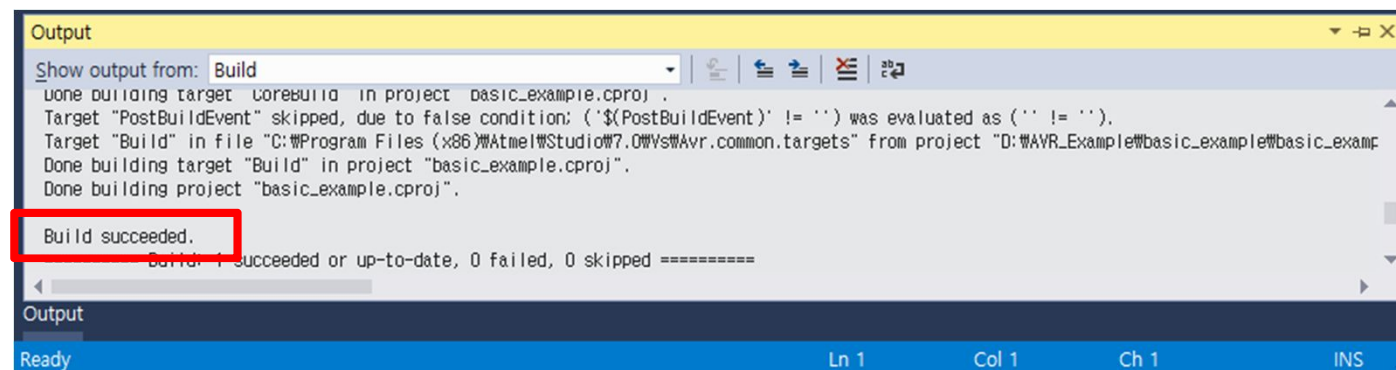
```
if(Switch_flag_pre == 0x01)           // SW0을 눌렀을때
    _delay_ms(250);                    // 250ms 시간지연
else if(Switch_flag_pre == 0x02)      // SW1을 눌렀을때
    _delay_ms(500);                    // 500ms 시간지연
else if(Switch_flag_pre == 0x04)      // SW2을 눌렀을때
    _delay_ms(750);                    // 750ms 시간지연
else if(Switch_flag_pre == 0x08)      // SW3을 눌렀을때
    _delay_ms(1000);                   // 1000ms 시간지연
}
```

5) 응용 : 스위치의 입력값에 따라 LED 점멸시간 변경하기

- Build 탭에서 "Build Solution" 을 클릭하여 컴파일을 실행

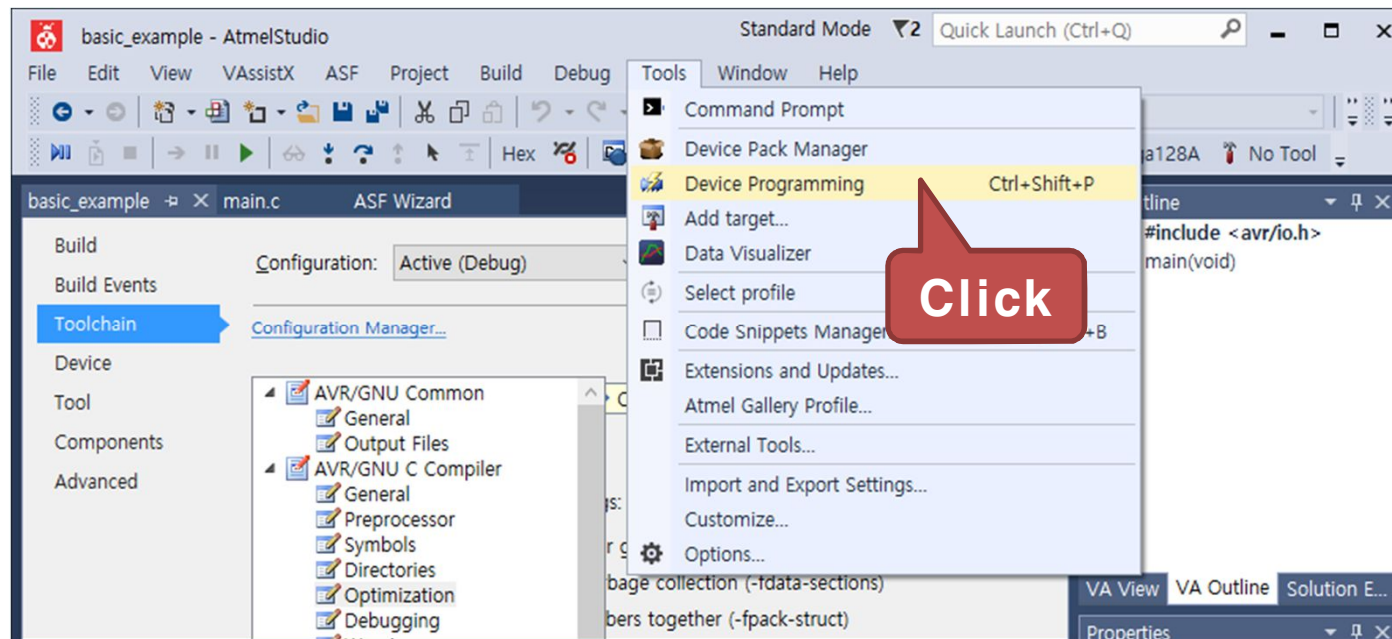


- 다음과 같이 Output 창에 "Build succeeded" 가 출력되면 컴파일이 완료



5) 응용 : 스위치의 입력값에 따라 LED 점멸시간 변경하기

- Tools 탭에서 "Device Programming" 을 클릭



5) 응용 : 스위치의 입력값에 따라 LED 점멸시간 변경하기

- Device Programming 창이 나타나면 Tool 에서 AVRISP mkII를 선택



- Device를 ATmega128A로 선택하고 "Apply" 버튼을 클릭

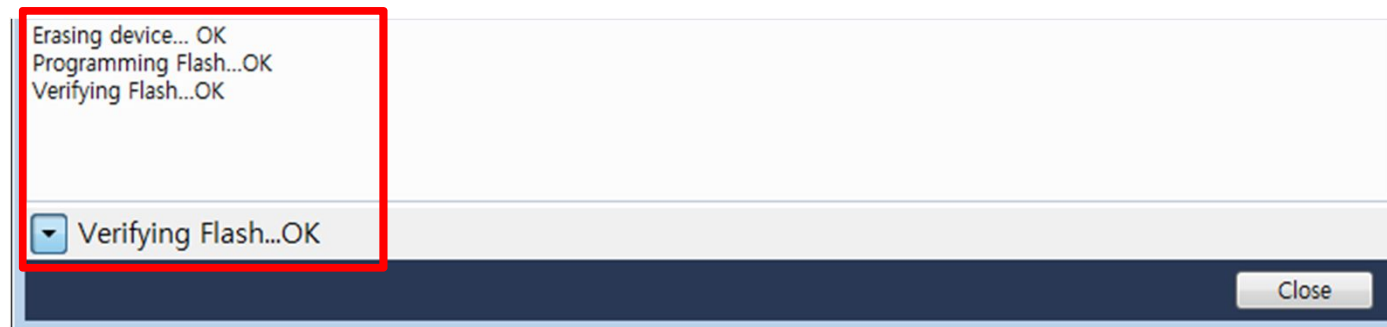


5) 응용 : 스위치의 입력값에 따라 LED 점멸시간 변경하기

- Device 인식이 완료되면 Memories 탭을 선택하고, "Program" 버튼을 클릭

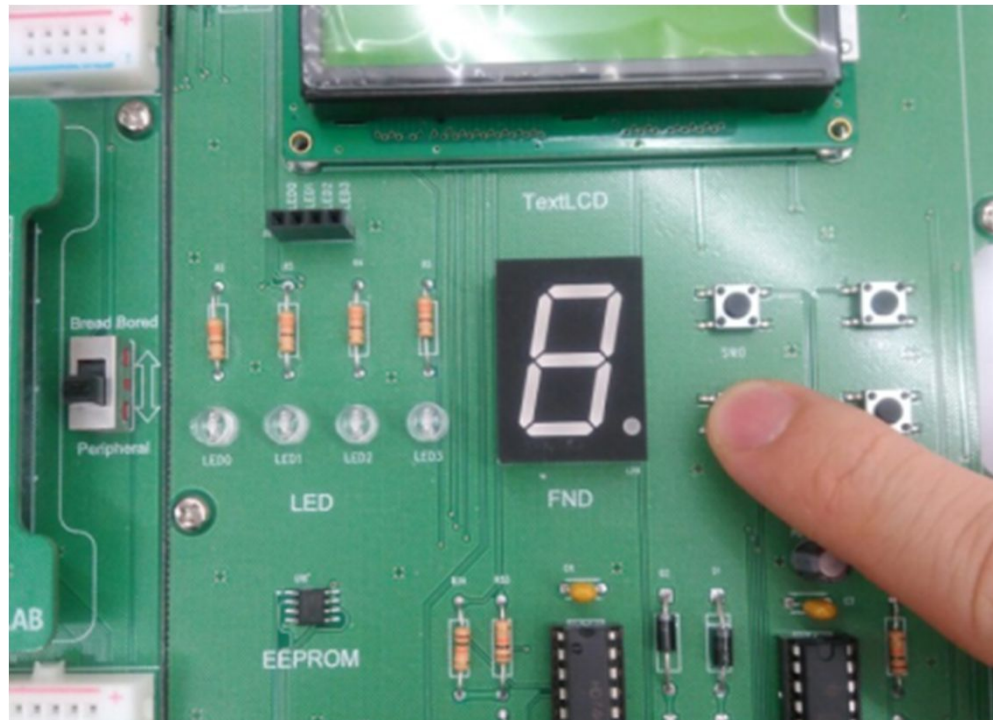


- 프로그램 다운로드가 성공하면 다음과 같은 메시지가 출력된다.



5) 응용 : 스위치의 입력값에 따라 LED 점멸시간 변경하기

- 실행 결과
 - 누른 스위치에 따라 LED On/Off 시간 변경

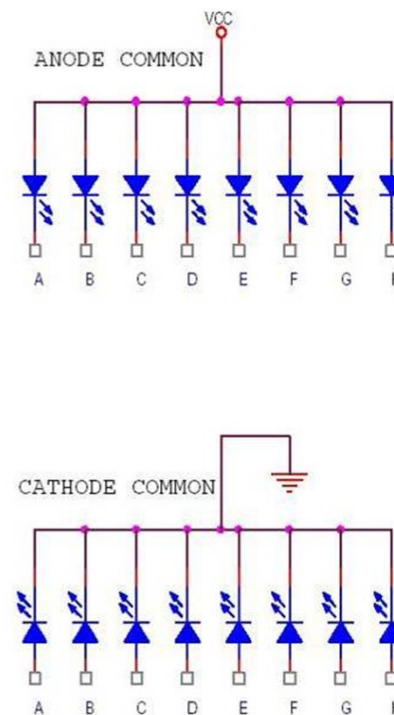
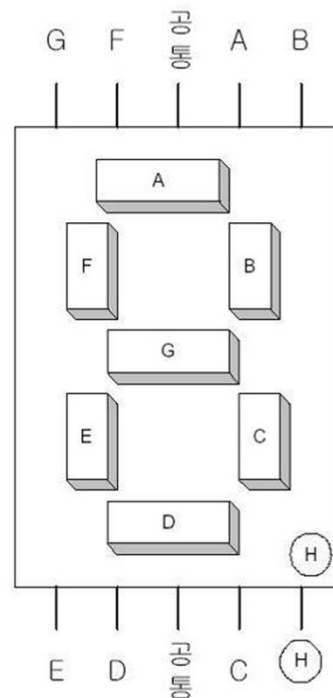


6)예제 : GPIO를 이용하여 FND LED 켜기

- 실습 개요
 - 단순 LED가 아닌 FND(Flexible Numeric Display: 7-Segment LED)를 이용하여 숫자를 표시하는 실습
 - 마이크로컨트롤러의 포트를 출력으로 선언하고, 이 포트를 Array FND 모듈의 신호선 포트에 연결함
 - 일정 시간마다 클럭에 의해 Array FND에 숫자와 문자가 디스플레이 되도록 함
- 실습 목표
 - GPIO 입출력 포트의 방향 제어 및 출력 제어 방법 습득
 - FND LED 동작원리 습득

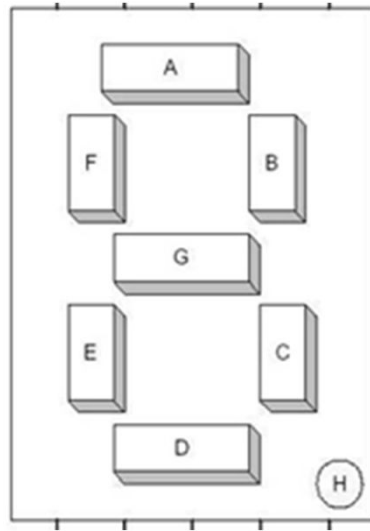
6)예제 : GPIO를 이용하여 FND LED 켜기

- FND(7-Segment LED) 구조
 - 7-세그먼트는 LED 8개를 그림과 같이 배열
 - 숫자나 간단한 기호 표현에 많이 사용됨
- 공통(Common)단자에 인가되는 전원에 따라서 Common Anode(+공통)과 Common Cathode(-공통)으로 분류



6)예제 : GPIO를 이용하여 FND LED 켜기

- FND(7-Segment LED) 구동방법
 - Common-Cathode방식 : 각 단자에 '1'이 입력되면 해당 LED가 켜짐

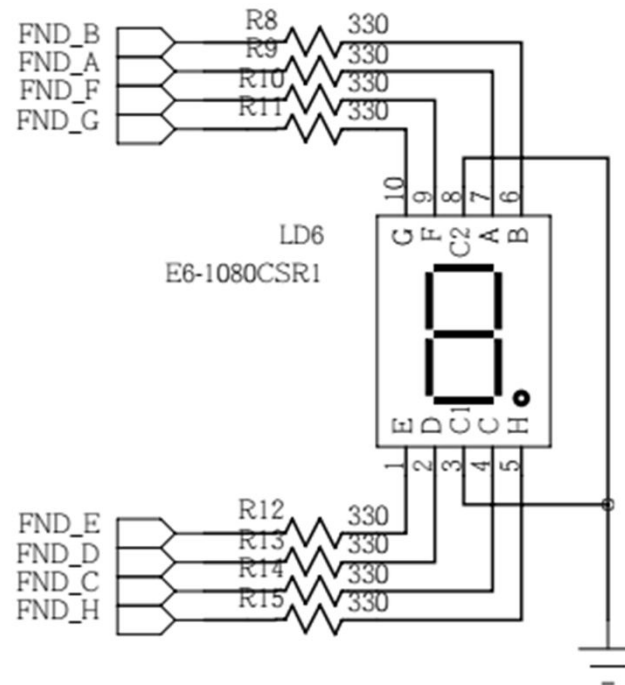


7-Segment
에서 16진수
표시방법

16 진수	7-세그먼트의 비트값								데이터 값 (HEX)
	H	G	F	E	D	C	B	A	
0	0	0	1	1	1	1	1	1	0X3F
1	0	0	0	0	0	1	1	0	0X06
2	0	1	0	1	1	0	1	1	0X5B
3	0	1	0	0	1	1	1	1	0X4F
4	0	1	1	0	0	1	1	0	0X66
5	0	1	1	0	1	1	0	1	0X6D
6	0	1	1	1	1	1	0	1	0X7D
7	0	0	1	0	0	1	1	1	0X27
8	0	1	1	1	1	1	1	1	0X7F
9	0	1	1	0	1	1	1	1	0X6F
A	0	1	1	1	0	1	1	1	0X77
B	0	1	1	1	1	1	0	0	0X7C
C	0	0	1	1	1	0	0	1	0X39
D	0	1	0	1	1	1	1	0	0X5E
E	0	1	1	1	1	0	0	1	0X79
F	0	1	1	1	0	0	0	1	0X71

6)예제 : GPIO를 이용하여 FND LED 켜기

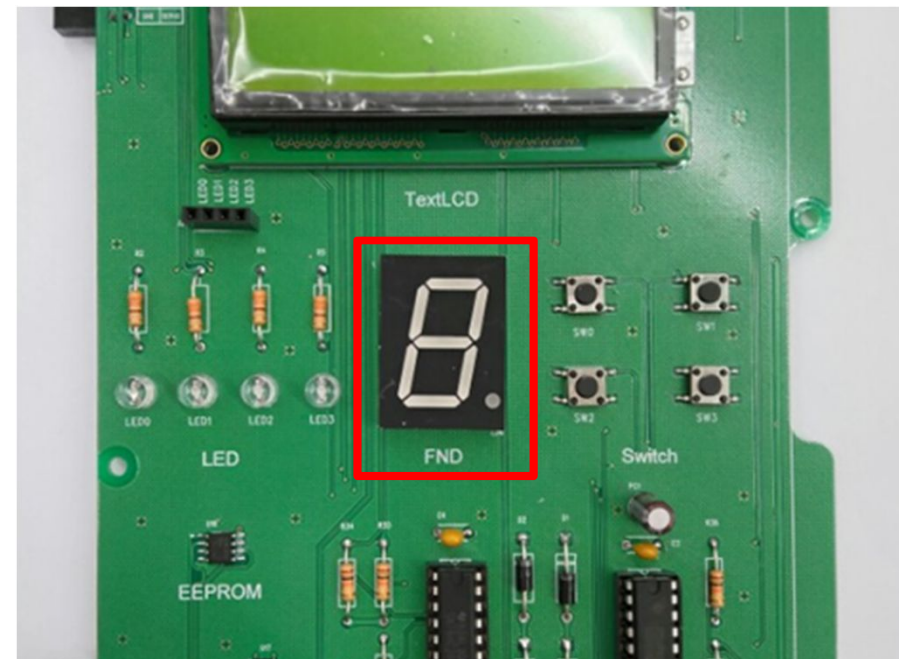
- 사용 모듈의 회로
 - FND 모듈(Common-Cathode)



6)예제 : GPIO를 이용하여 FND LED 켜기

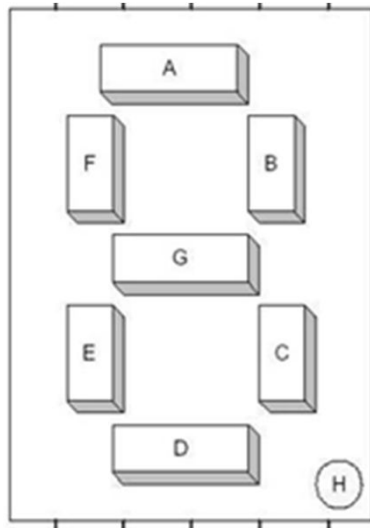
- Edge-MCU AVR
 - FND는 A 포트에 연결

Sensor	GPIO
DATA_A	PA0
DATA_B	PA1
DATA_C	PA2
DATA_D	PA3
DATA_E	PA4
DATA_F	PA5
DATA_G	PA6
DATA_H	PA7



6)예제 : GPIO를 이용하여 FND LED 켜기

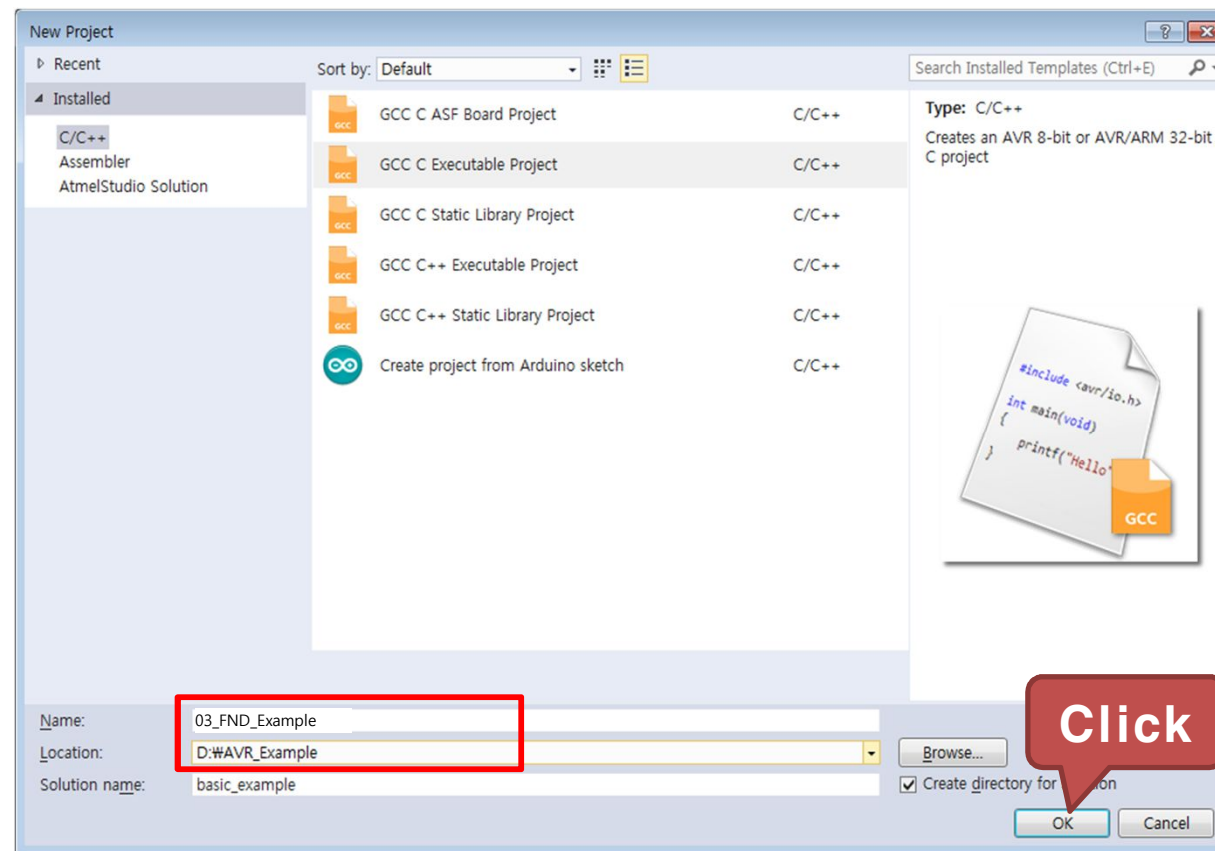
- 구동 프로그램 : 사전지식
 - MCU 모듈의 A포트를 FND의 불을 켜기 위한 출력 포트로 설정
 - DDRx 레지스터(여기서는 DDRA 레지스터)에 '1'로 설정
 - 표를 참조하여 PORTx(여기서는 PORTA 레지스터)에 '1'을 출력



16 진수	7-세그먼트의 비트값								데이터 값 (HEX)
	H	G	F	E	D	C	B	A	
0	0	0	1	1	1	1	1	1	0X3F
1	0	0	0	0	0	1	1	0	0X06
2	0	1	0	1	1	0	1	1	0X5B
3	0	1	0	0	1	1	1	1	0X4F
4	0	1	1	0	0	1	1	0	0X66
5	0	1	1	0	1	1	0	1	0X6D
6	0	1	1	1	1	1	0	1	0X7D
7	0	0	1	0	0	1	1	1	0X27
8	0	1	1	1	1	1	1	1	0X7F
9	0	1	1	0	1	1	1	1	0X6F
A	0	1	1	1	0	1	1	1	0X77
B	0	1	1	1	1	1	0	0	0X7C
C	0	0	1	1	1	0	0	1	0X39
D	0	1	0	1	1	1	1	0	0X5E
E	0	1	1	1	1	0	0	1	0X79
F	0	1	1	1	0	0	0	1	0X71

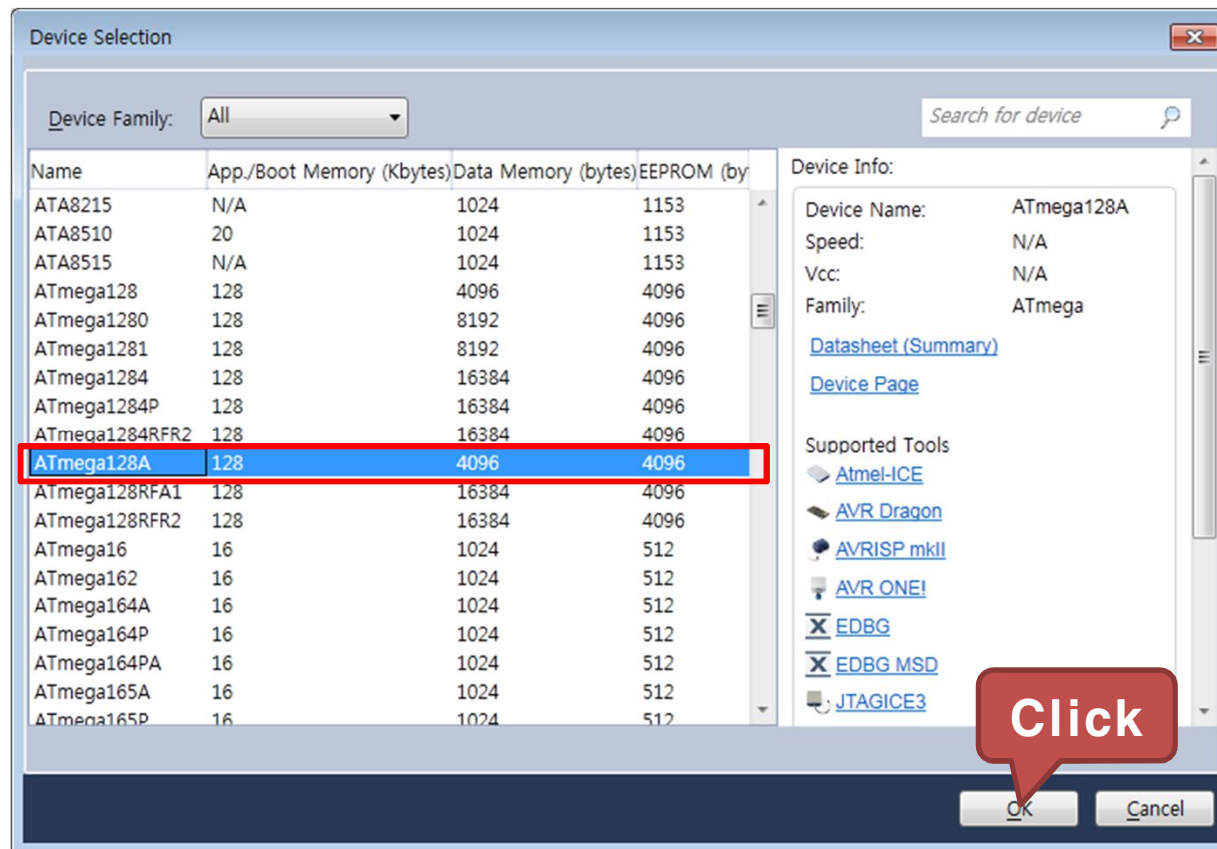
6)예제 : GPIO를 이용하여 FND LED 켜기

- GCC C Executable Project를 선택하고, 생성된 Project의 Name 과 Location을 설정
 - Name : 03_FND_Example
 - Location : D:\WAVR_Example



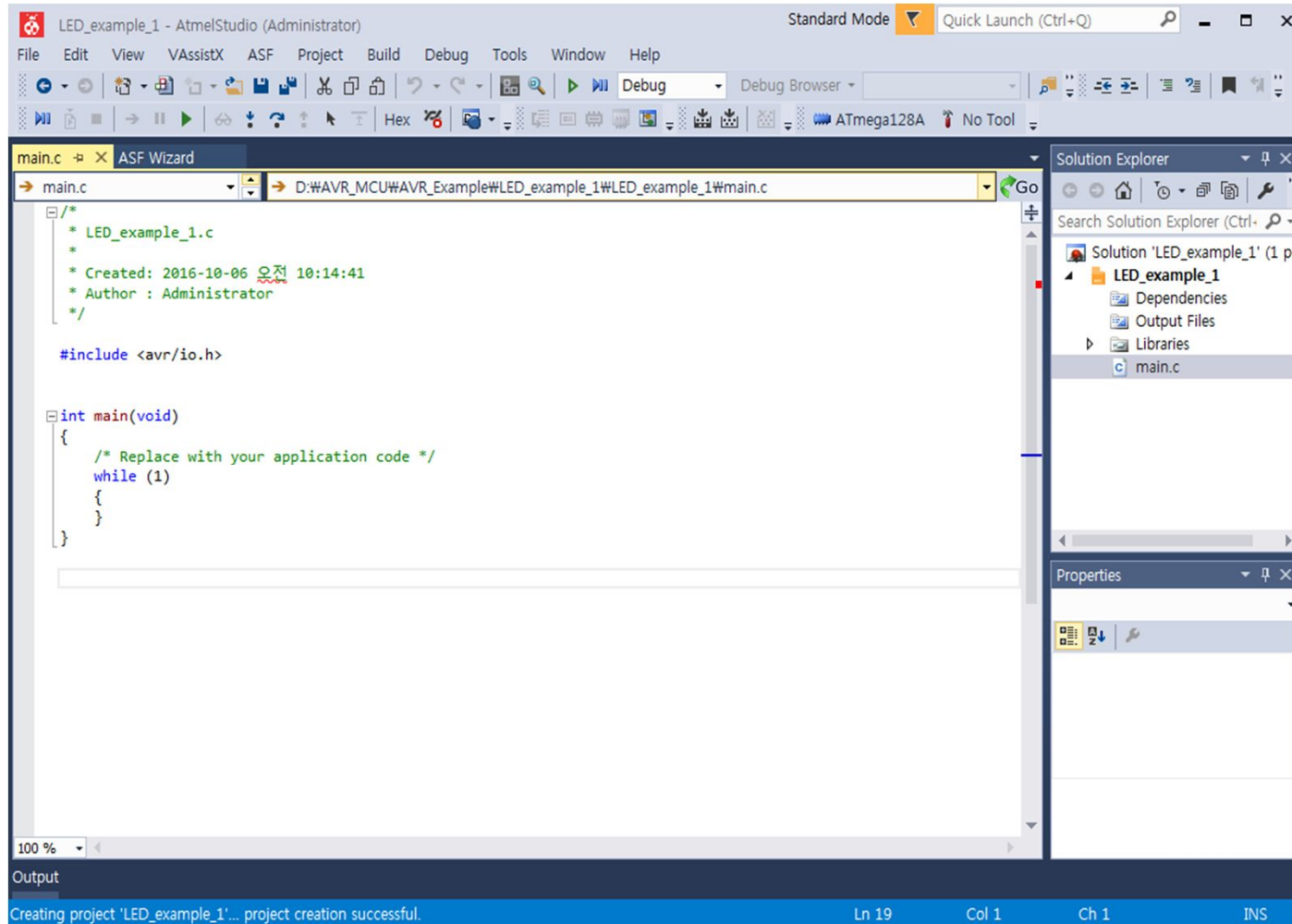
6)예제 : GPIO를 이용하여 FND LED 켜기

- ATmega128A를 선택하고 OK를 클릭



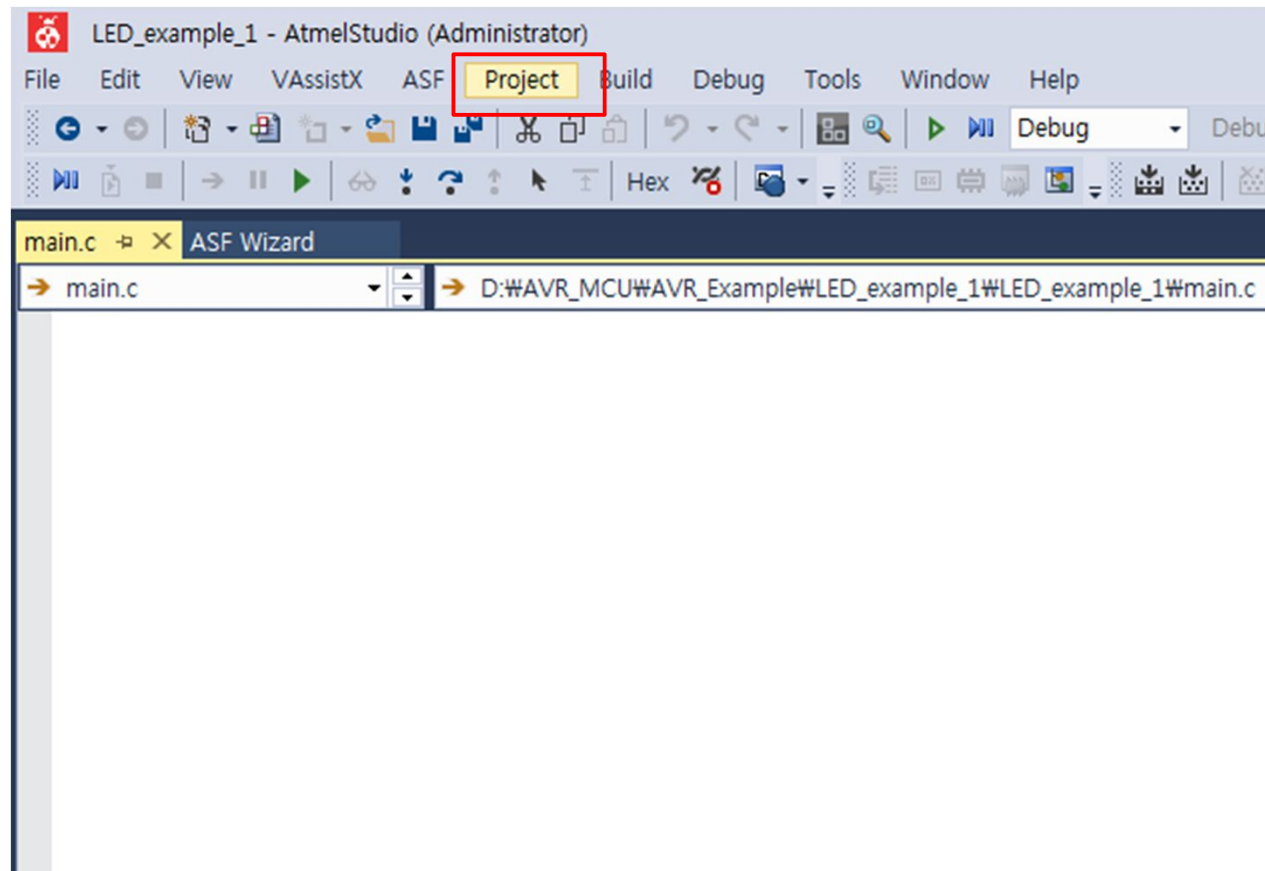
6)예제 : GPIO를 이용하여 FND LED 켜기

- 03_FND_Example 프로젝트 생성 완료



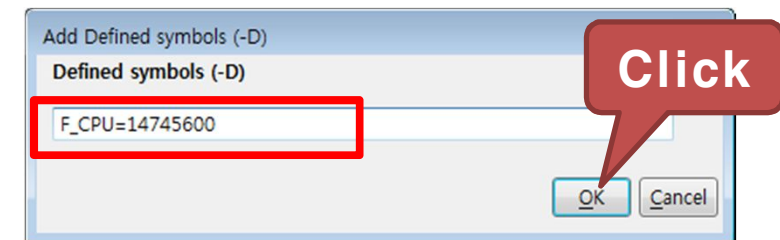
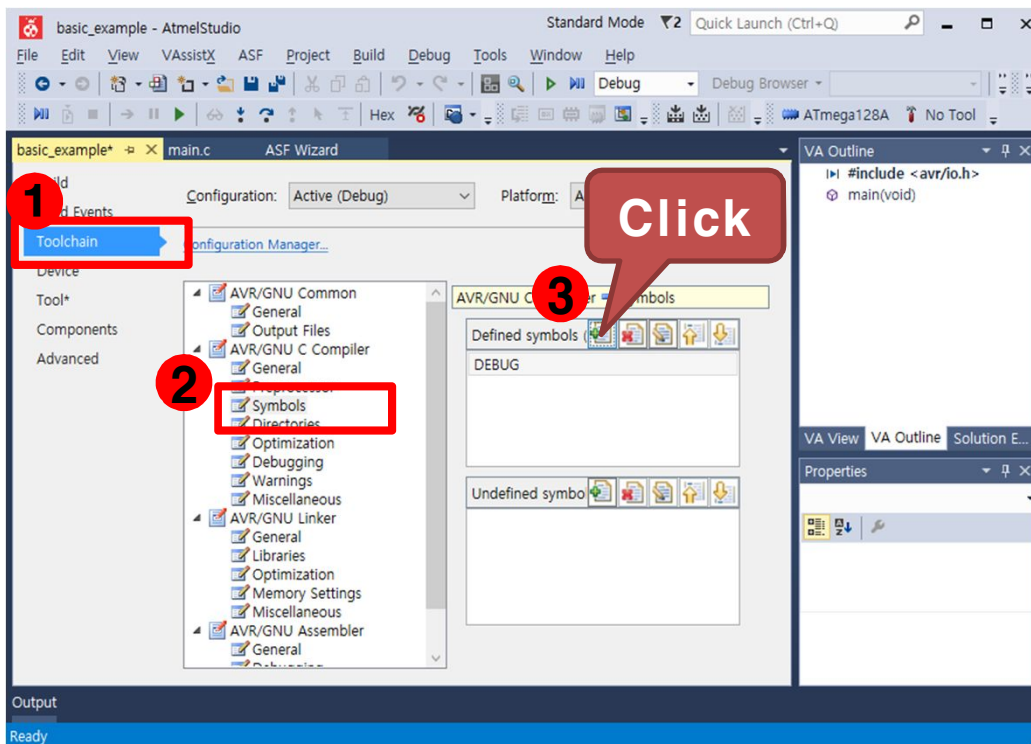
6)예제 : GPIO를 이용하여 FND LED 켜기

- 프로젝트 설정
 - Project 탭에서 "03_FND_Example Properties..." 를 선택



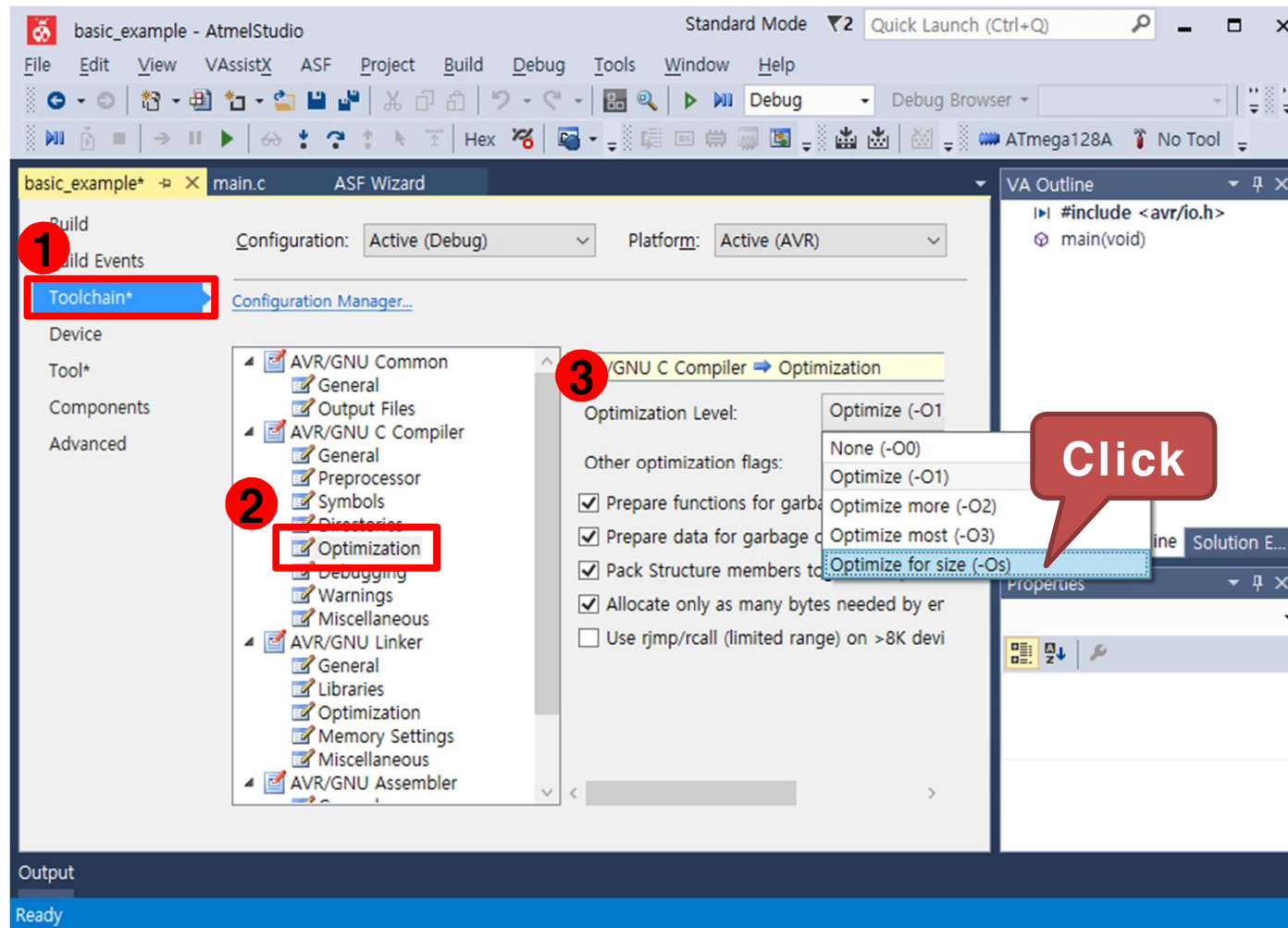
6)예제 : GPIO를 이용하여 FND LED 켜기

- Toolchain → AVR/GNU C Compiler → Symbols를 선택한 후 우측의 Defined symbols에서 "Add Item" 을 선택
- "F_CPU=14745600"을 입력하고 OK를 클릭. AVR의 외부 클럭 14.7456Mhz



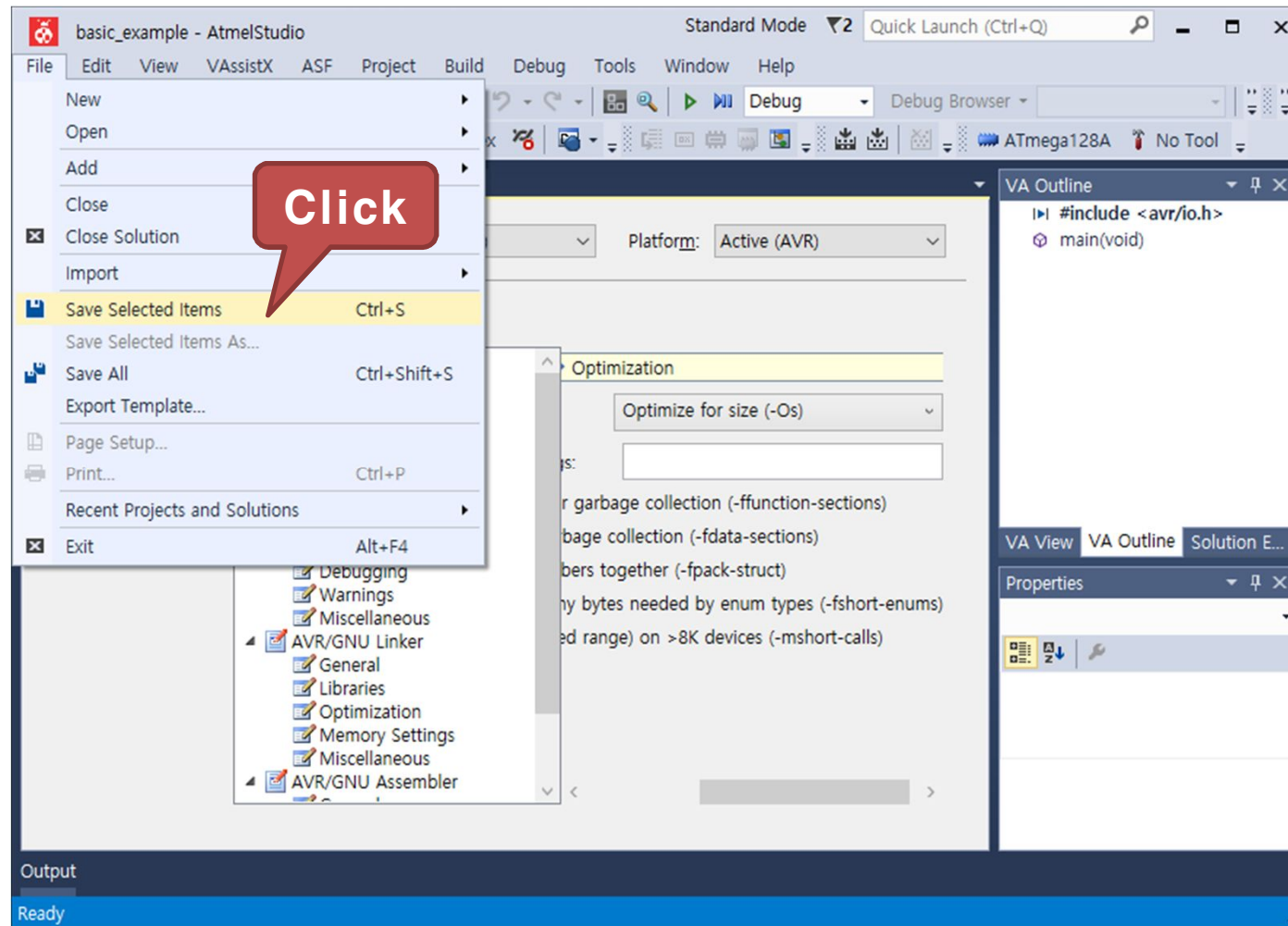
6)예제 : GPIO를 이용하여 FND LED 켜기

- Toolchain → AVR/GNU C Compiler → Optimization 을 선택한 후 우측의 Optimization Level 을 변경, "Optimize for size (-OS)" 선택



6)예제 : GPIO를 이용하여 FND LED 켜기

- File 탭에서 "Save Selected Items"를 클릭하여 설정값을 저장



6)예제 : GPIO를 이용하여 FND LED 켜기

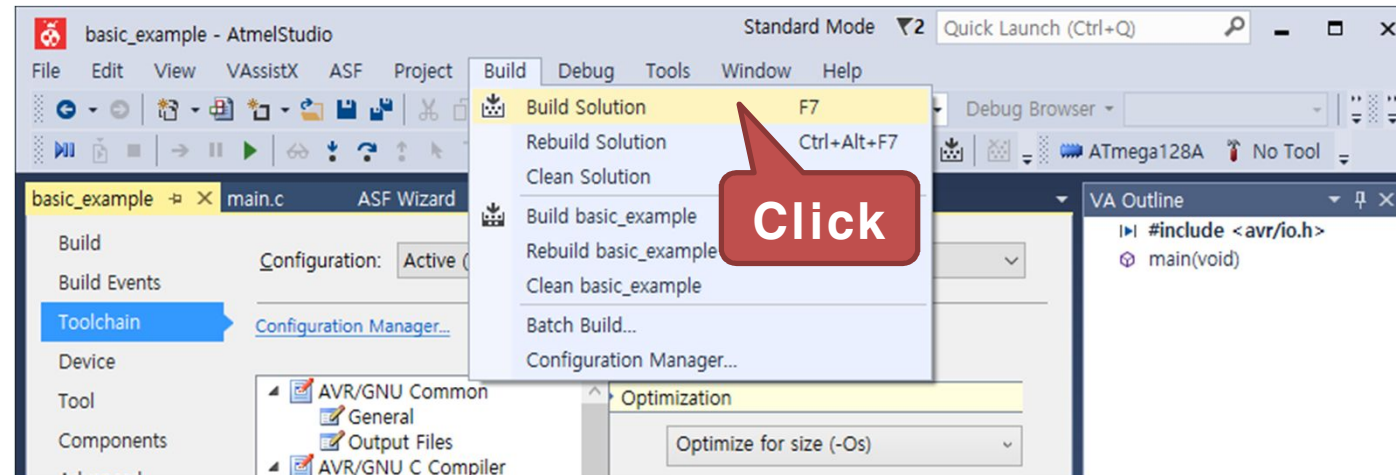
- 구동 프로그램
 - main.c 코드 작성

```
#include <avr/io.h>           // AVR 입출력에 대한 헤더 파일
#include <util/delay.h>       // delay 함수사용을 위한 헤더파일
int main(void) {
    // 7-Segment에 표시할 글자의 입력 데이터를 저장
    unsigned char FND_DATA_TBL []={0x3F,0X06,0X5B,0X4F,0X66,0X6D,
                                     0X7C,0X07,0X7F,0X67,0X77,0X7C,
                                     0X39,0X5E,0X79,0X71,0X08,0X80};

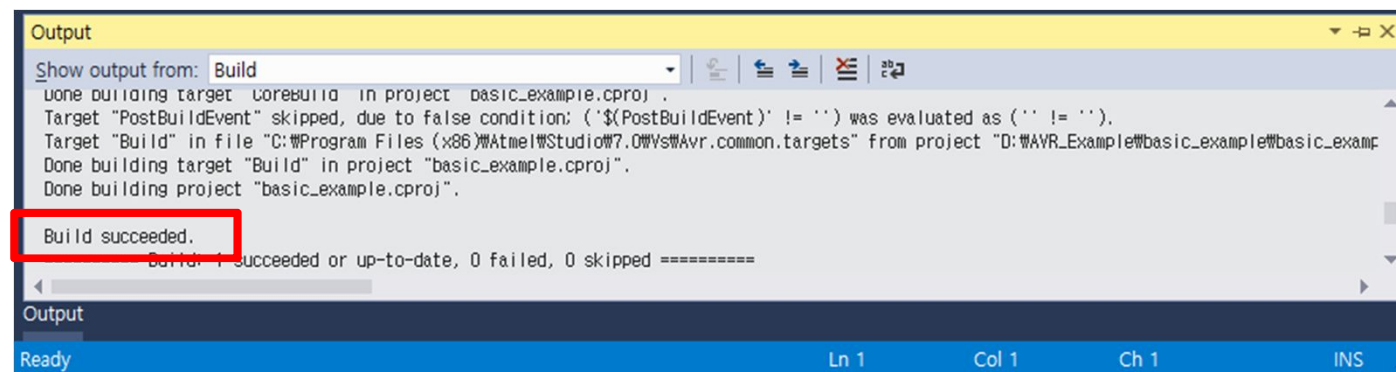
    unsigned char cnt=0;      // FND Table 카운터 변수
    DDRA = 0xFF;             // 포트A 를 출력포트로 설정
    while (1) {
        PORTA = FND_DATA_TBL[cnt]; // 포트 A에 FND Table 값을 출력
        cnt++;                  // FND Table 카운터 변수를 증가
        if(cnt>17) cnt=0;      // 테이블 크기를 초과하는 경우 방지
        _delay_ms(500);        // 500ms 시간 지연
    }
}
```

6)예제 : GPIO를 이용하여 FND LED 켜기

- Build 탭에서 "Build Solution" 을 클릭하여 컴파일을 실행

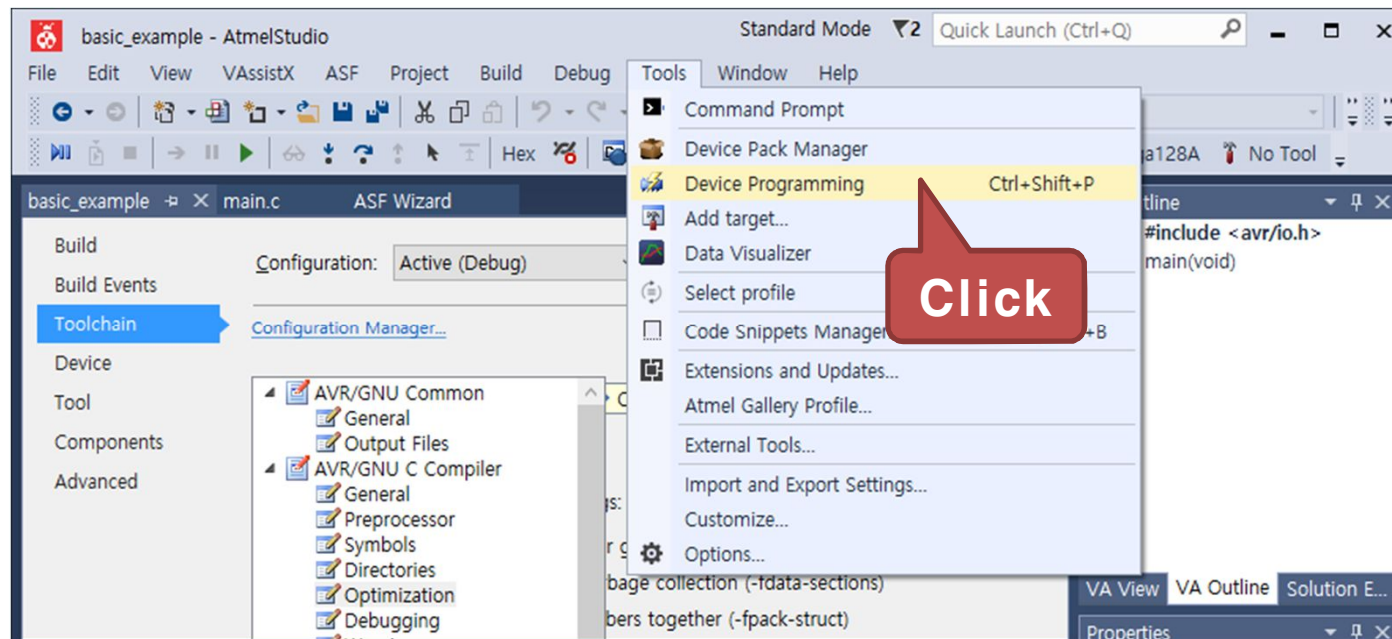


- 다음과 같이 Output 창에 "Build succeeded" 가 출력되면 컴파일이 완료



6)예제 : GPIO를 이용하여 FND LED 켜기

- Tools 탭에서 "Device Programming" 을 클릭



6)예제 : GPIO를 이용하여 FND LED 켜기

- Device Programming 창이 나타나면 Tool 에서 AVRISP mkII를 선택



- Device를 ATmega128A로 선택하고 "Apply" 버튼을 클릭

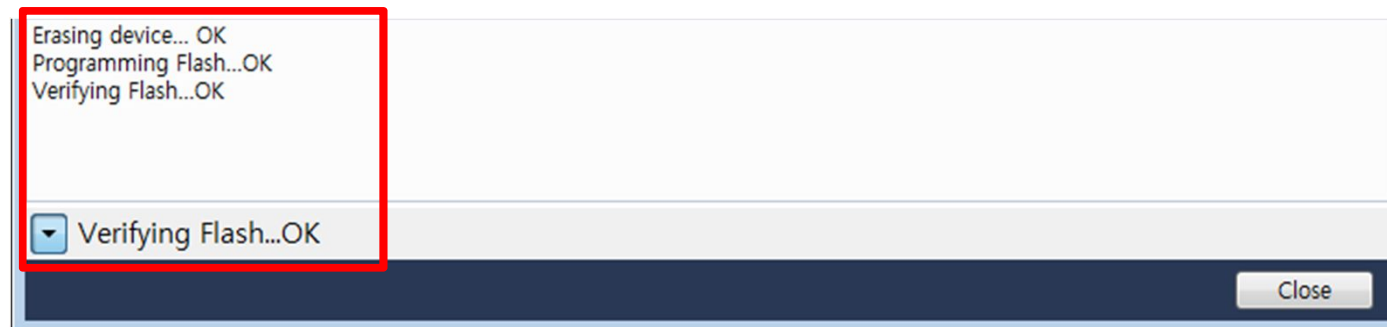


6)예제 : GPIO를 이용하여 FND LED 켜기

- Device 인식이 완료되면 Memories 탭을 선택하고, "Program" 버튼을 클릭

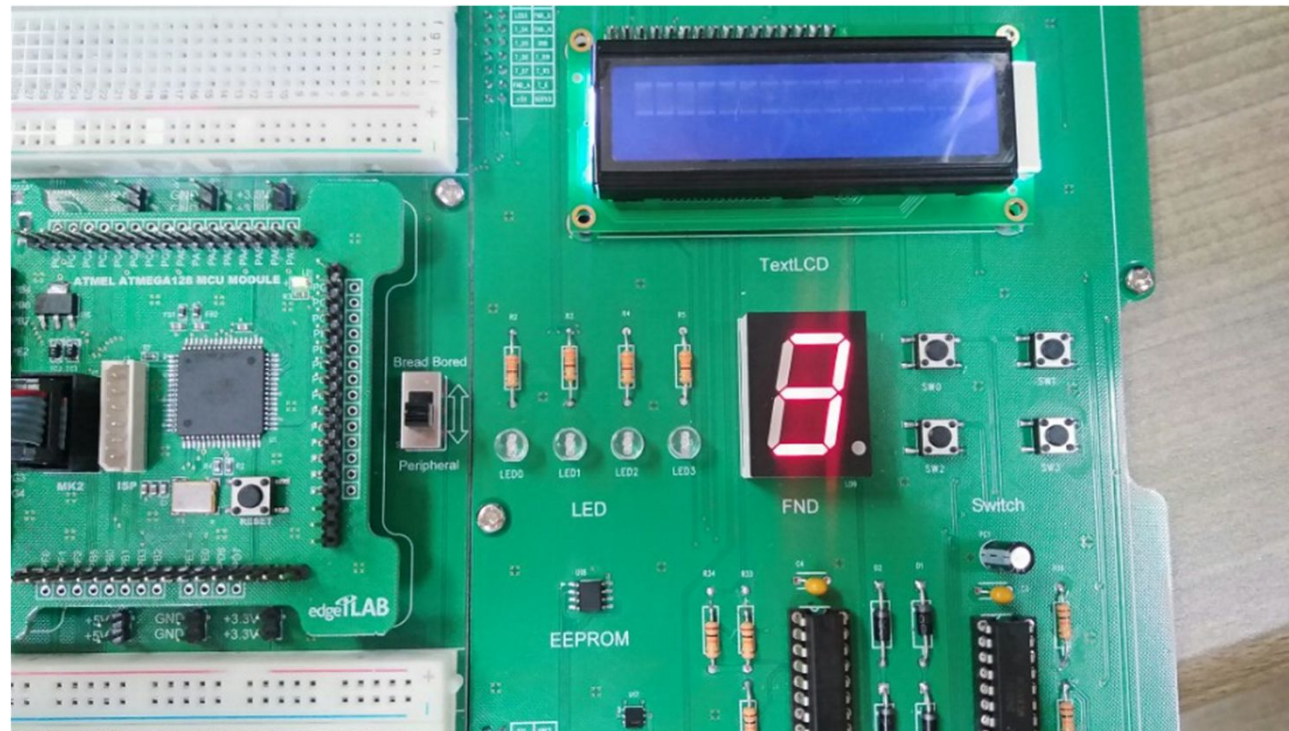


- 프로그램 다운로드가 성공하면 다음과 같은 메시지가 출력된다.



6)예제 : GPIO를 이용하여 FND LED 켜기

- 실행 결과
 - 프로그램이 시작하면 500ms 마다 FND 에 0부터 9 , A ~ F 그리고 ' _ ' , ' ! ' 을 순차적으로 출력

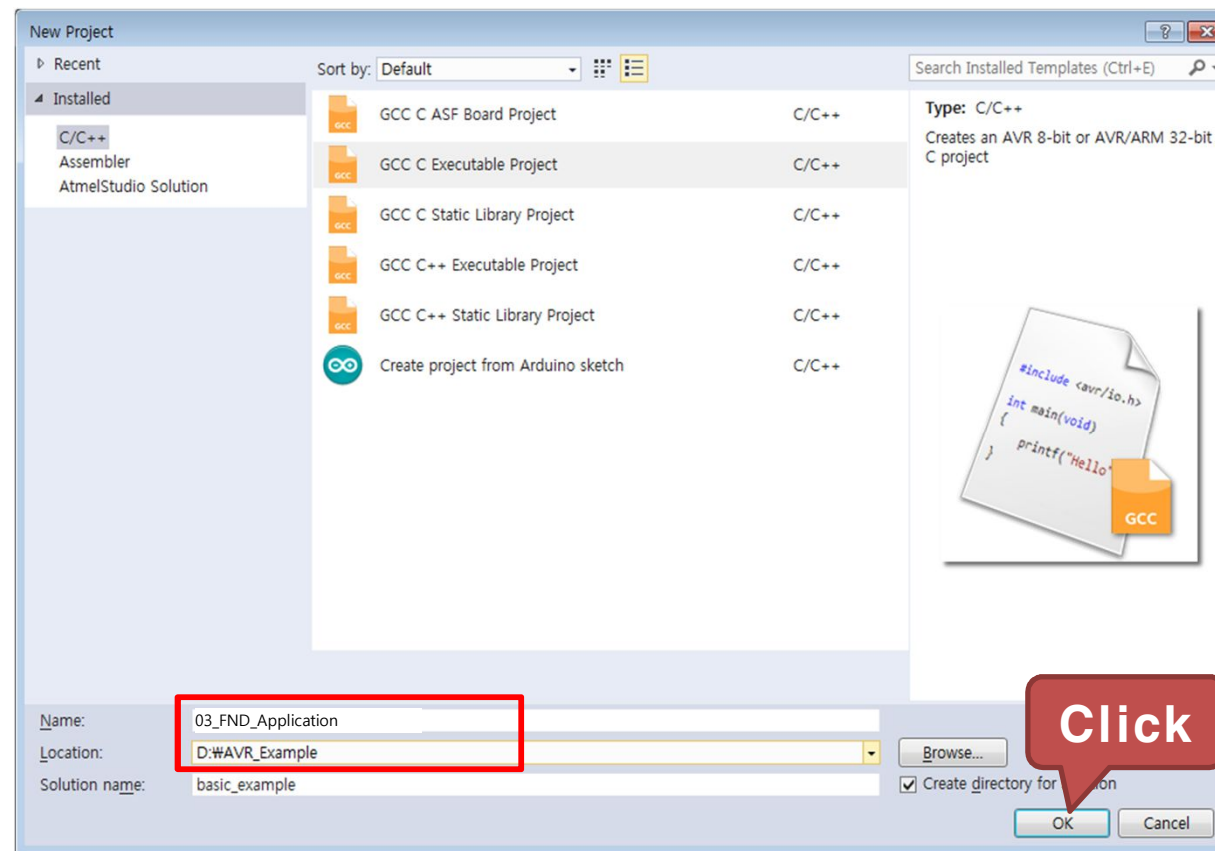


7) 응용 : 스위치의 입력값에 따라 FND LED 값 변경하기

- 실습 개요
 - 스위치의 입력값에 따라 FND에 출력되는 숫자를 +1, +2, -1, -2씩 더하거나 빼보도록 함
 - 마이크로컨트롤러의 포트를 출력으로 선언하고, 이 포트를 Array FND 모듈의 신호선 포트에 연결함
- 실습 목표
 - GPIO 입출력 포트의 방향 제어 및 출력 제어 방법 습득
 - FND LED 동작원리 습득

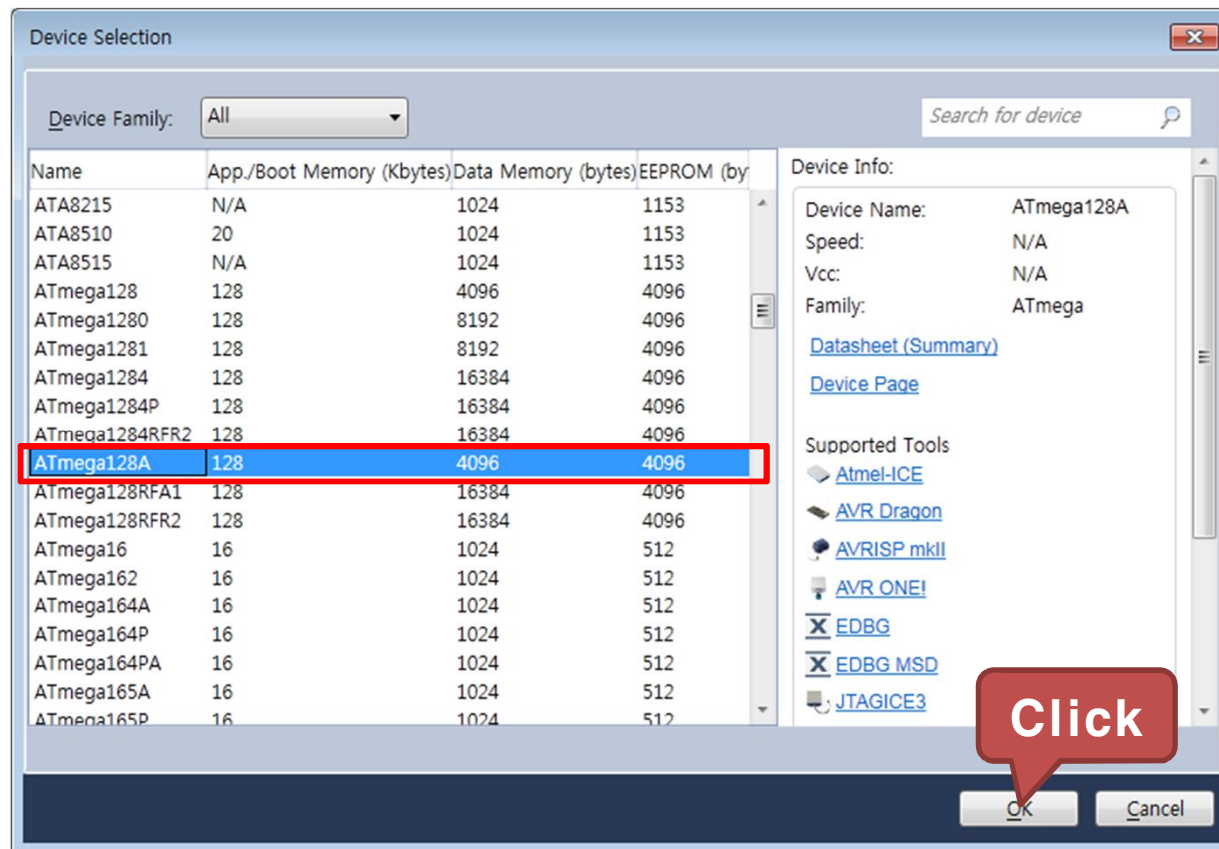
7) 응용 : 스위치의 입력값에 따라 FND LED 값 변경하기

- GCC C Executable Project를 선택하고, 생성된 Project의 Name 과 Location을 설정
 - Name : 03_FND_Application
 - Location : D:\WAVR_Example



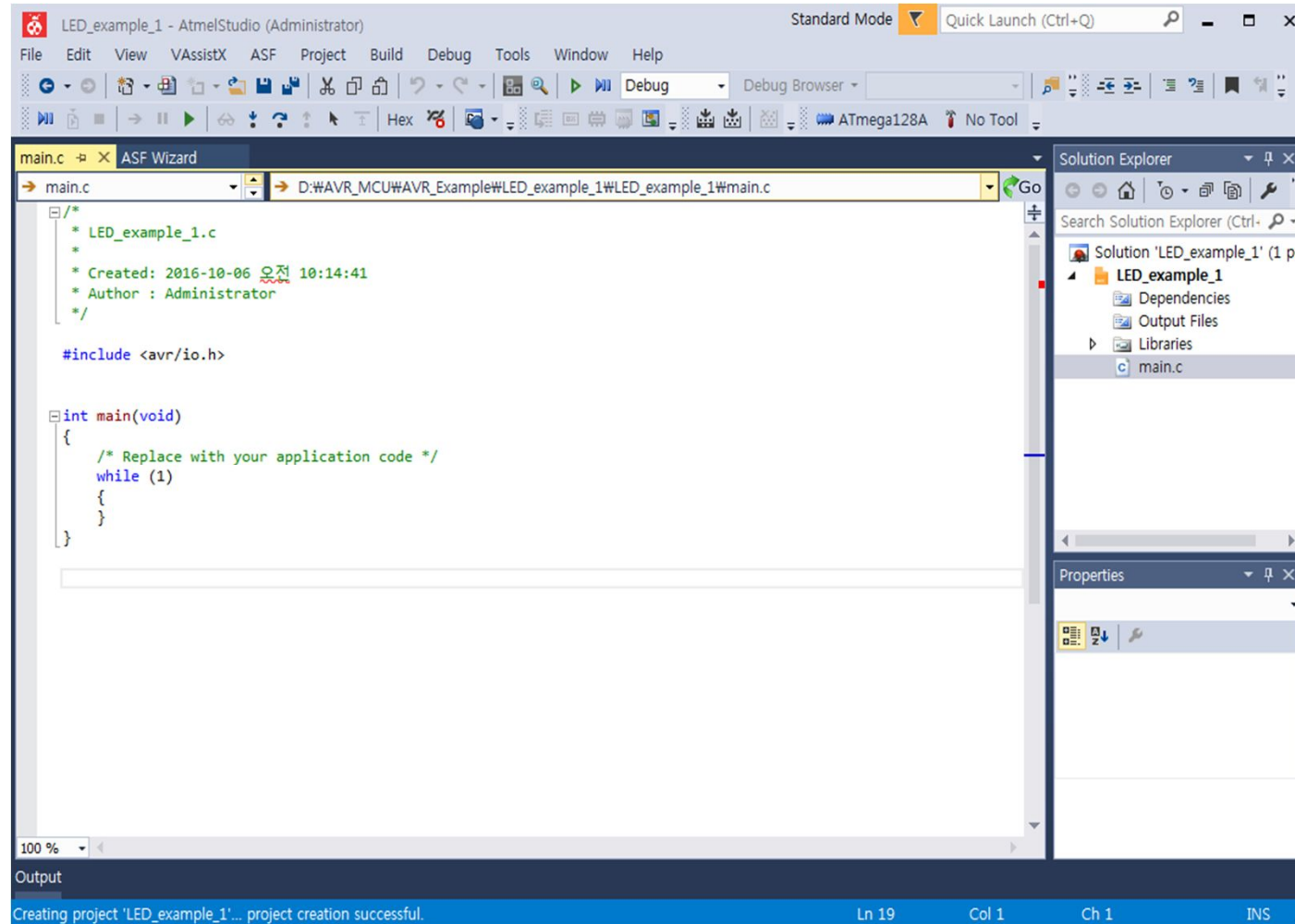
7) 응용 : 스위치의 입력값에 따라 FND LED 값 변경하기

- ATmega128A를 선택하고 OK를 클릭



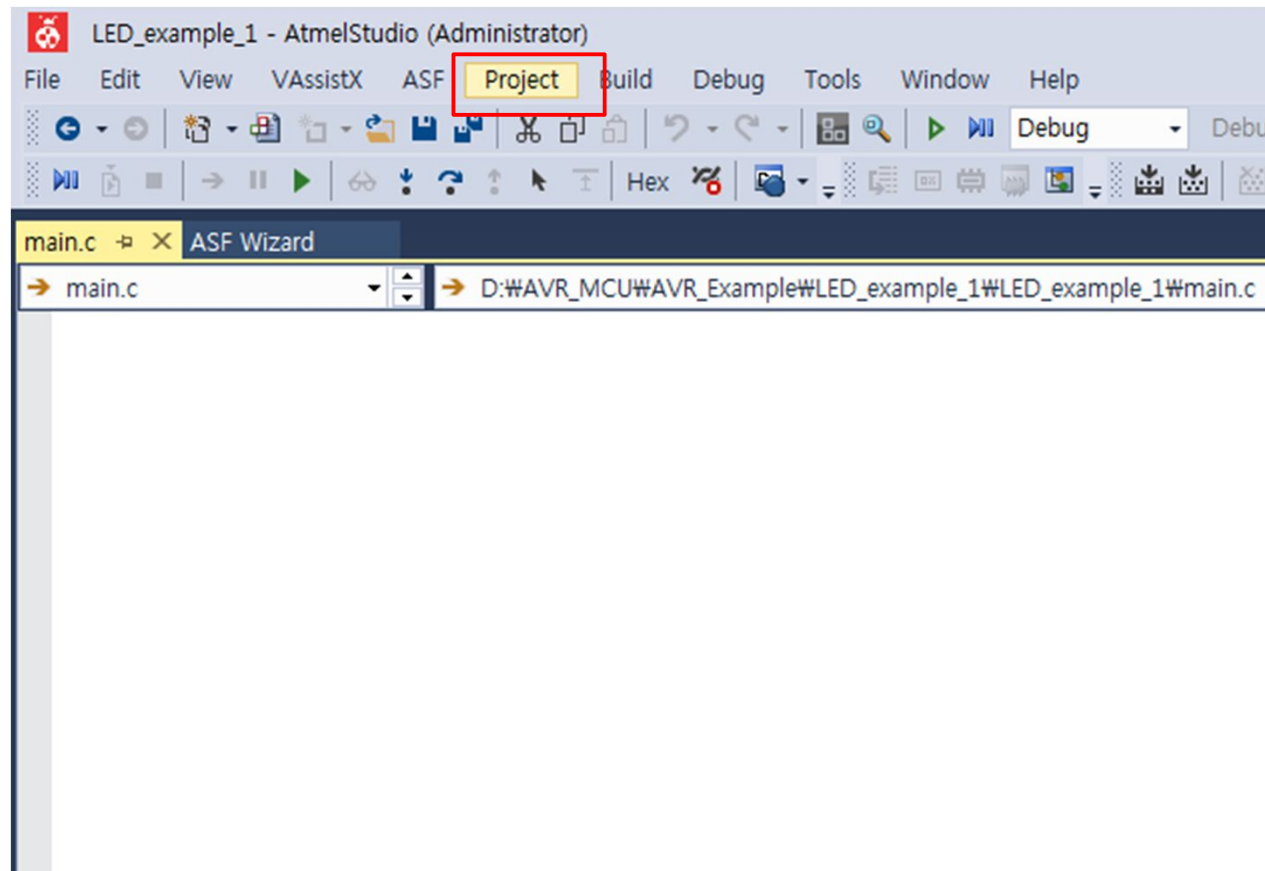
7) 응용 : 스위치의 입력값에 따라 FND LED 값 변경하기

- 03_FND_Application 프로젝트 생성 완료



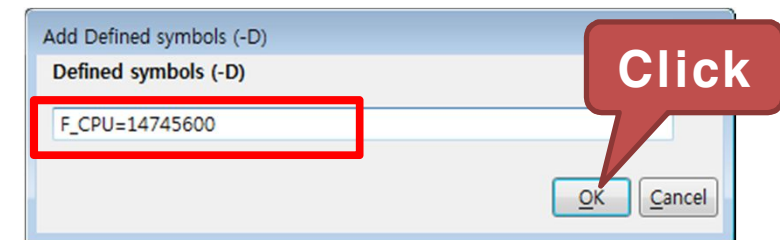
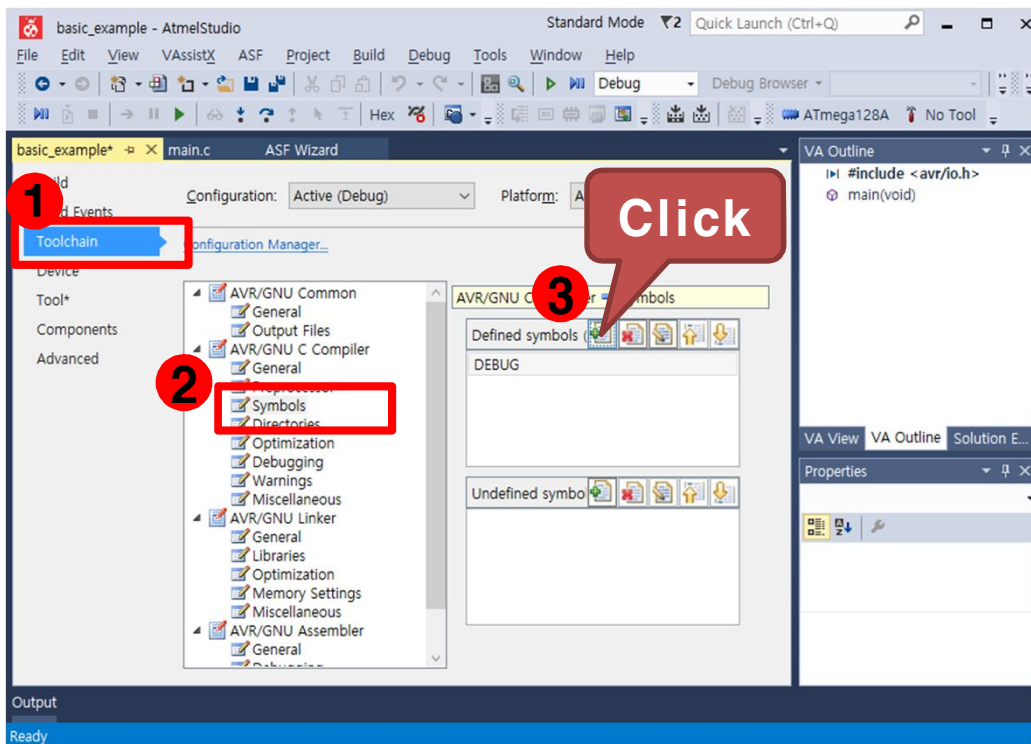
7) 응용 : 스위치의 입력값에 따라 FND LED 값 변경하기

- 프로젝트 설정
 - Project 탭에서 "03_FND_Application Properties..." 를 선택



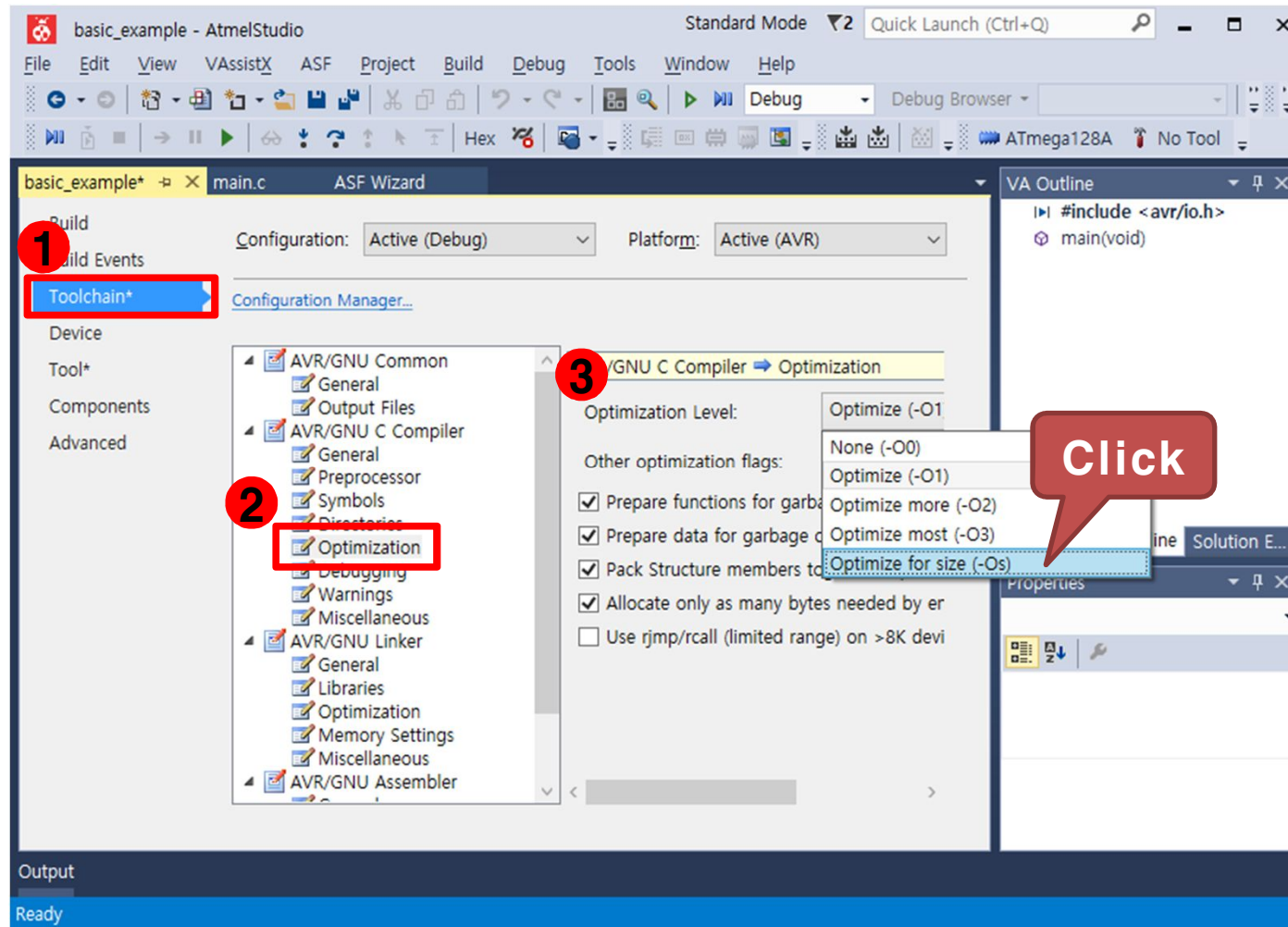
7) 응용 : 스위치의 입력값에 따라 FND LED 값 변경하기

- Toolchain → AVR/GNU C Compiler → Symbols를 선택한 후 우측의 Defined symbols에서 "Add Item" 을 선택
- "F_CPU=14745600"을 입력하고 OK를 클릭. AVR의 외부 클럭 14.7456Mhz



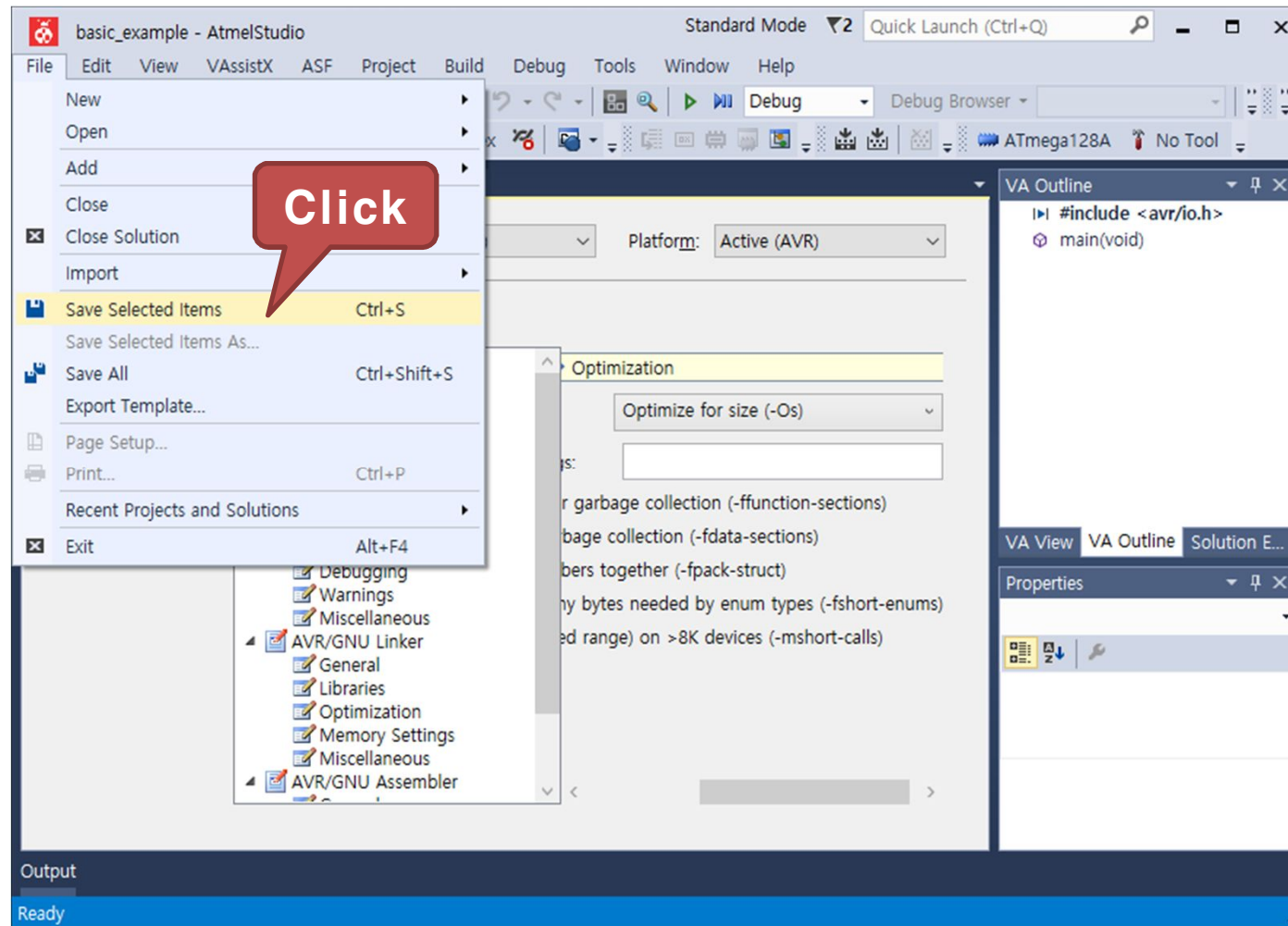
7) 응용 : 스위치의 입력값에 따라 FND LED 값 변경하기

- Toolchain → AVR/GNU C Compiler → Optimization 을 선택한 후 우측의 Optimization Level 을 변경, "Optimize for size (-Os)" 선택



7) 응용 : 스위치의 입력값에 따라 FND LED 값 변경하기

- File 탭에서 "Save Selected Items"를 클릭하여 설정값을 저장



7) 응용 : 스위치의 입력값에 따라 FND LED 값 변경하기

- 구동 프로그램
 - main.c 코드 작성

```
#include <avr/io.h>
#include <util/delay.h> // delay 함수사용을 위한 헤더파일

int Count_TR(unsigned char flag);

int main(void) {
    // 7-Segment에 표시할 글자의 입력 데이터를 저장
    unsigned char FND_DATA_TBL []={0x3F,0X06,0X5B,0X4F,0X66,0X6D,
                                     0X7C,0X07,0X7F,0X67,0X77,0X7C,
                                     0X39,0X5E,0X79,0X71,0X08,0X80};

    unsigned int cnt=0;           // FND Table 카운터 변수
    unsigned char Switch_flag = 0; // 스위치 값 저장 변수

    DDRA = 0xFF; // 포트A 를 출력포트로 설정
    DDRE = 0x00; // 포트E 를 입력포트로 설정
```

7) 응용 : 스위치의 입력값에 따라 FND LED 값 변경하기

- 구동 프로그램
 - main.c 코드 작성

```
while (1) {  
    // 입력핀이 포트 E의 상위 비트이므로 우측으로 4비트 쉬프트  
    Switch_flag = PINE >> 4;  
    while(PINE >> 4 != 0x00); // 스위치를 눌렀을 경우 땔 때까지 대기  
  
    if(Switch_flag != 0) // 스위치가 눌렀을 경우에 만 실행  
        // 스위치 톤에 따라 변수 cnt 값을 증가 또는 감소  
        cnt += Count_TR(Switch_flag);  
  
    // 변수 cnt 값의 범위를 0 ~ 15로 설정  
    if(cnt < 0)  
        cnt = 0;  
    else if(cnt > 15)  
        cnt = 15;  
  
    PORTA = FND_DATA_TBL[cnt]; // 포트 A에 FND Table 값을 출력  
    _delay_ms(100); // 100ms 시간 지연  
}
```

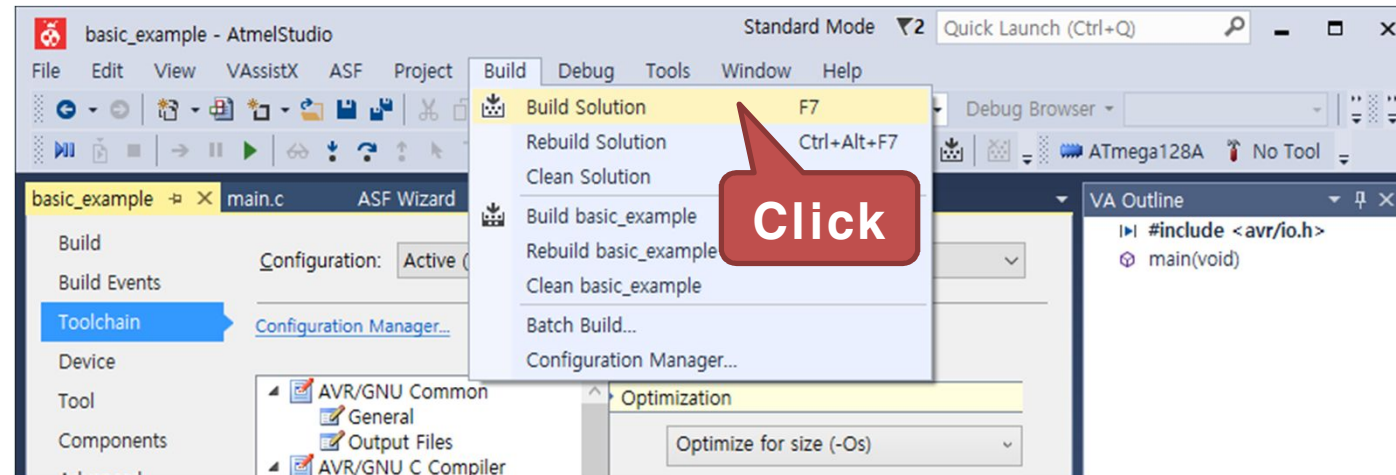
7) 응용 : 스위치의 입력값에 따라 FND LED 값 변경하기

- 구동 프로그램
 - main.c 코드 작성

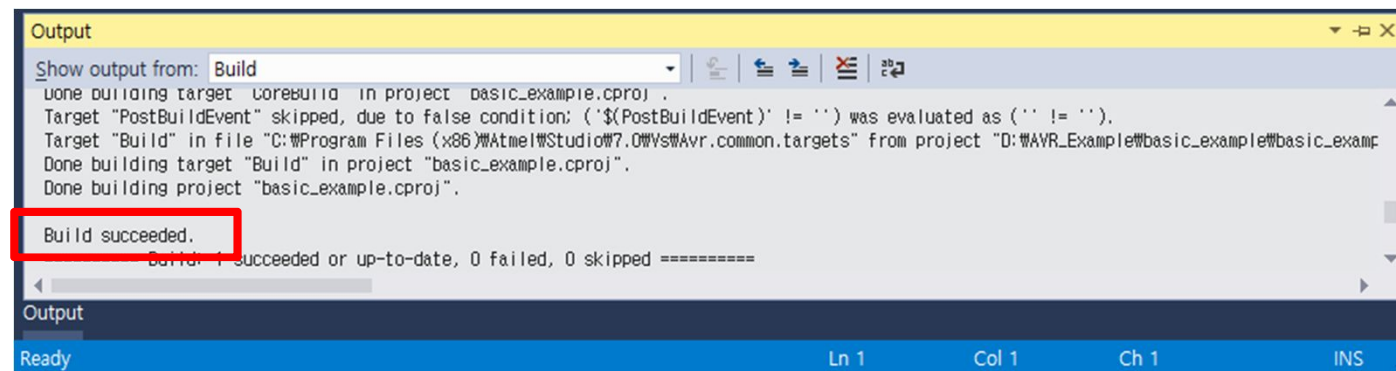
```
// 스위치 누른 상태에 따라 증감 감소를 값을 반환하는 함수
int Count_TR(unsigned char flag) {
    int count = 0;
    switch (flag) {
        case 0x01:          // SW0을 눌렀을때
            count = 1;
            break;
        case 0x02:          // SW1을 눌렀을때
            count = 2;
            break;
        case 0x04:          // SW2을 눌렀을때
            count = -1;
            break;
        case 0x08:          // SW3을 눌렀을때
            count = -2;
            break;
    }
    return count;
}
```

7) 응용 : 스위치의 입력값에 따라 FND LED 값 변경하기

- Build 탭에서 "Build Solution" 을 클릭하여 컴파일을 실행

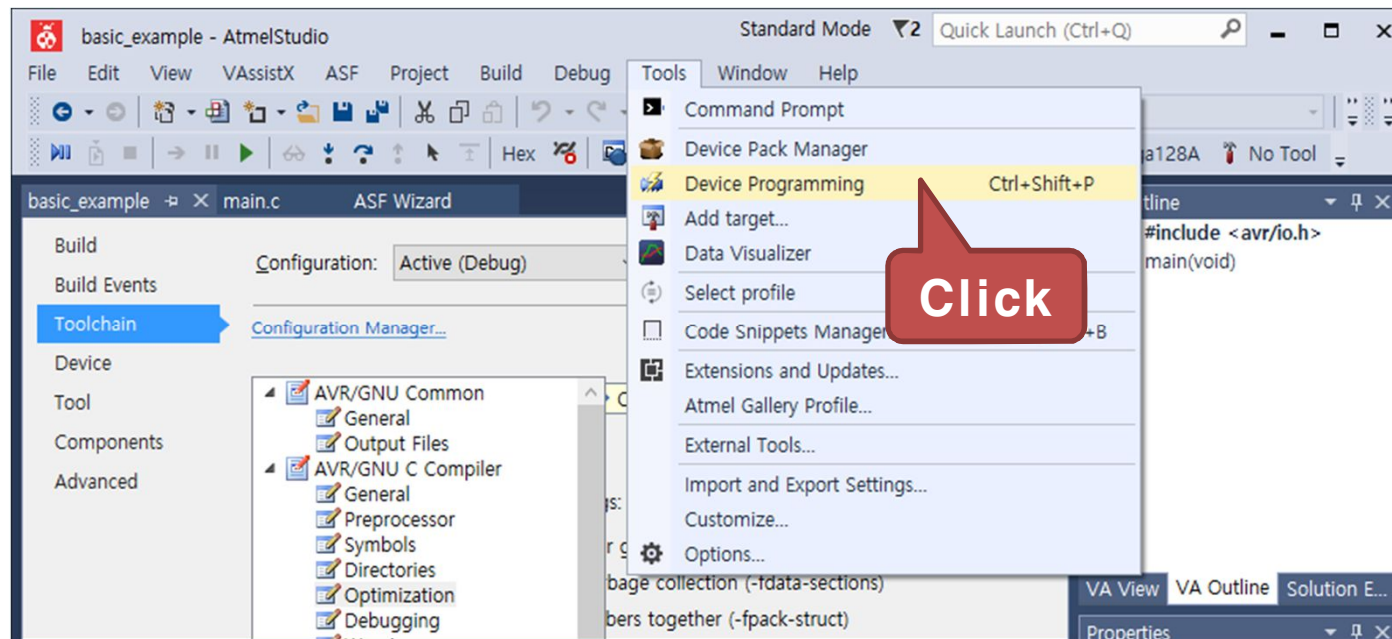


- 다음과 같이 Output 창에 "Build succeeded" 가 출력되면 컴파일이 완료



7) 응용 : 스위치의 입력값에 따라 FND LED 값 변경하기

- Tools 탭에서 "Device Programming" 을 클릭



7) 응용 : 스위치의 입력값에 따라 FND LED 값 변경하기

- Device Programming 창이 나타나면 Tool 에서 AVRISP mkII를 선택



- Device를 ATmega128A로 선택하고 "Apply" 버튼을 클릭

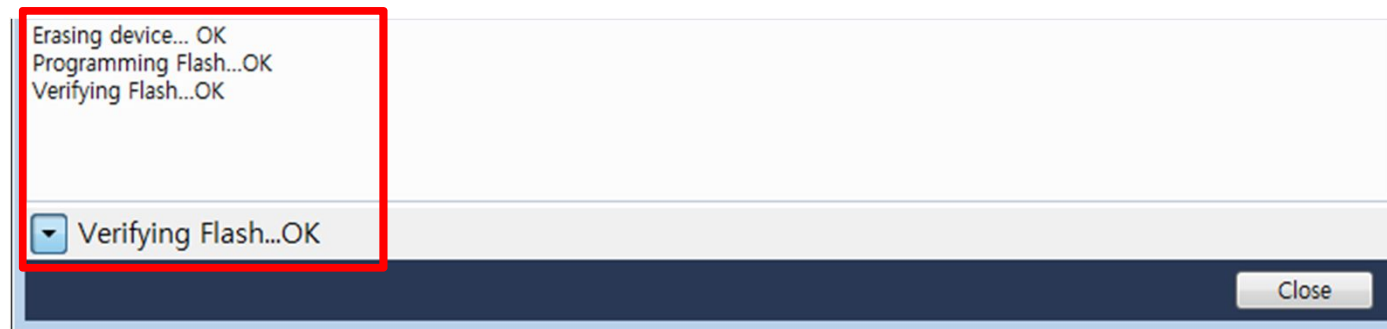


7) 응용 : 스위치의 입력값에 따라 FND LED 값 변경하기

- Device 인식이 완료되면 Memories 탭을 선택하고, "Program" 버튼을 클릭



- 프로그램 다운로드가 성공하면 다음과 같은 메시지가 출력된다.



7) 응용 : 스위치의 입력값에 따라 FND LED 값 변경하기

- 실행 결과
 - 스위치 입력값(SW0 : +1, SW1 : +2, SW2 : -1, SW3 : -2)에 따라 증감소하여 FND에 숫자를 표기

